Unit-1

Introduction to Python: Rapid Introduction to Procedural Programming, Data Types: Identifiers and Keywords, Integral Types, Floating Point Types
Strings: Strings, Comparing Strings, Slicing and Striding Strings, String Operators and Methods, String formatting with str.format
Collections Data Types: Tuples, Lists, Sets, dictionaries, Iterating and copying collections

---

Introduction to Procedural programming paradigms

In Procedure Oriented programming paradigms, series of computational steps are divided modules which means that the code is grouped in functions and the code is serially executed step by step so basically, it combines the serial code to instruct a computer with each step to perform a certain task. This paradigm helps in the modularity of code and modularization is usually done by the functional implementation. This programming paradigm helps in an easy organization related items without difficulty and so each file acts as a container

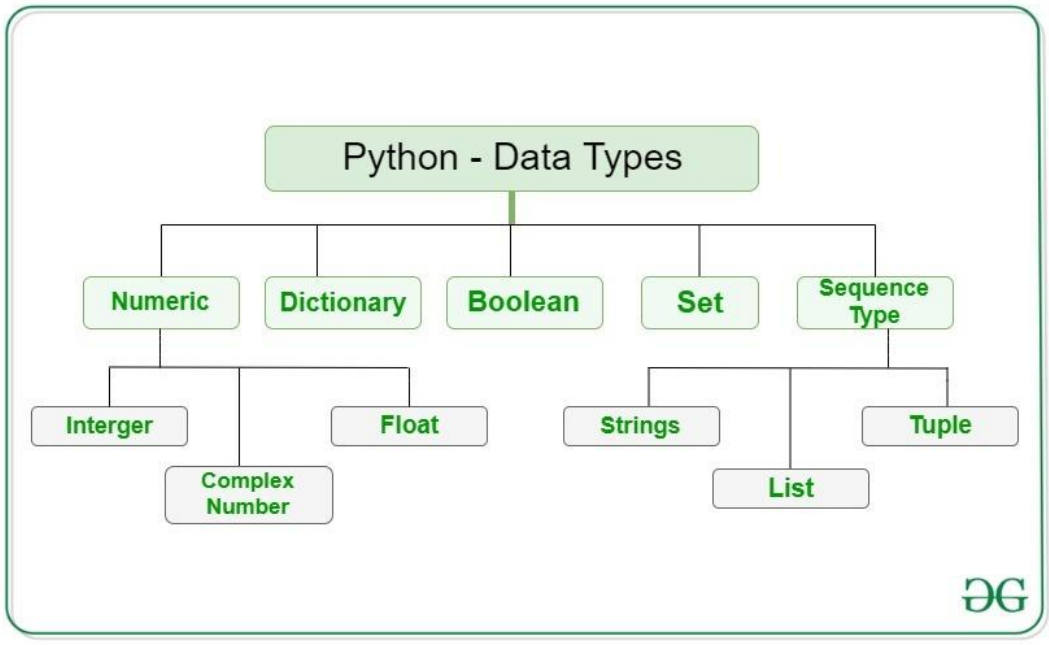
Advantages

- General-purpose programming
- Code reusability
- Portable source code

Disadvantages

- Data protection
- Not suitable for real-world objects
- Harder to write

Data Types:



In Python, numeric data type represent the data which has numeric value. Numeric value can be integer, floating number or even complex numbers. These values are defined as int, float and complex class in Python.

- Integers – This value is represented by int class. It contains positive or negative whole numbers (without fraction or decimal). In Python there is no limit to how long an integer value can be.
- Float – This value is represented by float class. It is a real number with floating point representation. It is specified by a decimal point. Optionally, the character e or E followed by a positive or negative integer may be appended to specify scientific notation.
- Complex Numbers – Complex number is represented by complex class. It is specified as *(real part) + (imaginary part)j*. For example – 2+3j

Example:

```
# Python program to demonstrate numeric value

a = 5
print("Type of a: ", type(a))

b = 5.0
print("\nType of b: ", type(b))

c = 2 + 4j
print("\nType of c: ", type(c))
```

Output:

```
Type of a:  <class 'int'>

Type of b:  <class 'float'>

Type of c:  <class 'complex'>
```

In Python, sequence is the ordered collection of similar or different data types. Sequences allows to store multiple values in an organized and efficient fashion. There are several sequence types in Python –

- String
- List
- Tuple

1) String

In Python, Strings are arrays of bytes representing Unicode characters. A string is a collection of one or more characters put in a single quote, double-quote or triple quote. In python there is no character data type, a character is a string of length one. It is represented by str class.

2) List

Lists are just like the arrays, declared in other languages which is a ordered collection of data. It is very flexible as the items in a list do not need to be of the same type

3) Tuple

tuple is also an ordered collection of Python objects. The only difference between tuple and list is that tuples are immutable i.e. tuples cannot be modified after it is created. It is represented by tuple class.

Boolean

Data type with one of the two built-in values, True or False. Boolean objects that are equal to True are truthy (true), and those equal to False are falsy (false). But non-Boolean objects can be evaluated in Boolean context as well and determined to be true or false. It is denoted by the class bool.

Set

In Python, Set is an unordered collection of data type that is iterable, mutable and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements.

Dictionary

Dictionary in Python is an unordered collection of data values, used to store data values like a map, which unlike other Data Types that hold only single value as an element, Dictionary holds key:value pair. Key-value is provided in the dictionary to make it more optimized. Each key-value pair in a Dictionary is separated by a colon :, whereas each key is separated by a 'comma'.

Keywords:

Python keywords are special reserved words that have specific meanings and purposes and can't be used for anything but those specific purposes. In Python we have 35 keywords:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

No.	Keywords	Description
1	and	This is a logical operator it returns true if both the operands are true else return false.
2	Or	This is also a logical operator it returns true if anyone operand is true else return false.

No.	Keywords	Description
3	not	This is again a logical operator it returns True if the operand is false else return false.
4	if	This is used to make a conditional statement.
5	elif	Elif is a condition statement used with if statement the elif statement is executed if the previous conditions were not true
6	else	Else is used with if and elif conditional statement the else block is executed if the given condition is not true.
7	for	This is created for a loop.
8	while	This keyword is used to create a while loop.
9	break	This is used to terminate the loop.
10	as	This is used to create an alternative.
11	def	It helps us to define functions.
12	lambda	It used to define the anonymous function.
13	pass	This is a null statement that means it will do nothing.
14	return	It will return a value and exit the function.
15	True	This is a boolean value.
16	False	This is also a boolean value.
17	try	It makes a try-except statement.
18	with	The with keyword is used to simplify exception handling.

No.	Keywords	Description
19	assert	This function is used for debugging purposes. Usually used to check the correctness of code
20	class	It helps us to define a class.
21	continue	It continues to the next iteration of a loop
22	del	It deletes a reference to an object.
23	except	Used with exceptions, what to do when an exception occurs
24	finally	Finally is use with exceptions, a block of code that will be executed no matter if there is an exception or not.
25	from	The form is used to import specific parts of any module.
26	global	This declares a global variable.
27	import	This is used to import a module.
28	in	It's used to check if a value is present in a list, tuple, etc, or not.
29	is	This is used to check if the two variables are equal or not.
30	None	This is a special constant used to denote a null value or avoid. It's important to remember, 0, any empty container(e.g empty list) do not compute to None
31	nonlocal	It's declared a non-local variable.
32	raise	This raises an exception
33	yield	It's ends a function and returns a generator.

Variables:

- Variable is nothing but, the value that we assign for a letter or word.
- variables are a storage placeholder for texts and numbers.
- Python is dynamically typed, which means that you don't have to
- declare what type each variable is.

Rules for variables:

- A variable name must start with a letter or the underscore character.
- A variable name cannot start with a number.
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _).
- Variable names are case-sensitive (age, Age and AGE are three different variables).
- Example: x = "python"
  y = "programming"
  print(x+y)
  //output = "python programming"

Identifiers:

An identifier is a name given to entities like class, functions, variables, etc. It helps to differentiate one entity from another.

Rules for writing identifiers

1. Identifiers can be a combination of letters in lowercase (a to z) or uppercase (A to Z) or digits (0 to 9) or an underscore _. Names like myClass, var_1 and print_this_to_screen, all are valid example.
2. An identifier cannot start with a digit. 1variable is invalid, but variable1 is a valid name.
3. Keywords cannot be used as identifiers

Number Types: int, float, complex

Python includes three numeric types to represent numbers: integers, float, and complex number.

Integer:

In Python, integers are zero, positive or negative whole numbers without a fractional part and having unlimited precision, e.g. 0, 100, -10. The followings are valid integer literals in Python. Integers can be binary, octal, and hexadecimal values.

Example:

```
>>> 0b11011000 # binary
216
>>> 0o12 # octal
10
>>> 0x12 # hexadecimal
15
```

All integer literals or variables are objects of the int class. Use the type() method to get the class name

Binary

A number having 0b with eight digits in the combination of 0 and 1 represent the binary numbers in Python. For example, 0b11011000 is a binary number equivalent to integer 216.

```
>>> x=0b11011000
>>> x
216
>>> x=0b_1101_1000
>>> x
216
>>> type(x)
<class 'int'>
```

Octal:

A number having 0o or 0O as prefix represents an octal number. For example, 0O12 is equivalent to integer 10.

```
>>> x=0012
>>> x
10
>>> type(x)
<class 'int'>
```

Hexadecimal

A number with 0x or 0X as prefix represents hexadecimal number. For example, 0x12 is equivalent to integer 18.

```
>>> x=0x12
>>> x
18
>>> type(x)
<class 'int'>
```

Floating Point

In Python, floating point numbers (float) are positive and negative real numbers with a fractional part denoted by the decimal symbol . or the scientific notation E or e, e.g. 1234.56, 3.142, -1.55, 0.23

```
>>> f=1.2
>>> f
1.2
>>> type(f)
<class 'float'>
```

Floats can be separated by the underscore _, e.g. 123_42.222_013 is a valid float.

```
>>> f=123_42.222_013
>>> f
12342.222013
```

Floats has the maximum size depends on your system. The float beyond its maximum size referred as "inf", "Inf", "INFINITY", or "infinity". Float 2e400 will be considered as infinity for most systems.

```
>>> f=2e400
>>> f
```

inf
Complex Number
A complex number is a number with real and imaginary components. For example, 5 + 6j is a complex number where 5 is the real component and 6 multiplied by j is an imaginary component.
Example:

```
>>> a=5+2j
>>> a
(5+2j)
>>> type(a)
<class 'complex'>
```

Collections Data Types: Tuples, Lists, Sets, dictionaries, Iterating and copying collections

List:
A list is a collection which is ordered and changeable. In Python lists are written with square brackets.

- Create a List:

```
thislist = ["apple", "banana", "cherry"]
print(thislist)
```

- Output: ['apple', 'banana', 'cherry']

List Methods:
append(): The append() method appends an element to the end of the list
Syntax: *list*.append(*elmnt*)

Example:

```
fruits = ['apple', 'banana', 'cherry']
fruits.append("orange")
print(fruits)
```

Output: ['apple','banana','cherry','orange']

Clear(): The clear() method removes all the elements from a list
Syntax: *list*.clear()

Example:

```
fruits = ['apple', 'banana', 'cherry', 'orange']
fruits.clear()
print(fruits)
```

Output: []
Copy(): The copy() method returns a copy of the specified list
Syntax: *list*.copy()

Example:

```
fruits = ["apple", "banana", "cherry"]
x = fruits.copy()
print(x)
```

Output: ['apple','banana','cherry']

Count: The count() method returns the number of elements with the specified value.
Syntax: *list*.count(*value*)

Example:

```
fruits = [1, 4, 2, 9, 7, 8, 9, 3, 1]
x = fruits.count(9)
print(x)
```

Output: 2

Extend(): The extend() method adds the specified list elements (or any iterable) to the end of the current list

Syntax: *list*.extend(*iterable*)

Example:

```
fruits = ['apple', 'banana', 'cherry']
points = (1, 4, 5, 9)
fruits.extend(points)
print(fruits)
```

Output: ['apple', 'banana', 'cherry', 1, 4, 5, 9]

Index():

Returns the index of the first element with the specified value

Syntax: list.index(elmnt)

Example: What is the position of the value "cherry":

```
fruits = ['apple', 'banana', 'cherry']
x = fruits.index("cherry")
```

Output: 2

Insert( ): The insert() method inserts the specified value at the specified position

Syntax: *list*.insert(*pos*, *elmnt*)

Example:

```
fruits = ['apple', 'banana', 'cherry']
fruits.insert(1, "orange")
print(fruits)
```

Output: ['apple','orange','banana','cherry']

Pop( ): The pop() method removes the element at the specified position

Syntax: *list*.pop(*pos*)

Example:

```
fruits = ['apple', 'banana', 'cherry']
fruits.pop(1)
print(fruits)
```

Output: ['apple','cherry']

Remove(): The remove() method removes the first occurrence of the element with the specified value

Syntax: *list*.remove(*elmnt*)

Example:

```
fruits = ['apple', 'banana', 'cherry']
fruits.remove("banana")
print(fruits)
```

Output: ['apple','cherry']

Sort(): The sort() method sorts the list ascending by default.

Syntax: *list*.sort(reverse=True|False, key=myFunc)

Example:

```
cars = ['Ford', 'BMW', 'Volvo']
cars.sort()
print(cars)
```

Output: ['BMW', 'Ford', 'Volvo']

Concatenate():

The concatenate() method used to add two lists

Example:

```
list1=[2,3,4]
list2 = [6,7,8]
list3 = list1 + list2
print(list3)
```

Output: [2,3,4,6,7,8]

Tuple:

A tuple is a collection which is ordered and unchangeable. In Python tuples are written with round brackets

Example:

```
Create a Tuple:
thistuple = ("apple", "banana", "cherry")
print(thistuple)
```

Output: ('apple', 'banana', 'cherry')

Tuple Methods:

Count( ): The count() method returns the number of times a specified value appears in the tuple.

Syntax: *tuple*.count(*value*)

Example:

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
x = thistuple.count(5)
print(x)
```

Output: 2

Index( ): The index() method finds the first occurrence of the specified value

The index() method raises an exception if the value is not found.

Syntax: *tuple*.index(*value*)

Example:

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
x = thistuple.index(8)
print(x)
```

Output: 3

Dictionary:

A dictionary is a collection which is unordered, changeable and indexed. In Python dictionaries are written with curly brackets, and have keys and values

Example:

```
Create and print a dictionary:
thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}
 print(thisdict)
```

Output: {'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

Accessing Items:

You can access the items of a dictionary by referring to its key name, inside square brackets

Dictionary Methods:

Clear( ): The clear() method removes all the elements from a dictionary

Syntax: dictionary.clear()

Example:

```
car = {"brand": "Ford",
        "model": "Mustang",
        "year": 1964
        }
        car.clear()
        print(car)
```

Output:{}

Copy( ): The copy() method returns a copy of the specified dictionary

Syntax: *dictionary*.copy()

Example:

```
        car = {"brand": "Ford",
        "model": "Mustang",
        "year": 1964
        }
x = car.copy()
print(x)
```

Output: {'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

Items( ):

The items() method returns a view object. The view object contains the key-value pairs of the dictionary, as tuples in a list

Syntax: *dictionary*.items()

Example:

```
        car = {"brand": "Ford",
                "model": "Mustang",
                "year": 1964
                }
         x = car.items()
         print(x)
```

Output: dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 1964)])

Popitem(): The popitem() method removes the item that was last inserted into the dictionary

Syntax: *dictionary*.popitem(*keyname, defaultvalue*)

Example:

```
car = {
        "brand": "Ford",
        "model": "Mustang",
        "year": 1964 }
        car.popitem()
                print(car)
```

Output: {'brand': 'Ford', 'model': 'Mustang'}

Setdefault(): The setdefault() method returns the value of the item with the specified key. If key exists no effect,if not it assigns the same value.

Syntax: *dictionary*.setdefault(*keyname, value*)

Example:

```
 car = {
        "brand": "Ford",
        "model": "Mustang",
        "year": 1964
         }
      x = car.setdefault("model", "Bronco")
print(x)
        y = car.setdefault("place","India")
        print(y)
    print(car)
```

Output: Mustang

India

{"brand":"Ford", "model":"Mustang","year": 1964,"place":"India"}

Update(): The update() method inserts the specified items to the

Syntax: *dictionary.update(iterable)*

Example:

```
car = {
        "brand": "Ford",
        "model": "Mustang",
        "year": 1964
         }
        car.update({"color": "White"})
        print(car)
```

Output: {'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'White'}

Set:

A set is a collection which is unordered and un indexed. The set list is unordered, meaning: the items will appear in a random order.

Set Methods:

Add(): The add() method adds an element to the set.

Syntax: *set*.add(*elmnt*)

Example:

```
thisset = {"apple", "banana", "cherry"}
        thisset.add("orange")
         print(thisset)
```

Output: {'orange', 'cherry', 'banana', 'apple'}

Clear(): The clear() method removes all elements in a set.

Syntax: *set*.clear()

Example:

```
thisset = {"apple", "banana", "cherry"}
thisset.clear()
print(thisset)
```

Output: set()

Discard( ): The discard() method removes the specified item from the set

Syntax: *set*.discard(*value*)

Example:

```
thisset = {"apple", "banana",   "cherry"}
thisset.discard("banana")
print(thisset)
```

Output: {'cherry', 'apple'}

Pop(): The pop() method removes a random item from the set. This method returns the removed item.

Syntax: *set*.pop()

Example:

```
fruits = {"apple", "banana", "cherry"}
fruits.pop()
print(fruits)
```

Output: {'banana', 'apple'}

Remove(): The remove() method removes the specified element from the set

Syntax: *set*.remove(*item*)

Example:

```
fruits = {"apple", "banana", "cherry"}
fruits.remove("banana")
print(fruits)
```

Output: {'cherry', 'apple'}

Update(): The update() method updates the current set, by adding items from another set

Syntax: *set*.update(*set*)

Example:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
x.update(y)
print(x)
```

Output: {'banana', 'apple', 'google', 'microsoft', 'cherry'}

difference_update() : removes the items that exists in both sets

Syntax: *set*.difference_update(*set*)

Example:

```
a = {1,2,3,4}
b = {2,3,5}
a.difference_update(b)
```

#Output: {1,4}

Union(): The union() method returns a set that contains all items from the original set, and all items from the specified sets.

Syntax: *set*.union(*set1, set2...*)

Example:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
z = x.union(y)
print(z)
```

Output: {'apple', 'banana', 'microsoft', 'google', 'cherry'}

Intersection():

returns a set, i.e. intersection of two sets

Syntax: *set*.intersection(*set1,set2..*)

Example:

```
a = {1,2,3,4,5}
b = {3,4,5}
a.intersection(b)
```

Output: {3,4,5}

Symmetric_difference():Return a set that contains all items from both sets, except items that are present in both sets

Syntax: *set*.symmetric_difference(*set*)

Example:

```
a = {1,2,3,4}
b = {2,3,5}
a.symmetric_difference(b)
```

Output: {1,4,5}

isdisjoint(): return True if no items in set x is present in set y

Syntax: *set*.isdisjoint(*set*)

Example:

```
a = {1,2,3}
b = { 4,5,6}
a.isdisjoint(b)
```

Output: True

issubset():returns True if all items in set x present in set y

Syntax: *set*.issubset(*set*)

Example:

```
a = {1,2,3}
b = {1,2,3,4,5}
a.issubset(b)
```

Output: True

issuperset():returns True if all items set y are present in set x

Syntax: *set*.issuperset(*set*)

Example:

```
x = {1,2,3,4,5,6}
y = {2,4,6}
x.issuperset(y)
```

Output: True

Iterating and copying collections

In Python, there are two ways to create copies :

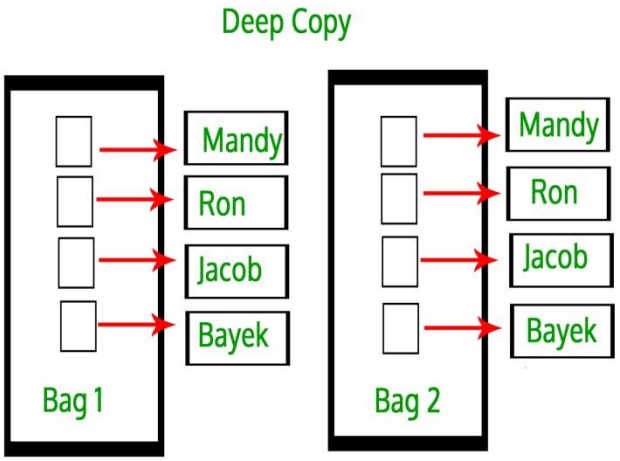
- Deep copy
- Shallow copy

In order to make these copy, we use copy module. We use copy module for shallow and deep copy operations

Deep Copy:

Deep copy is a process in which the copying process occurs recursively. It means first constructing a new collection object and then recursively populating it with copies of the child objects found in the original. In case of deep copy, a copy of object is copied in other object. It means that **any changes** made to a copy of object **do not reflect** in the original object. In python, this is implemented using "**deepcopy()**" function.

Deep copy



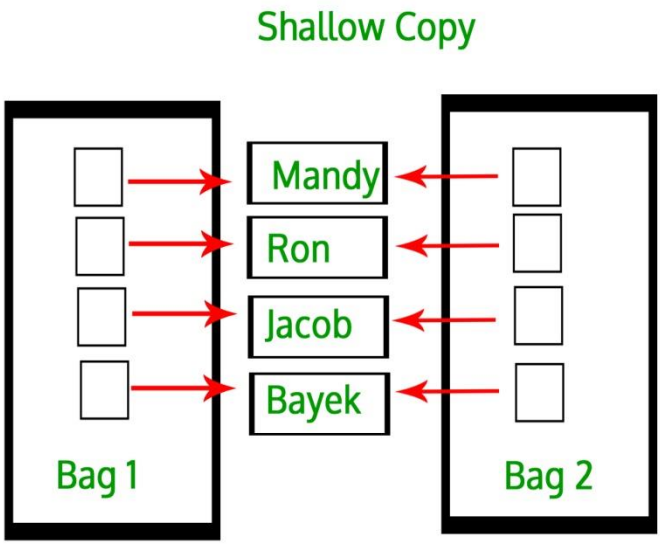
Example:

```
import copy

# initializing list 1
li1 = [1, 2, [3,5], 4]

# using deepcopy to deep copy
li2 = copy.deepcopy(li1)

# original elements of list
print ("The original elements before deep copying")
for i in range(0,len(li1)):
   print (li1[i],end=" ")

print("\r")

# adding and element to new list
li2[2][0] = 7

# Change is reflected in l2
print ("The new list of elements after deep copying ")
for i in range(0,len( li1)):
   print (li2[i],end=" ")
print("\r")
# Change is NOT reflected in original list
# as it is a deep copy
print ("The original elements after deep copying")
for i in range(0,len( li1)):
   print (li1[i],end=" ")
```

Output:

```
The original elements before deep copying
1 2 [3, 5] 4
The new list of elements after deep copying
1 2 [7, 5] 4
The original elements after deep copying
1 2 [3, 5] 4
```

Shallow copy



A shallow copy means constructing a new collection object and then populating it with references to the child objects found in the original. The copying process does not recurse and therefore won't create copies of the child objects themselves. In case of shallow copy, a reference of object is copied in other object. It means that **any changes** made to a copy of object **do reflect** in the original object. In python, this is implemented using "**copy()**" function.

Example:

```
import copy

# initializing list 1
li1 = [1, 2, [3,5], 4]

# using copy to shallow copy
li2 = copy.copy(li1)

# original elements of list
print ("The original elements before shallow copying")
for i in range(0,len(li1)):
   print (li1[i],end=" ")

print("\r")

# adding and element to new list
li2[2][0] = 7
```

```
# checking if change is reflected
print ("The original elements after shallow copying")
for i in range(0,len( li1)):
   print (li1[i],end=" ")
```

Output:

```
The original elements before shallow copying
1 2 [3, 5] 4
The original elements after shallow copying
1 2 [7, 5] 4
```

Strings:

In Python, **Strings** are arrays of bytes representing Unicode characters. However, Python does not have a character data type, a single character is simply a string with a length of 1. Square brackets can be used to access elements of the string.

Creating a String

Strings in Python can be created using single quotes or double quotes or even triple quotes.

```
# Python Program for
# Creation of String

# Creating a String
# with single Quotes
String1 = 'Welcome to the Geeks World'
print("String with the use of Single Quotes: ")
print(String1)

# Creating a String
# with double Quotes
String1 = "I'm a Geek"
print("\nString with the use of Double Quotes: ")
print(String1)

# Creating a String
# with triple Quotes
String1 = '''I'm a Geek and I live in a world of "Geeks"'''
print("\nString with the use of Triple Quotes: ")
print(String1)
```

```
# Creating String with triple
# Quotes allows multiple lines
String1 = '''Live For Life'''
print("\nCreating a multiline String: ")
print(String1)
```

Output:

String with the use of Single Quotes:

Welcome to the Live World

String with the use of Double Quotes:

I'm a Live

String with the use of Triple Quotes:

I'm a Geek and I live in a world of "Live"

Creating a multiline String:

Live

For

Life

Slicing

we can return a range of characters by using the slice syntax.

Specify the start index and the end index, separated by a colon, to return a part of the string.

Example

Get the characters from position 2 to position 5 (not included):

```
b = "Hello, World!"
print(b[2:5])
```

Output:

llo

Slice From the Start

By leaving out the start index, the range will start at the first character:

Example

Get the characters from the start to position 5 (not included):

```
b = "Hello, World!"
print(b[:5])
```

Output:

```
Hello
```

Slice To the End

By leaving out the *end* index, the range will go to the end:

Example

Get the characters from position 2, and all the way to the end:

```
b = "Hello, World!"
print(b[2:])
```

Output:

```
llo, World!
```

Negative Indexing

Use negative indexes to start the slice from the end of the string:

Example

Get the characters:

From: "o" in "World!" (position -5)

To, but not included: "d" in "World!" (position -2):

```
b = "Hello, World!"
print(b[-5:-2])
```

Output:

Orl

Modify Strings

Python has a set of built-in methods that you can use on strings.

Upper Case

Example

The upper() method returns the string in upper case:

```
a = "Hello, World!"
print(a.upper())
```

Output:

HELLO, WORLD!

Lower Case

Example

The lower() method returns the string in lower case:

```
a = "Hello, World!"
print(a.lower())
```

Output:

hello, world!

Remove Whitespace

Whitespace is the space before and/or after the actual text, and very often you want to remove this space.

Example

The strip() method removes any whitespace from the beginning or the end:

```
a = " Hello, World! "
print(a.strip())
```

Output:

Hello, World!

Replace String

Example

The replace() method replaces a string with another string:

```
a = "Hello, World!"
print(a.replace("H", "J"))
```

Output:

Jello, World!

Split String

The split() method returns a list where the text between the specified separator becomes the list items.

Example

The split() method splits the string into substrings if it finds instances of the separator:

```
a = "Hello, World!"
print(a.split(",")) # returns ['Hello', ' World!']
```

Output:

['Hello', ' World!']

String Concatenation

To concatenate, or combine, two strings you can use the + operator.

Example

Merge variable a with variable b into variable c:

```
a = "Hello"
b = "World"
c = a + b
print(c)
```

Output:

```
HelloWorld
```

Example

To add a space between them, add a " ":

```
a = "Hello"
b = "World"
c = a + " " + b
print(c)
```

Output:

```
Hello World
```

Escape Character

To insert characters that are illegal in a string, use an escape character.

An escape character is a backslash \ followed by the character you want to insert.

An example of an illegal character is a double quote inside a string that is surrounded by double quotes:

Example

The escape character allows you to use double quotes when you normally would not be allowed:

```
txt = "We are the so-called \"Vikings\" from the north."
```

```
We are the so-called "Vikings" from the north.
```

Escape Characters

Other escape characters used in Python:

Code	Result
\'	Single Quote
\\	Backslash
\n	New Line
\r	Carriage Return
\t	Tab
\b	Backspace
\f	Form Feed
\ooo	Octal value
\xhh	Hex value

String methods:

Method	Description
capitalize()	Converts the first character to upper case
casefold()	Converts string into lower case
center()	Returns a centered string
count()	Returns the number of times a specified value occurs in a string
encode()	Returns an encoded version of the string
endswith()	Returns true if the string ends with the specified value
expandtabs()	Sets the tab size of the string
find()	Searches the string for a specified value and returns the position of where it was found
format()	Formats specified values in a string
format_map()	Formats specified values in a string
index()	Searches the string for a specified value and returns the position of where it was found
isalnum()	Returns True if all characters in the string are alphanumeric
isalpha()	Returns True if all characters in the string are in the alphabet
isdecimal()	Returns True if all characters in the string are decimals
isdigit()	Returns True if all characters in the string are digits
isidentifier()	Returns True if the string is an identifier
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric
isprintable()	Returns True if all characters in the string are printable
isspace()	Returns True if all characters in the string are whitespaces
istitle()	Returns True if the string follows the rules of a title

isupper()	Returns True if all characters in the string are upper case
join()	Joins the elements of an iterable to the end of the string
ljust()	Returns a left justified version of the string
lower()	Converts a string into lower case
lstrip()	Returns a left trim version of the string
maketrans()	Returns a translation table to be used in translations
partition()	Returns a tuple where the string is parted into three parts
replace()	Returns a string where a specified value is replaced with a specified value
rfind()	Searches the string for a specified value and returns the last position of where it was found
rindex()	Searches the string for a specified value and returns the last position of where it was found
rjust()	Returns a right justified version of the string
rpartition()	Returns a tuple where the string is parted into three parts
rsplit()	Splits the string at the specified separator, and returns a list
rstrip()	Returns a right trim version of the string
split()	Splits the string at the specified separator, and returns a list

splitlines()	Splits the string at line breaks and returns a list
startswith()	Returns true if the string starts with the specified value
strip()	Returns a trimmed version of the string
swapcase()	Swaps cases, lower case becomes upper case and vice versa
title()	Converts the first character of each word to upper case
translate()	Returns a translated string
upper()	Converts a string into upper case
zfill()	Fills the string with a specified number of 0 values at the beginning

Unit-2:

Python Control Structures, Functions and OOP:Control Structures and Functions: Conditional Branching, Looping, Exception Handling, Custom Fuctions
Python Library Modules: random, math, time, os, shutil, sys, glob, re, statistics,creating a custom module Object Oriented Programming: Object Oriented Concepts and Terminology, Custom Classes, Attributes and Methods, Inheritance and Polymorphism, Using Properties to Control Attribute Access
File Handling: Writing and Reading Binary Data, Writing and Parsing Text Files

Decisions in a program are used when the program has conditional choices to execute a code block.

.

Python provides various types of conditional statements:

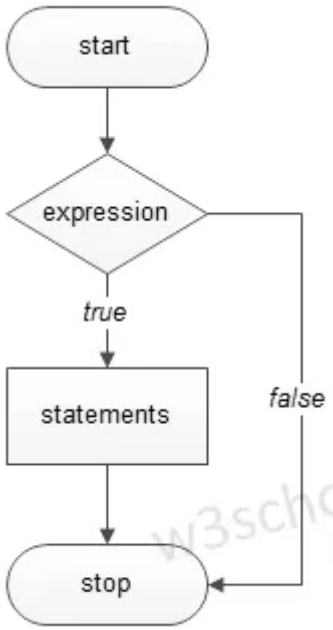
Statement	Description
if Statements	It consists of a Boolean expression which results are either TRUE or FALSE, followed by one or more statements.
if else Statements	It also contains a Boolean expression. The if the statement is followed by an optional else statement & if the expression results in FALSE, then else statement gets executed. It is also called alternative execution in which there are two possibilities of the condition determined in which any one of them will get executed.
Nested Statements	We can implement if statement and or if-else statement inside another if or if - else statement. Here more than one if conditions are applied & there can be more than one if within elif.

Python Conditional Statements

If Statement

The decision-making structures can be recognized and understood using flowcharts.

Figure - If condition Flowchart:



Syntax :

```
if expression:
  #execute your code
```

Example :

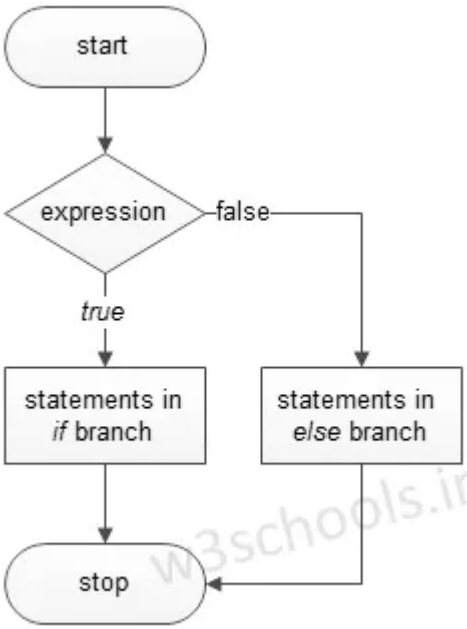
```
a = 15

if a > 10:
  print("a is greater")
```

Output

a is greater

ifelse Statements

Figure - If else condition Flowchart:



Syntax :

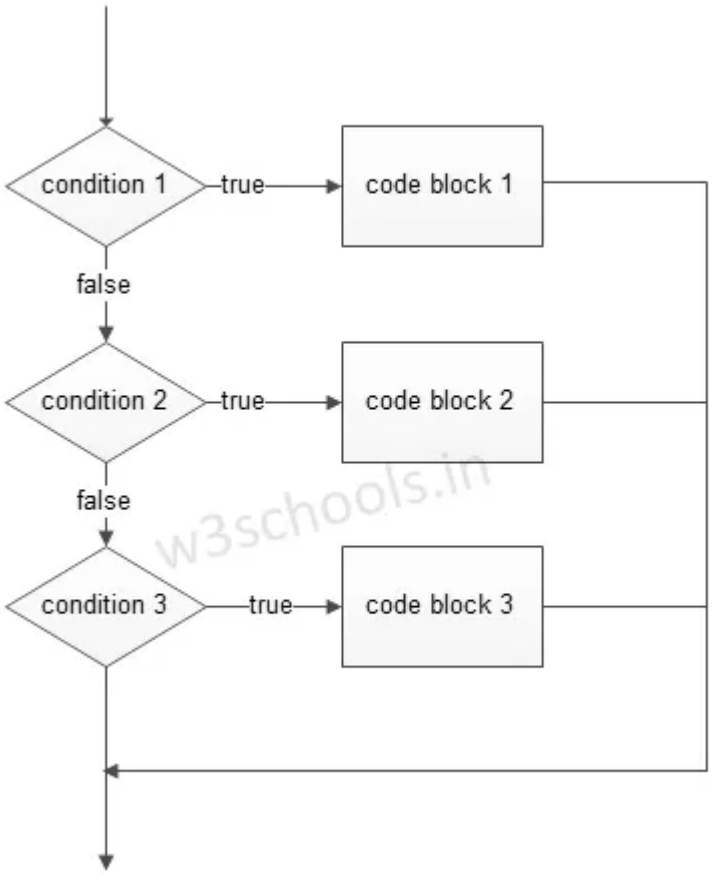
```
if expression:
  #execute your code
else:
  #execute your code
```

Source Code

```
a = 15
b = 20

if a > b:
  print("a is greater")
else:
  print("b is greater")
```

Output

b is greater
elif - is a keyword used in Python replacement of else if to place another condition in the program. This is called chained conditional.
Figure - elif condition Flowchart:



Syntax :

```
if expression:
  #execute your code
elif expression:
  #execute your code
else:
  #execute your code
```

Example :

```
a = 15
b = 15

if a > b:
  print("a is greater")
elif a == b:
  print("both are equal")
else:
```

```
print("b is greater")
```

Output :
both are equal

Single Statement Condition

If the block of an executable statement of if - clause contains only a single line, programmers can write it on the same line as a header statement.

Example

```
a = 15

if (a == 15): print("The value of a is 15")
```

Loops

In programming, loops are a sequence of instructions that does a specific set of instructions or tasks based on some conditions and continue the tasks until it reaches certain conditions.

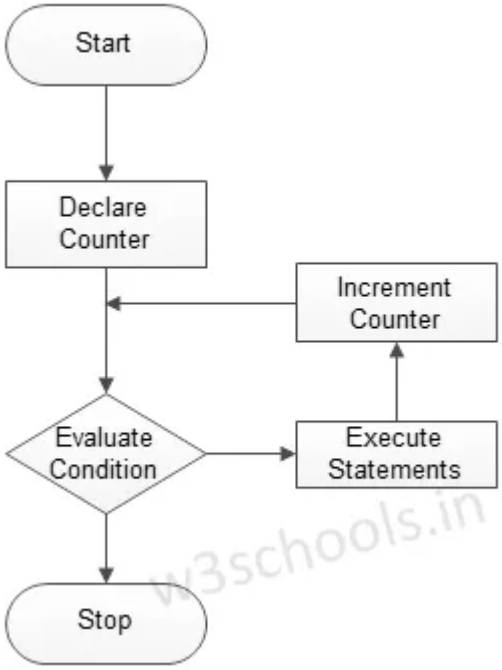
Python provides three types of looping techniques:

Loop	Description
for Loop	This is traditionally used when programmers had a piece of code and wanted to repeat that 'n' number of times.
while Loop	The loop gets repeated until the specific Boolean condition is met.
Nested Loops	Programmers can use one loop inside another; i.e., they can use for loop inside while or vice - versa or for loop inside for loop or while inside while.

Python Loops

For Loop

Figure - for loop Flowchart:



Syntax :

for iterating_var in sequence:

  #execute your code

Example 1

Source Code

```
for x in range (0,3) :
   print ('Loop execution %d' % (x))
```

OUTPUT

Loop execution 0

Loop execution 1

Loop execution 2

Example 2

Source Code

```
for letter in 'Python Programming':
   print ('Current letter is:', letter)
```

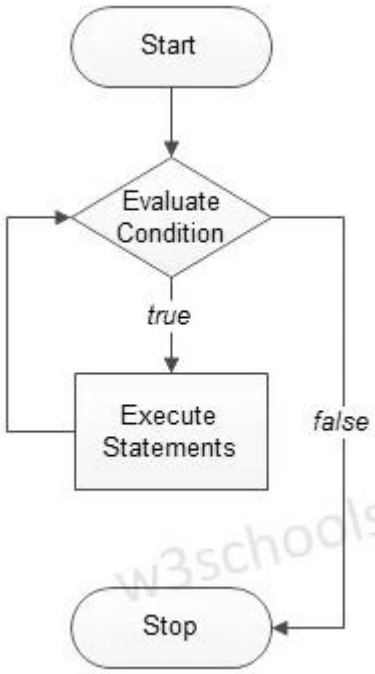
OUTPUT

```
Current letter is: P
Current letter is: y
Current letter is: t
Current letter is: h
Current letter is: o
Current letter is: n
Current letter is:
Current letter is: P
Current letter is: r
Current letter is: o
Current letter is: g
Current letter is: r
Current letter is: a
Current letter is: m
Current letter is: m
Current letter is: i
Current letter is: n
Current letter is: g
```

While Loop

The graphical representation of the logic behind while looping is shown below:

Figure - while loop Flowchart:



Syntax

while expression:

  #execute your code

Example

Source Code

```
#initialize count variable to 1
count =1

while count < 6 :
   print (count)
   count+=1
#the above line means count = count + 1
```

OUTPUT

```
1
```

2

3

4

5

Nested Loops

Syntax

```
for iterating_var in sequence:
  for iterating_var in sequence:
    #execute your code
      #execute your code
```

Example

Source Code

```
for g in range(1, 6):
   for k in range(1, 3):
      print ("%d * %d = %d" % ( g, k, g*k))
```

OUTPUT

```
1 * 1 = 1
1 * 2 = 2
2 * 1 = 2
2 * 2 = 4
```

3 * 1 = 3

3 * 2 = 6

4 * 1 = 4

4 * 2 = 8

5 * 1 = 5

5 * 2 = 10

Loop Control Statements

These statements are used to change execution from its normal sequence.

Python supports three types of loop control statements:

Python Loop Control Statements

Control Statements	Description
Break statement	It is used to exit a while loop or a for a loop. It terminates the looping & transfers execution to the statement next to the loop.
Continue statement	It causes the looping to skip the rest part of its body & start re-testing its condition.
Pass statement	It is used in Python to when a statement is required syntactically, and the programmer does not want to execute any code block or command.

Break Statement :

Syntax

Break

Source Code

```
count = 0

while count <= 100: print (count) count += 1 if count >= 3:
```

```
    break
```

OUTPUT

```
0

1

2
```

Continue Statement

Syntax :

```
continue
```

Example

Source Code

```
for x in range(10):
  #check whether x is even
  if x % 2 == 0:
    continue
  print (x)
```

OUTPUT

```
1

3

5

7

9
```

Pass Statement

Syntax

```
Pass
```

Source Code

```
for letter in 'Python Programming':
  if letter == 'P':
     pass
     print ('Pass block')
  print ('Current letter is:', letter)
```

OUTPUT

```
Pass block
Current letter is: P
Current letter is: y
Current letter is: t
Current letter is: h
Current letter is: o
Current letter is: n
Current letter is:
Pass block
Current letter is: P
Current letter is: r
Current letter is: o
Current letter is: g
Current letter is: r
Current letter is: a
Current letter is: m
Current letter is: m
Current letter is: i
Current letter is: n
Current letter is: g
```

Source Code

```
for letter in 'Python Programming':
  if letter == 'h':
    pass
    print ('Pass block')
  print ('Current letter is:', letter)
```

OUTPUT

```
Current letter is: P
Current letter is: y
Current letter is: t
Pass block
Current letter is: h
Current letter is: o
Current letter is: n
Current letter is:
Current letter is: P
Current letter is: r
Current letter is: o
Current letter is: g
Current letter is: r
Current letter is: a
Current letter is: m
Current letter is: m
Current letter is: i
Current letter is: n
Current letter is: g
```

Source Code

```
for letter in 'Python Programming':
  if letter == 'm':
    pass
    print ('Pass block')
  print ('Current letter is:', letter)
```

OUTPUT

```
Current letter is: P
Current letter is: y
Current letter is: t
Current letter is: h
Current letter is: o
Current letter is: n
Current letter is:
Current letter is: P
Current letter is: r
Current letter is: o
Current letter is: g
Current letter is: r
Current letter is: a
Pass block
Current letter is: m
Pass block
Current letter is: m
Current letter is: i
Current letter is: n
Current letter is: g
```

Python Functions

A function is a block of code which only runs when it is called.You can pass data, known as parameters, into a function.
A function can return data as a result.

Creating a Function

In Python a function is defined using the def keyword:

Example

```
def my_function():
  print("Hello from a function")
```

Calling a Function

To call a function, use the function name followed by parenthesis:

Example

Source Code

```
def my_function():
  print("Hello from a function")
my_function()
```

Output

```
Hello from a function
```

Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. Youcan add as many arguments as you want, just separate them with a comma

Source Code

```
def my_function(fname):
  print(fname + " Sirivennela")
my_function("Seetha")
my_function("Rama")
my_function("Shastry")
```

Output

```
Seetha Sirivennela
Rama Sirivennela
Shastry Sirivennela
```

Number of Arguments

By default, a function must be called with the correct number of arguments. Meaning that if your function expects 2 arguments, you have to call the functionwith 2 arguments, not more, and not less.

Example
This function expects 2 arguments, and gets 2 arguments:
Source Code

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Sirivennela", "Seetharam Shastry")
```

Output

```
Sirivennela Seetharam Shastry
```

Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function,add a * before the parameter name in the function definition.

This way the function will receive a *tuple* of arguments, and can access theitems accordingly:

Example

If the number of arguments is unknown, add a * before the parameter name:

Source Code

```
def my_function(*kids):
  print("The youngest child is " + kids[2])
my_function("one", "Two", "three")
```

Output

```
The youngest child is three
```

Keyword Arguments

You can also send arguments with the *key = value* syntax.This way the order of the arguments does not matter.

Source Code

```
def my_function(child3, child2, child1):
  print("The youngest child is " + child3)
my_function(child1 = "hi1", child2 = "hello2", child3 = "hii3")
```

Output

```
The youngest child is hii3
```

Arbitrary Keyword Arguments, **kw args

If you do not know how many keyword arguments that will be passed into yourfunction, add two asterisk: ** before the parameter name in the function definition.

This way the function will receive a *dictionary* of arguments, and can access theitems accordingly:

Source Code

```
def my_function(**kid):
  print("His last name is " + kid["lname"])
my_function(fname = "lilly", lname = "jasmine")
```

Output

```
His last name is jasmine
```

Default Parameter Value

The following example shows how to use a default parameter value.If we call the function without argument, it uses the default value:

Example

Source Code

```
def my_function(country = "Norway"):
  print("I am from " + country)
my_function("Sweden")
my_function("India") my_function()
my_function("Brazil")
```

Output

```
I am from Sweden
I am from India
I am from Norway
I am from Brazil
```

Passing a List as an Argument

You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches thefunction

Source Code

```
def my_function(food):
  for x in food:
    print(x)
fruits = ["apple", "banana", "cherry"]
my_function(fruits)
```

Output

```
apple
banana
cherry
```

Recursion

Python also accepts function recursion, which meansdefined function can call itself

Source Code

```
def tri_recursion(k):

  if(k > 0):

    result = k + tri_recursion(k - 1)
    print(result)
  else:

    result = 0 return
  result
print("\n\nRecursion Example Results")
```

```
tri_recursion(6)
```

Python Exception

An exception can be defined as an unusual condition in a program resulting in the interruption in the flow of the program.

Whenever an exception occurs, the program stops the execution, and thus the further code is not executed. Therefore, an exception is the run-time errors that are unable to handle to Python script. An exception is a Python object that represents an error

Python provides a way to handle the exception so that the code can be executed without any interruption. If we do not handle the exception, the interpreter doesn't execute all the code that exists after the exception.

Python has many **built-in exceptions** that enable our program to run without interruption and give the output. These exceptions are given below:

Common Exceptions

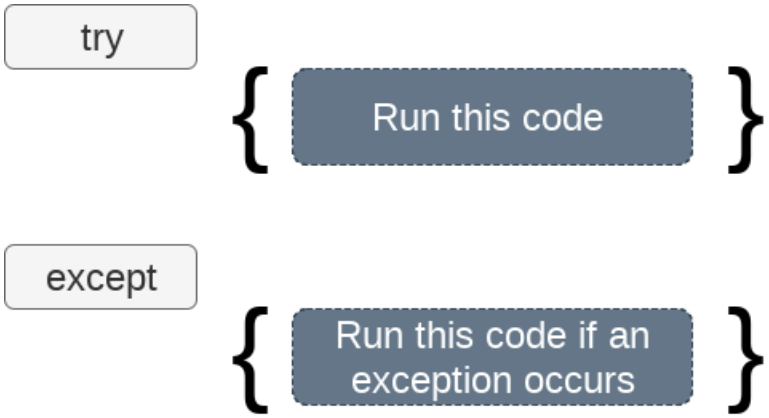
Python provides the number of built-in exceptions, but here we are describing the common standard exceptions. A list of common exceptions that can be thrown from a standard Python program is given below.

1. **ZeroDivisionError:** Occurs when a number is divided by zero.
2. **NameError:** It occurs when a name is not found. It may be local or global.
3. **IndentationError:** If incorrect indentation is given.
4. **IOError:** It occurs when Input Output operation fails.
5. **EOFError:** It occurs when the end of the file is reached, and yet operations are being performed.

Exception handling in python

The try-expect statement

If the Python program contains suspicious code that may throw the exception, we must place that code in the **try** block. The **try** block must be followed with the **except** statement, which contains a block of code that will be executed if there is some exception in the try block.



Syntax

```
try:
   #block of code

except Exception1:
   #block of code

except Exception2:
   #block of code

#other code
```

Example:

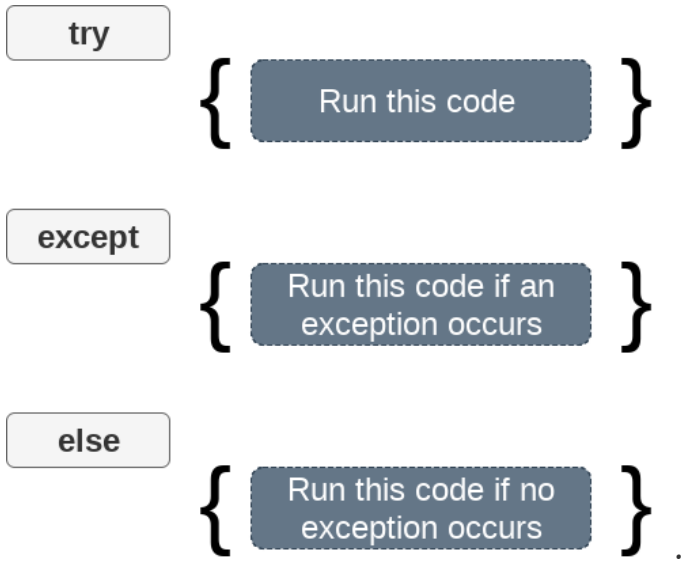
```
try:
   a = int(input("Enter a:"))
   b = int(input("Enter b:"))
   c = a/b
except:
   print("Can't divide with zero")
```

Output:

```
Enter a:10
Enter b:0
Can't divide with zero
```

The syntax to use the else statement with the try-except statement is given below.

```
try:
   #block of code

except Exception1:
   #block of code

else:
   #this code executes if no except block is executed
```



Example

```
try:
   a = int(input("Enter a:"))
   b = int(input("Enter b:"))
   c = a/b
   print("a/b = %d"%c)
# Using Exception with except statement. If we print(Exception) it will return exception class
except Exception:
   print("can't divide by zero")
   print(Exception)
else:
   print("Hi I am else block")
```

Output:

```
Enter a:10
Enter b:0
can't divide by zero
<class 'Exception'>
```

The except statement with no exception

Python provides the flexibility not to specify the name of exception with the exception statement.

Example

```
try:
   a = int(input("Enter a:"))
   b = int(input("Enter b:"))
   c = a/b;
   print("a/b = %d"%c)
except:
   print("can't divide by zero")
else:
   print("Hi I am else block")
```

The except statement using with exception variable

We can use the exception variable with the **except** statement. It is used by using the **as** keyword. this object will return the cause of the exception. Consider the following example:

```
try:
   a = int(input("Enter a:"))
   b = int(input("Enter b:"))
   c = a/b
   print("a/b = %d"%c)
   # Using exception object with the except statement
except Exception as e:
   print("can't divide by zero")
   print(e)
else:
   print("Hi I am else block")
```

Output:

```
Enter a:10
Enter b:0
can't divide by zero
division by zero
```

PYTHON RANDOM MODULE:

The Python random module functions depend on a pseudo-random number generator function random(), which generates the float number between 0.0 and 1.0.

There are different types of functions used in a random module which is given below:

random.random():This function generates a random float number between 0.0 and 1.0.

random.randint():This function returns a random integer between the specified integers.

random.choice():This function returns a randomly selected element from a non-empty sequence.

Example:

```
# importing "random"  module.

import random

# We are using the choice() function to generate a random number from

# the given list of numbers.

print ("The random number from list is : ",end="")

print (random.choice([50, 41, 84, 40, 31]))
```

Output:

The random number from list is : 84

random.shuffle():

This function randomly reorders the elements in the list.random.

randrange(beg,end,step):

This function is used to generate a number within the range specified in its argument. It accepts three arguments, beginning number, last number, and step, which is used to skip a number in the range. Consider the following example.

```
# We are using randrange() function to generate in range from 100

# to 500. The last parameter 10 is step size to skip

# ten numbers when selecting.

import random

print ("A random number from range is : ",end="")

print (random.randrange(100, 500, 10))
```

Output:

A random number from range is : 290

random.seed():

This function is used to apply on the particular random number with the seed argument. It returns the mapper value. Consider the following example.

```
# importing "random" module.

import random

# using random() to generate a random number

# between 0 and 1

print("The random number between 0 and 1 is : ", end="")

print(random.random())

random.seed(4) :using seed() to seed a random number
```

Output:

The random number between 0 and 1 is : 0.4405576668981033

PYTHON MATH MODULE:

Python math module is defined as the most famous mathematical functions, which includes trigonometric functions, representation functions, logarithmic functions, etc. Furthermore, it also defines two mathematical constants, i.e., Pie and Euler number, etc.

Pie (n): It is a well-known mathematical constant and defined as the ratio of circumstance to the diameter of a circle. Its value is 3.141592653589793.

Euler's number(e): It is defined as the base of the natural logarithmic, and its value is 2.718281828459045.

There are different math modules which are :

math.log():

This method returns the natural logarithm of a given number. It is calculated to the base e.

Example:

```
import math

number = 2e-7  # small value of of x

print('log(fabs(x), base) is :', math.log(math.fabs(number), 10))
```

Output:

log(fabs(x), base) is : -6.698970004336019

math.log10():

This method returns base 10 logarithm of the given number and called the standard logarithm.

Example

```
import math

x=13  # small value of of x

print('log10(x) is :', math.log10(x))
```

Output:

log10(x) is : 1.1139433523068367

math.exp():

This method returns a floating-point number after raising e to the given number.

Example

```
import math

number = 5e-2  # small value of of x

print('The given number (x) is :', number)

print('e^x (using exp() function) is :', math.exp(number)-1)
```

Output:

The given number (x) is : 0.05

e^x (using exp() function) is : 0.05127109637602412

math.pow(x,y):

This method returns the power of the x corresponding to the value of y. If value of x is negative or y is not integer value than it raises a ValueError.

Example

```
import math

number = math.pow(10,2)

print("The power of number:",number)
```

Output:

The power of number: 100.0

math.floor(x):

This method returns the floor value of the x. It returns the less than or equal value to x.

Example:

```
import math

number = math.floor(10.25201)

print("The floor value is:",number)
```

Output:

The floor value is: 10

math.ceil(x):

This method returns the ceil value of the x. It returns the greater than or equal value to x.

```
import math

number = math.ceil(10.25201)

print("The floor value is:",number)
```

Output:

The floor value is: 11

math.fabs(x):

This method returns the absolute value of x.

Example:

```
import math

number = math.fabs(10.001)

print("The floor absolute is:",number)
```

Output:

The absolute value is: 10.001

math.factorial():

This method returns the factorial of the given number x. If x is not integral, it raises a ValueError.

Example

```
import math

number = math.factorial(7)

print("The factorial of number:",number)
```

Output:

The factorial of number: 5040

PYTHON OS MODULE:

Python OS module provides the facility to establish the interaction between the user and the operating system. It offers many useful OS functions that are used to perform OS-based tasks and get related information about operating system.The OS comes under Python's standard utility modules. This module offers a portable way of using operating system dependent functionality.

The Python OS module lets us work with the files and directories.To work with the OS module, we need to import the OS module.

```
import os
```

There are some functions in the OS module which are :

os.name():

This function provides the name of the operating system module that it imports.

Currently, it registers 'posix', 'nt', 'os2', 'ce', 'java' and 'riscos'.

Example

```
import os

print(os.name)
```

Output:

```
nt
```

os.mkdir():

The os.mkdir() function is used to create new directory. Consider the following example.

```
import os

os.mkdir("d:\\newdir")  :It will create the new directory to the path in the string argument of the function in the D drive named folder newdir.
```

os.getcwd():

It returns the current working directory(CWD) of the file.

Example

```
import os

print(os.getcwd())
```

Output:

C:\Users\Python\Desktop\ModuleOS

os.chdir():

The os module provides the chdir() function to change the current working directory.

```
import os
```

os.rmdir():

The rmdir() function removes the specified directory with an absolute or related path. First, we have to change the current working directory and remove the folder.

Example

```
import os

# It will throw a Permission error; that's why we have to change the current working directory.

os.rmdir("d:\\newdir")

os.chdir("..")

os.rmdir("newdir")
```

os.error():

The os.error() function defines the OS level errors. It raises OSError in case of invalid or inaccessible file names and path etc.

Example

```
import os
```

```
 try:
   # If file does not exist,
   # then it throw an IOError
   filename = 'Python.txt'
   f = open(filename, 'rU')
   text = f.read()
   f.close()
 # The Control jumps directly to here if
# any lines throws IOError.
except IOError:
    # print(os.error) will <class 'OSError'>
   print('Problem reading: ' + filename)
```

Output:

Problem reading: Python.txt

os.popen():

This function opens a file or from the command specified, and it returns a file object which is connected to a pipe.

Example

```
import os
fd = "python.txt"
# popen() is similar to open()
file = open(fd, 'w')
file.write("This is awesome")
file.close()
```

```
file = open(fd, 'r')

text = file.read()

print(text)

# popen() provides gateway and accesses the file directly

file = os.popen(fd, 'w')

file.write("This is awesome")

# File not closed, shown in next function.
```

Output:

```
This is awesome
```

os.close():

This function closes the associated file with descriptor fr.

Example

```
import os

fr = "Python1.txt"

file = open(fr, 'r')

text = file.read()

print(text)

os.close(file)
```

Output:

```
Traceback (most recent call last):

  File "main.py", line 3, in

    file = open(fr, 'r')

FileNotFoundError: [Errno 2] No such file or directory: 'Python1.txt'
```

os.rename():

A file or directory can be renamed by using the function os.rename(). A user can rename the file if it has privilege to change the file.

Example

```
import os

fd = "python.txt"

os.rename(fd,'Python1.txt')

os.rename(fd,'Python1.txt')
```

Output:

```
Traceback (most recent call last):

  File "main.py", line 3, in

    os.rename(fd,'Python1.txt')

FileNotFoundError: [Errno 2]

No such file or directory: 'python.txt' -> 'Python1.txt'
```

PYHTON SYS MODULE:

The python sys module provides functions and variables which are used to manipulate different parts of the Python Runtime Environment. It lets us access system-specific parameters and functions.

```
import sys
```

First, we have to import the sys module in our program before running any functions.

sys.modules:This function provides the name of the existing python modules which have been imported.

sys.argv:This function returns a list of command line arguments passed to a Python script. The name of the script is always the item at index 0, and the rest of the arguments are stored at subsequent indices.

sys.base_exec_prefix:This function provides an efficient way to the same value as exec_prefix. If not running a virtual environment, the value will remain the same.

sys.base_prefix:It is set up during Python startup, before site.py is run, to the same value as prefix.

sys.byteorder:It is an indication of the native byteorder that provides an efficient way to do something.

sys.maxsize:This function returns the largest integer of a variable.

sys.path:This function shows the PYTHONPATH set in the current system. It is an environment variable that is a search path for all the python modules.

sys.stdin:It is an object that contains the original values of stdin at the start of the program and used during finalization. It can restore the files.

sys.getrefcount:This function returns the reference count of an object.

sys.exit:This function is used to exit from either the Python console or command prompt, and also used to exit from the program in case of an exception.

sys executable:The value of this function is the absolute path to a Python interpreter. It is useful for knowing where python is installed on someone else machine.

sys.platform:This value of this function is used to identify the platform on which we are working.

PYTHON STATISTICS MODULE:

Python statistics module provides the functions to mathematical statistics of numeric data. There are some popular statistical functions defined in this module.

mean() function: The mean() function is used to calculate the arithmetic mean of the numbers in the list.

Example:

```
import statistics

# list of positive integer numbers

datasets = [5, 2, 7, 4, 2, 6, 8]

x = statistics.mean(datasets)

# Printing the mean
```

```
print("Mean is :", x)
```

Output:

```
Mean is : 4.857142857142857
```

median() function

:The median() function is used to return the middle value of the numeric data in the list.

Example

```
import statistics

datasets = [4, -5, 6, 6, 9, 4, 5, -2]

# Printing median of the

# random data-set

print("Median of data-set is : % s "

    % (statistics.median(datasets)))
```

Output:

```
Median of data-set is : 4.5
```

mode() function:

The mode() function returns the most common data that occurs in the list.

Example

```
import statistics

# declaring a simple data-set consisting of real valued positive integers.

dataset =[2, 4, 7, 7, 2, 2, 3, 6, 6, 8]

# Printing out the mode of given data-set
```

```
print("Calculated Mode % s" % (statistics.mode(dataset)))
```

Output:

```
Calculated Mode 2
```

stdev() function:

The stdev() function is used to calculate the standard deviation on a given sample which is available in the form of the list.

Example

```
import statistics

# creating a simple data - set

sample = [7, 8, 9, 10, 11]

# Prints standard deviation

print("Standard Deviation of sample is % s "

        % (statistics.stdev(sample)))
```

Output:

```
Standard Deviation of sample is 1.5811388300841898
```

median_low():

The median_low function is used to return the low median of numeric data in the list.

Example

```
import statistics

# simple list of a set of integers

set1 = [4, 6, 2, 5, 7, 7]

# Note: low median will always be a member of the data-set.

# Print low median of the data-set

print("Low median of data-set is % s "
```

```
    % (statistics.median_low(set1)))
```

Output:

Low median of the data-set is 5

median_high():

The median_high function is used to return the high median of numeric data in the list.

Example:

```
import statistics

# list of set of the integers

dataset = [2, 1, 7, 6, 1, 9]

print("High median of data-set is %s "

    % (statistics.median_high(dataset)))
```

Output:

High median of the data-set is 6

SHUTIL MODULE:

Shutil module offers high-level operation on a file like a copy, create, and remote operation on the file. It comes under Python's standard utility modules. This module helps in automating the process of copying and removal of files and directories.

shutil.copy() method in Python is used to copy the content of the source file to the destination file or directory. It also preserves the file's permission mode but other metadata of the file like the file's creation and modification times is not preserved.
The source must represent a file but the destination can be a file or a directory. If the destination is a directory then the file will be copied into the destination using the base filename from the source. Also, the destination must be writable. If the destination is a file and already exists then it will be replaced with the source file otherwise a new file will be created.
*Syntax: shutil.copy(source, destination, *, follow_symlinks = True)*
Parameter:

- *source: A string representing the path of the source file.*
- *destination: A string representing the path of the destination file or directory.*

- *follow_symlinks (optional) : The default value of this parameter is True. If it is False and source represents a symbolic link then destination will be created as a symbolic link.*

Return Type: This method returns a string which represents the path of newly created file.

Example

```
# Python program to explain shutil.copy() method

# importing shutil module
import shutil

source = "path/main.py"
destination ="path/main2.py"

# Copy the content of
# source to destination
dest = shutil.copy(source, destination)

# Print path of newly
# created file
print("Destination path:", dest)
```

Output:
Destination path: path/main2.py

Copying the Metadata along with File

shutil.copy2() method in Python is used to copy the content of the source file to the destination file or directory. This method is identical to shutil.copy() method but it also tries to preserve the file's metadata.
*Syntax: shutil.copy2(source, destination, *, follow_symlinks = True)*

Parameter:

- *source: A string representing the path of the source file.*
- *destination: A string representing the path of the destination file or directory.*
- *follow_symlinks (optional) : The default value of this parameter is True. If it is False and source represents a symbolic link then it attempts to copy all metadata from the source symbolic link to the newly-created destination symbolic link. This functionality is platform dependent.*

Return Type: This method returns a string which represents the path of newly created file.

```
# Python program to explain shutil.copy2() method
```

```
# importing os module
import os

# importing shutil module
import shutil

# path
path = 'csv/'

# List files and directories
# in '/home/User/Documents'
print("Before copying file:")
print(os.listdir(path))


# Source path
source = "csv/main.py"

# Print the metadeta
# of source file
metadata = os.stat(source)
print("Metadata:", metadata, "\n")

# Destination path
destination = "csv/gfg/check.txt"

# Copy the content of
# source to destination
dest = shutil.copy2(source, destination)

# List files and directories
# in "/home / User / Documents"
print("After copying file:")
print(os.listdir(path))

# Print the metadata
# of the destination file
matadata = os.stat(destination)
print("Metadata:", metadata)

# Print path of newly
# created file
print("Destination path:", dest)
```

Output:

Before copying file:
['archive (2)', 'c.jpg', 'c.PNG', 'Capture.PNG', 'cc.jpg', 'check.zip', 'cv.csv', 'd.png', 'Done! Terms And Conditions Generator – The Fastest Free Terms and Conditions Generator!.pdf', 'file1.csv', 'gfg', 'haarcascade_frontalface_alt2.xml', 'log_transformed.jpg', 'main.py', 'nba.csv', 'new_gfg.png', 'r.gif', 'Result -_ Terms and Conditions are Ready!.pdf', 'rockyou.txt', 'sample.txt']
Metadata: os.stat_result(st_mode=33206, st_ino=2251799814202896, st_dev=1689971230, st_nlink=1, st_uid=0, st_gid=0, st_size=1916, st_atime=1612953710, st_mtime=1612613202, st_ctime=1612522940)

After copying file:

['archive (2)', 'c.jpg', 'c.PNG', 'Capture.PNG', 'cc.jpg', 'check.zip', 'cv.csv', 'd.png', 'Done! Terms And Conditions Generator – The Fastest Free Terms and Conditions Generator!.pdf', 'file1.csv', 'gfg', 'haarcascade_frontalface_alt2.xml', 'log_transformed.jpg', 'main.py', 'nba.csv', 'new_gfg.png', 'r.gif', 'Result -_ Terms and Conditions are Ready!.pdf', 'rockyou.txt', 'sample.txt']
Metadata: os.stat_result(st_mode=33206, st_ino=2251799814202896, st_dev=1689971230, st_nlink=1, st_uid=0, st_gid=0, st_size=1916, st_atime=1612953710, st_mtime=1612613202, st_ctime=1612522940)

Copying the content of one file to another

shutil.copyfile() method in Python is used to copy the content of the source file to the destination file. The metadata of the file is not copied. Source and destination must represent a file and destination must be writable. If the destination already exists then it will be replaced with the source file otherwise a new file will be created.
If source and destination represent the same file then SameFileError exception will be raised.

Syntax: shutil.copyfile(source, destination, *, follow_symlinks = True)

Parameter:

- *source: A string representing the path of the source file.*
- *destination: A string representing the path of the destination file.*
- *follow_symlinks (optional) : The default value of this parameter is True. If False and source represents a symbolic link then a new symbolic link will be created instead of copying the file.*

Return Type: This method returns a string which represents the path of newly created file.

```
# Python program to explain shutil.copyfile() method
# importing shutil module
import shutil

# Source path
source = "csv/main.py"
```

```
# Destination path
destination = "csv/gfg/main_2.py"

dest = shutil.copyfile(source, destination)

print("Destination path:", dest)
```

Output:
Destination path: csv/gfg/main_2.py

Replicating complete Directory

shutil.copytree() method recursively copies an entire directory tree rooted at source (src) to the destination directory. The destination directory, named by (dst) must not already exist. It will be created during copying.

Syntax:
shutil.copytree(src, dst, symlinks = False, ignore = None, copy_function = copy2, igonre_dangling_symlinks = False)

Parameters:
src: A string representing the path of the source directory.
dest: A string representing the path of the destination.
symlinks (optional) : This parameter accepts True or False, depending on which the metadata of the original links or linked links will be copied to the new tree.
ignore (optional) : If ignore is given, it must be a callable that will receive as its arguments the directory being visited by copytree(), and a list of its contents, as returned by os.listdir().
copy_function (optional): The default value of this parameter is copy2. We can use other copy function like copy() for this parameter.
igonre_dangling_symlinks (optional) : This parameter value when set to True is used to put a silence on the exception raised if the file pointed by the symlink doesn't exist.
Return Value: This method returns a string which represents the path of newly created directory.

```
# Python program to explain shutil.copytree() method
# importing os module
import os

# importing shutil module
import shutil

# path
path = 'C:/Users/ksaty/csv/gfg'

print("Before copying file:")
```

```
print(os.listdir(path))

# Source path
src = 'C:/Users/ksaty/csv/gfg'

# Destination path
dest = 'C:/Users/ksaty/csv/gfg/dest'

# Copy the content of
# source to destination
destination = shutil.copytree(src, dest)

print("After copying file:")
print(os.listdir(path))

# Print path of newly
# created file
print("Destination path:", destination)
```

Output:

Before copying file:
['cc.jpg', 'check.txt', 'log_transformed.jpg', 'main.py', 'main2.py', 'main_2.py']

After copying file:
['cc.jpg', 'check.txt', 'dest', 'log_transformed.jpg', 'main.py', 'main2.py', 'main_2.py']
Destination path: C:/Users/ksaty/csv/gfg/dest

Removing a Directory

shutil.rmtree() is used to delete an entire directory tree, the path must point to a directory (but not a symbolic link to a directory).
Syntax: shutil.rmtree(path, ignore_errors=False, onerror=None)
Parameters:
path: A path-like object representing a file path. A path-like object is either a string or bytes object representing a path.
ignore_errors: If ignore_errors is true, errors resulting from failed removals will be ignored.
oneerror: If ignore_errors is false or omitted, such errors are handled by calling a handler specified by onerror.

```
# Python program to demonstrate
# shutil.rmtree()

import shutil
import os
```

```
# location
location = "csv/gfg/"

# directory
dir = "dest"

# path
path = os.path.join(location, dir)

# removing directory
shutil.rmtree(path)
```

Finding files

shutil.which() method tells the path to an executable application that would be run if the given cmd was called. This method can be used to find a file on a computer which is present on the PATH.

Syntax: shutil.which(cmd, mode = os.F_OK | os.X_OK, path = None)

Parameters:
cmd: A string representing the file.
mode: This parameter specifies mode by which method should execute. os.F_OK tests existence of the path and os.X_OK Checks if path can be executed or we can say mode determines if the file exists and executable.
path: This parameter specifies the path to be used, if no path is specified then the results of os.environ() are used
Return Value: This method returns the path to an executable application

```
# importing shutil module
import shutil

# file search
cmd = 'anaconda'

# Using shutil.which() method
locate = shutil.which(cmd)

# Print result
print(locate)
```

Output:
D:\Installation_bulk\Scripts\anaconda.EXE

PYTHON TIME MODULE:

Python has a module named time to handle time-related tasks. To use functions defined in the module, we need to import the module first. Here's how:

```
import time
```

Here are commonly used time-related functions.

Python time.time()

The time() function returns the number of seconds passed since epoch.

For Unix system, January 1, 1970, 00:00:00 at UTC is epoch (the point where time begins).

```
import time
seconds = time.time()
print("Seconds since epoch =", seconds)
```

Python time.ctime()

The time.ctime() function takes seconds passed since epoch as an argument and returns a string representing local time.

```
import time

# seconds passed since epoch
seconds = 1545925769.9618232
local_time = time.ctime(seconds)
print("Local time:", local_time)
```

If you run the program, the output will be something like:

```
Local time: Thu Dec 27 15:49:29 2018
```

Python time.sleep()

The sleep() function suspends (delays) execution of the current thread for the given number of seconds.

```
import time

print("This is printed immediately.")
time.sleep(2.4)
print("This is printed after 2.4 seconds.")
```

time.struct_time Class

Several functions in the time module such as gmtime(), asctime() etc. either take time.struct_time object as an argument or return it.

```
time.struct_time(tm_year=2018, tm_mon=12, tm_mday=27,
                 tm_hour=6, tm_min=35, tm_sec=17,
                 tm_wday=3, tm_yday=361, tm_isdst=0)
```

Index	Attribute	Values
0	tm_year	0000, ..., 2018, ..., 9999
1	tm_mon	1, 2, ..., 12
2	tm_mday	1, 2, ..., 31
3	tm_hour	0, 1, ..., 23
4	tm_min	0, 1, ..., 59
5	tm_sec	0, 1, ..., 61
6	tm_wday	0, 1, ..., 6; Monday is 0
7	tm_yday	1, 2, ..., 366
8	tm_isdst	0, 1 or -1

The values (elements) of the time.struct_time object are accessible using both indices and attributes.

Python time.localtime()

The localtime() function takes the number of seconds passed since epoch as an argument and returns struct_time in local time.

```
import time

result = time.localtime(1545925769)
print("result:", result)
print("\nyear:", result.tm_year)
print("tm_hour:", result.tm_hour)
```

Python time.gmtime()

The gmtime() function takes the number of seconds passed since epoch as an argument and returns struct_time in UTC.

```
import time

result = time.gmtime(1545925769)
print("result:", result)
print("\nyear:", result.tm_year)
print("tm_hour:", result.tm_hour)
```

When you run the program, the output will be:

```
result = time.struct_time(tm_year=2018, tm_mon=12, tm_mday=28, tm_hour=8, tm_min=44,
tm_sec=4, tm_wday=4, tm_yday=362, tm_isdst=0)

year = 2018
tm_hour = 8
```

Glob Module in Python

With the help of the Python glob module, we can search for all the path names which are looking for files matching a specific pattern (which is defined by us). The specified pattern for file matching is defined according to the rules dictated by the Unix shell. The result obtained by following these rules for a specific pattern file matching is returned in the arbitrary order in the output of the program. While using the file matching pattern, we have to fulfil some requirements of the glob module because the module can travel through the list of the files at some location in our local disk.

Pattern Matching Functions

In Python, we have several functions which we can use to list down the files that match with the specific pattern which we have defined inside the function in a program. With the help of these functions, we can get the result list of the files which will match the given pattern in the specified folder in an arbitrary order in the output.

1. fnmatch()
2. scandir()
3. path.expandvars()
4. path.expanduser()

The first two functions present in the above-given list, i.e., fnmatch.fnmatch() and os.scandir() function, is actually used to perform the pattern matching task and not by invoking the sub-shell in the Python. These two functions perform the pattern matching task and get the list of all filenames and that too in arbitrary order

Rules of Pattern

We have to follow a specific set of rules while defining the pattern for the filename pattern matching functions in the glob module.

Following are set of rules for the pattern that we define inside the glob module's pattern matching functions:

- We have to follow all the standard set of rules of the UNIX path expansion in the pattern matching.
- The path we define inside the pattern should be either absolute or relative, and we can't define any unclear path inside the pattern.
- The special characters allowed inside the pattern are only two wild-cards, i.e., '*, ?' and the normal characters that can be expressed inside the pattern are expressed in [].
- The rules of the pattern for glob module functions are applied to the filename segment (which is provided in the functions), and it stops at the path separator, i.e., '/' of the files.

.

GLOB PYTHON MODULE:

1. iglob()
2. glob()
3. escape()

1. iglob() Function: The iglob() function of the glob module is very helpful in yielding the arbitrary values of the list of files in the output. We can create a Python generator with the iglob() method. We can use the Python generator created by the glob module to list down the files under a given directory. This function also returns an iterator when called, and the iterator returned by it yields the values (list of files) without storing all of the filenames simultaneously.

Syntax:

1. **iglob**(pathname, *, recursive=False)

As we can see in the syntax of iglob() function, it takes a total of three parameters in it, which can be defined as given below:

(i) pathname: The pathname parameter is the optional parameter of the function, and we can even leave it while we are working on the file directory that is the same as where our Python is

installed. We have to define the pathname from where we have to collect the list of files that following a similar pattern (which is also defined inside the function).
(ii) recursive: It is also an optional parameter for the iglob() function, and it takes only bool values (true or false) in it. The recursive parameter is used to set if the function is following the recursive approach for finding file names or not.
(iii) '*': This is the mandatory parameter of the iglob() function as here we have to define the pattern for which the iglob() function will collect the file names and list them down in the output. The pattern we define inside the iglob() function (such as the extension of file) for the pattern matching should start with the '*' symbol.
Now, let's use this iglob() function in an example program so that we can understand its implementation and function in a better way.

Example :

:

```
1. # Import glob module in the program
2. import glob as gb
3. # Initialize a variable
4. inVar = gb.iglob("*.py") # Set Pattern in iglob() function
5. # Returning class type of variable
6. print(type(inVar))
7. # Printing list of names of all files that matched the pattern
8. print("List of the all the files in the directory having extension .py: ")
9. for py in inVar:
10.    print(py)
```

Output:

```
<class 'generator'>
List of the all the files in the directory having extension .py:
adding.py
changing.py
code#1.py
code#2.py
code-3.py
code-4.py
code.py
code37.py
code_5.py
code_6.py
configuring.py
```

2. glob() Function: With the help of the glob() function, we can also get the list of files that matching a specific pattern (We have to define that specific pattern inside the function). The list returned by the glob() function will be a string that should contain a path specification according to the path we have defined inside the function. The string or iterator for glob() function actually returns the same value as returned by the iglob() function without actually storing these values (filenames) in it.

Syntax:

1. glob(pathname, *, recursive = True)

Example

```
# Import glob module in the program
    import glob as gb
    # Initialize a variable
    genVar = gb.glob("*.py") # Set Pattern in glob() function
    # Printing list of names of all files that matched the pattern
    print("List of the all the files in the directory having extension .py: ")
    for py in genVar:
      print(py)
```

Output:

```
List of the all the files in the directory having extension .py:
adding.py
changing.py
code#1.py
code#2.py
code-3.py
code-4.py
code.py
code37.py
code_5.py
code_6.py
configuring.py
```

.

3**. escape() Function:** The escape() becomes very impactful as it allows us to escape the given character sequence, which we defined in the function. The escape() function is very handy for locating files that having certain characters (as we will define in the function) in their file names. It will match the sequence by matching an arbitrary literal string in the file names with that special character in them.

Syntax:

1. >> escape(pathname)

Example

```
# Import glob module in the program
import glob as gb
# Initialize a variable
charSeq = "-_#"
print("Following is the list of filenames that match the special character sequence of escape fu
nction: ")
# Using nested for loop to get the filenames
for splChar in charSeq:
  # Pathname for the glob() function
  escSet = "*" + gb.escape(splChar) + "*" + ".py"
  # Printing list of filenames with glob() function
  for py in (gb.glob(escSet)):
```

```
print(py)
```

Output:

```
Following is the list of filenames that match the special character sequence of escape function:
code-3.py
code-4.py
code_5.py
code_6.py
code#1.py
code#2.py
```

PYTHON REGEX MODULE:

The regular expressions can be defined as the sequence of characters which are used to search for a pattern in a string. The module re provides the support to use regex in the python program. The re module throws an exception if there is some error while using the regular expression.
The re module must be imported to use the regex functionalities in python.

```
1. import re
```

Regex Functions

The following regex functions are used in the python.

SN	Function	Description
1	match	This method matches the regex pattern in the string with the optional flag. It returns true if a match is found in the string otherwise it returns false.
2	search	This method returns the match object if there is a match found in the string.
3	findall	It returns a list that contains all the matches of a pattern in the string.
4	split	Returns a list in which the string has been split in each match.
5	sub	Replace one or many matches in the string.

Forming a regular expression

A regular expression can be formed by using the mix of meta-characters, special sequences, and sets.

Meta-Characters

Metacharacter	Description	Example
[]	It represents the set of characters.	"[a-z]"

\	It represents the special sequence.	"\r"
.	It signals that any character is present at some specific place.	"Ja.v."
^	It represents the pattern present at the beginning of the string.	"^Java"
$	It represents the pattern present at the end of the string.	"point"
*	It represents zero or more occurrences of a pattern in the string.	"hello*"
+	It represents one or more occurrences of a pattern in the string.	"hello+"
{}	The specified number of occurrences of a pattern the string.	"java{2}"
|	It represents either this or that character is present.	"java|point"
()	Capture and group	

Special Sequences

Special sequences are the sequences containing \ followed by one of the characters.

Character	Description
\A	It returns a match if the specified characters are present at the beginning of the string.
\b	It returns a match if the specified characters are present at the beginning or the end of the string.
\B	It returns a match if the specified characters are present at the beginning of the string but not at the end.
\d	It returns a match if the string contains digits [0-9].
\D	It returns a match if the string doesn't contain the digits [0-9].
\s	It returns a match if the string contains any white space character.
\S	It returns a match if the string doesn't contain any white space character.
\w	It returns a match if the string contains any word characters.
\W	It returns a match if the string doesn't contain any word.

\Z	Returns a match if the specified characters are at the end of the string.

Sets

A set is a group of characters given inside a pair of square brackets. It represents the special meaning.

SN	Set	Description
1	[arn]	Returns a match if the string contains any of the specified characters in the set.
2	[a-n]	Returns a match if the string contains any of the characters between a to n.
3	[^arn]	Returns a match if the string contains the characters except a, r, and n.
4	[0123]	Returns a match if the string contains any of the specified digits.
5	[0-9]	Returns a match if the string contains any digit between 0 and 9.
6	[0-5][0-9]	Returns a match if the string contains any digit between 00 and 59.
10	[a-zA-Z]	Returns a match if the string contains any alphabet (lower-case or upper-case).

The Match object methods

There are the following methods associated with the Match object.

1. span(): It returns the tuple containing the starting and end position of the match.
2. string(): It returns a string passed into the function.
3. group(): The part of the string is returned where the match is found.

Example

```
import re

str = "How are you. How is everything"

matches = re.search("How", str)

print(matches.span())

print(matches.group())
```

```
print(matches.string)
```

Output:

```
(0, 3)
How
How are you. How is everything
```

OBJECT OREINTED PROGRAMMING

Overview of OOP Terminology

- Class – A user-defined prototype for an object that defines a set of attributes that characterize any object of the class. The attributes are data members (class variables and instance variables) and methods, accessed via dot notation.
- Class variable – A variable that is shared by all instances of a class. Class variables are defined within a class but outside any of the class's methods. Class variables are not used as frequently as instance variables are.
- Data member – A class variable or instance variable that holds data associated with a class and its objects.
- Function overloading – The assignment of more than one behavior to a particular function. The operation performed varies by the types of objects or arguments involved.
- Instance variable – A variable that is defined inside a method and belongs only to the current instance of a class.
- Inheritance – The transfer of the characteristics of a class to other classes that are derived from it.
- Instance – An individual object of a certain class. An object obj that belongs to a class Circle, for example, is an instance of the class Circle.
- Instantiation – The creation of an instance of a class.
- Method – A special kind of function that is defined in a class definition.
- Object – A unique instance of a data structure that's defined by its class. An object comprises both data members (class variables and instance variables) and methods.
- Operator overloading – The assignment of more than one function to a particular operator.

ATTRIBUTE AND METHODS IN PYTHON:

Attributes of a class are function objects that define corresponding methods of its instances. They are used to implement access controls of the classes.Attributes of a class can also be accessed using the following built-in methods and functions :

1. getattr() – This function is used to access the attribute of object.
2. hasattr() – This function is used to check if an attribute exist or not.
3. setattr() – This function is used to set an attribute. If the attribute does not exist, then it would be created.
4. delattr() – This function is used to delete an attribute. If you are accessing the attribute after deleting it raises error "class has no attribute".

```
# Python code for accessing attributes of class
class emp:
   name='Harsh'
   salary='25000'
   def show(self):
      print (self.name)
      print (self.salary)
e1 = emp()
# Use getattr instead of e1.name
print (getattr(e1,'name'))

# returns true if object has attribute
print (hasattr(e1,'name'))

# sets an attribute
setattr(e1,'height',152)

# returns the value of attribute name height
print (getattr(e1,'height'))

# delete the attribute
delattr(emp,'salary')
```

Static methods : A static method is a method[member function] that don't use argument self at all. To declare a static method, proceed it with the statement "@staticmethod".

```
# Python code for accessing methods using static method
class test:
   @staticmethod
   def square(x):
      test.result = x*x
```

```
# object 1 for class
t1=test()

# object 2 for class
t2 = test()
t1.square(2)

# printing result for square(2)
print (t1.result)
t2.square(3)

# printing result for square(3)
print (t2.result)

# printing the last value of result as we declared the method static
print (t1.result)
```

Accessing attributes and methods of one class in another class
Accessing attributes and methods of one class in another class is done by passing the object of one class to another.
Explained with the example given below :

```
# Python code for Accessing attributes and methods
# of one class in another class

class ClassA():
    def __init__(self):
        self.var1 = 1
        self.var2 = 2

    def methodA(self):
        self.var1 = self.var1 + self.var2
        return self.var1

class ClassB(ClassA):
    def __init__(self, class_a):
        self.var1 = class_a.var1
        self.var2 = class_a.var2

object1 = ClassA()
# updates the value of var1
```

```
summ = object1.methodA()

# return the value of var1
print (summ)

# passes object of classA
object2 = ClassB(object1)

# return the values carried by var1,var2
print( object2.var1)
print (object2.var2)
```

INHERITANCE AND POLYMORPHISM:

Inheritance is a mechanism which allows us to create a new class - known as child class - that is based upon an existing class - the parent class, by adding new attributes and methods on top of the existing class. When you do so, the child class inherits attributes and methods of the parent class.

By using inheritance, we can abstract out common properties to a general Shape class (parent class) and then we can create child classes such as Rectangle, Triangle and Circle that inherits from the Shape class. A child class class inherits all the attributes and methods from it's parent class, but it can also

```
class ParentClass:
   # body of ParentClass
   # method1
   # method2

class ChildClass(ParentClass):
   # body of ChildClass
   # method 1
   # method 2
```

Example:

It creates a class named Shape, which contains attributes and methods common to all shapes, then it creates two child classes Rectangle and Triangle which contains attributes and methods specific to them only.

```
1 import math
```

```
2
3 class Shape:
4
5     def __init__(self, color='black', filled=False):
6         self.__color = color
7         self.__filled = filled
8
9     def get_color(self):
10        return self.__color
11
12    def set_color(self, color):
13        self.__color = color
14
15    def get_filled(self):
16        return self.__filled
17
18    def set_filled(self, filled):
19        self.__filled = filled
20
21
22 class Rectangle(Shape):
23
24    def __init__(self, length, breadth):
25        super().__init__()
26        self.__length = length
27        self.__breadth = breadth
28
29    def get_length(self):
30        return self.__length
31
32    def set_length(self, length):
33        self.__length = length
34
35    def get_breadth(self):
36        return self.__breadth
37
38    def set_breadth(self, breadth):
39        self.__breadth = breadth
40
41    def get_area(self):
42        return self.__length * self.__breadth
43
44    def get_perimeter(self):
45        return 2 * (self.__length + self.__breadth)
46
```

```
47
48class Circle(Shape):
49    def __init__(self, radius):
50        super().__init__()
51        self.__radius = radius
52
53    def get_radius(self):
54        return self.__radius
55
56    def set_radius(self, radius):
57        self.__radius = radius
58
59    def get_area(self):
60        return math.pi * self.__radius ** 2
61
62    def get_perimeter(self):
63        return 2 * math.pi * self.__radius
64
65
66r1 = Rectangle(10.5, 2.5)
67
68print("Area of rectangle r1:", r1.get_area())
69print("Perimeter of rectangle r1:", r1.get_perimeter())
70print("Color of rectangle r1:", r1.get_color())
71 print("Is rectangle r1 filled ? ", r1.get_filled())
72 r1.set_filled(True)
73 print("Is rectangle r1 filled ? ", r1.get_filled())
74 r1.set_color("orange")
75 print("Color of rectangle r1:", r1.get_color())
76
77 c1 = Circle(12)
78
79print("\nArea of circle c1:", format(c1.get_area(), "0.2f"))
80print("Perimeter of circle c1:", format(c1.get_perimeter(), "0.2f"))
81 print("Color of circle c1:", c1.get_color())
82 print("Is circle c1 filled ? ", c1.get_filled())
83 c1.set_filled(True)
84 print("Is circle c1 filled ? ", c1.get_filled())
85 c1.set_color("blue")
86 print("Color of circle c1:", c1.get_color())
```

Output:

```
Area of rectagle r1: 26.25
Perimeter of rectagle r1: 26.0
Color of rectagle r1: black
Is rectagle r1 filled ?  False
Is rectagle r1 filled ?  True
Color of rectagle r1: orange

Area of circle c1: 452.39
Perimeter of circle c1: 75.40
Color of circle c1: black
Is circle c1 filled ?  False
Is circle c1 filled ?  True
Color of circle c1: blue
```

Multiple Inheritance

Python allows us to derive a class from several classes at once, this is known as Multiple Inheritance. Its general format is:

```
Class ParentClass_1:
    # body of ParentClass_1

Class ParentClass_2:
    # body of ParentClass_2

Class ParentClass_3:
    # body of ParentClass_1

Class ChildClass(ParentClass_1, ParentClass_2, ParentClass_3):
    # body of ChildClass

```

The ChildClass is derived from three classes ParentClass_1, ParentClass_2, ParentClass_3. As a result, it will inherit attributes and methods from all the three classes.The following program demonstrates multiple inheritance in action:

python101/Chapter-16/multiple_inheritance.py

```
class A:
    def explore(self):
        print("explore() method called")
```

```
4
5 class B:
6   def search(self):
7     print("search() method called")
8
9 class C:
10  def discover(self):
11    print("discover() method called")
12
13 class D(A, B, C):
14  def test(self):
15    print("test() method called")
16
17
18 d_obj = D()
19 d_obj.explore()
20 d_obj.search()
21 d_obj.discover()
22 d_obj.test()
```

Output:

```
1 explore() method called
2 search() method called
3 discover() method called
4 test() method called
```

Polymorphism and Method Overriding

Polymorphism means the ability to take various forms. In Python, Polymorphism allows us to define methods in the child class with the same name as defined in their parent class.

As we know, a child class inherits all the methods from the parent class. However, you will encounter situations where the method inherited from the parent class doesn't quite fit into the child class. In such cases, you will have to re-implement method in the child class. This process is known as Method Overriding.

In you have overridden a method in child class, then the version of the method will be called based upon the the type of the object used to call it. If a child class object is used to call an overridden method then the child class version of the method is called. On the other hand, if parent class object is used to call an overridden method, then the parent class version of the method is called.

Example:

```
class A:
    def explore(self):
        print("explore() method from class A")

class B(A):
    def explore(self):
        print("explore() method from class B")


b_obj = B()
a_obj = A()

b_obj.explore()
a_obj.explore()
```

Output:

```
explore() method from class B
explore() method from class A
```

FILE HANDLING:

Python provides inbuilt functions for creating, writing and reading files. There are two types of files that can be handled in python, normal text files and binary files (written in binary language, 0s and 1s).

- Text files: In this type of file, Each line of text is terminated with a special character called EOL (End of Line), which is the new line character ('\n') in python by default.
- Binary files: In this type of file, there is no terminator for a line and the data is stored after converting it into machine understandable binary language.

In this article, we will be focusing on opening, closing, reading, and writing data in a text file.

File Access Modes

Access modes govern the type of operations possible in the opened file. It refers to how the file will be used once its opened. These modes also define the location of the File Handle in the file. File handle is like a cursor, which defines from where the data has to be read or written in the file. There are 6 access modes in python.

1. Read Only ('r') : Open text file for reading. The handle is positioned at the beginning of the file. If the file does not exists, raises I/O error. This is also the default mode in which file is opened.
2. Read and Write ('r+') : Open the file for reading and writing. The handle is positioned at the beginning of the file. Raises I/O error if the file does not exists.
3. Write Only ('w') : Open the file for writing. For existing file, the data is truncated and over-written. The handle is positioned at the beginning of the file. Creates the file if the file does not exists.
4. Write and Read ('w+') : Open the file for reading and writing. For existing file, data is truncated and over-written. The handle is positioned at the beginning of the file.
5. Append Only ('a') : Open the file for writing. The file is created if it does not exist. The handle is positioned at the end of the file. The data being written will be inserted at the end, after the existing data.
6. Append and Read ('a+') : Open the file for reading and writing. The file is created if it does not exist. The handle is positioned at the end of the file. The data being written will be inserted at the end, after the existing data.

Opening a File

File_object = open(r"File_Name","Access_Mode")

The file should exist in the same directory as the python program file else, full address of the file should be written on place of filename.

```
# Open function to open the file "MyFile1.txt"

# (same directory) in append mode and

file1 = open("MyFile.txt","a")



# store its reference in the variable file1

# and "MyFile2.txt" in D:\Text in file2

file2 = open(r"D:\Text\MyFile2.txt","w+")
```

Here, file1 is created as object for MyFile1 and file2 as object for MyFile2

Closing a file

close() function closes the file and frees the memory space acquired by that file. It is used at the time when the file is no longer needed or if it is to be opened in a different file mode.

File_object.close()

```
# Opening and Closing a file "MyFile.txt"

# for object name file1.

file1 = open("MyFile.txt","a")

file1.close()
```

Writing to a file

There are two ways to write in a file.

1. write() : Inserts the string str1 in a single line in the text file.
   File_object.write(str1)
2. writelines() : For a list of string elements, each string is inserted in the text file.Used to insert multiple strings at a single time.
   File_object.writelines(L) for L = [str1, str2, str3]

Reading from a file

There are three ways to read data from a text file.

1. read() : Returns the read bytes in form of a string. Reads n bytes, if no n specified, reads the entire file.
   File_object.read([n])
2. readline() : Reads a line of the file and returns in form of a string.For specified n, reads at most n bytes. However, does not reads more than one line, even if n exceeds the length of the line.
   File_object.readline([n])
3. readlines() : Reads all the lines and return them as each line a string element in a list.
   File_object.readlines()

```
# Program to show various ways to read and
# write data in a file.
file1 = open("myfile.txt","w")
L = ["This is Delhi \n","This is Paris \n","This is London \n"]

# \n is placed to indicate EOL (End of Line)
file1.write("Hello \n")
file1.writelines(L)
file1.close() #to change file access modes
```

```
file1 = open("myfile.txt","r+")

print("Output of Read function is ")
print(file1.read())
print()

# seek(n) takes the file handle to the nth
# bite from the beginning.
file1.seek(0)

print( "Output of Readline function is ")
print(file1.readline())
print()

file1.seek(0)

# To show difference between read and readline
print("Output of Read(9) function is ")
print(file1.read(9))
print()

file1.seek(0)

print("Output of Readline(9) function is ")
print(file1.readline(9))

file1.seek(0)
# readlines function
print("Output of Readlines function is ")
print(file1.readlines())
print()
file1.close()
```

```
Output:
Output of Read function is
Hello
This is Delhi
This is Paris
This is London


Output of Readline function is
Hello
```

```
Output of Read(9) function is
Hello
Th
Output of Readline(9) function is
Hello
Output of Readlines function is
['Hello \n', 'This is Delhi \n', 'This is Paris \n', 'This is London \n']
```

Appending to a file

```
# Python program to illustrate
# Append vs write mode
file1 = open("myfile.txt","w")
L = ["This is Delhi \n","This is Paris \n","This is London \n"]
file1.writelines(L)
file1.close()

# Append-adds at last
file1 = open("myfile.txt","a")#append mode
file1.write("Today \n")
file1.close()

file1 = open("myfile.txt","r")
print("Output of Readlines after appending")
print(file1.readlines())
print()
```

```
file1.close()
# Write-Overwrites
file1 = open("myfile.txt","w")#write mode
file1.write("Tomorrow \n")
file1.close()
 file1 = open("myfile.txt","r")
print("Output of Readlines after writing")
print(file1.readlines())
print()
file1.close()
```

Output:

```
Output of Readlines after appending
['This is Delhi \n', 'This is Paris \n', 'This is London \n', 'Today \n']

Output of Readlines after writing
['Tomorrow \n']
```

Unit-3

NumPy Arrays and Vectorized Computation: NumPy arrays, Array creation, Indexing and slicing, Fancy indexing, Numerical operations on arrays, Array functions, Data processing using arrays, Loading and saving data, Saving an array, Loading an array, Linear algebra with NumPy, NumPy random numbers

Numpy array and vectirized computation:

Numpy Arrays:

Numpy is the core library for scientific computing in Python. It provides a high-performance multidimensional array object, and tools for working with these arrays. If you are already familiar with MATLAB, you might find this tutorial useful to get started with Numpy.

Arrays

A numpy array is a grid of values, all of the same type, and is indexed by a tuple of nonnegative integers. The number of dimensions is the *rank* of the array; the *shape* of an array is a tuple of integers giving the size of the array along each dimension.

We can initialize numpy arrays from nested Python lists, and access elements using square brackets:

```
import numpy as np

a = np.array([1, 2, 3])  # Create a rank 1 array
print(type(a))           # Prints "<class 'numpy.ndarray'>"
print(a.shape)           # Prints "(3,)"
print(a[0], a[1], a[2])  # Prints "1 2 3"
a[0] = 5                 # Change an element of the array
print(a)                 # Prints "[5, 2, 3]"

b = np.array([[1,2,3],[4,5,6]])   # Create a rank 2 array
print(b.shape)                    # Prints "(2, 3)"
print(b[0, 0], b[0, 1], b[1, 0])  # Prints "1 2 4"
```

Numpy also provides many functions to create arrays:

```
import numpy as np

a = np.zeros((2,2))  # Create an array of all zeros
print(a)             # Prints "[[ 0.  0.]
                     #          [ 0.  0.]]"

b = np.ones((1,2))   # Create an array of all ones
print(b)             # Prints "[[ 1.  1.]]"

c = np.full((2,2), 7)  # Create a constant array
print(c)               # Prints "[[ 7.  7.]
                       #          [ 7.  7.]]"

d = np.eye(2)        # Create a 2x2 identity matrix
print(d)             # Prints "[[ 1.  0.]
                     #          [ 0.  1.]]"

e = np.random.random((2,2))  # Create an array filled with random values
print(e)                     # Might print "[[ 0.91940167  0.08143941]
```

```
#          [ 0.68744134  0.87236687]]"
```

Array indexing

Numpy offers several ways to index into arrays.

Slicing: Similar to Python lists, numpy arrays can be sliced. Since arrays may be multidimensional, you must specify a slice for each dimension of the array:

```
import numpy as np

# Create the following rank 2 array with shape (3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])

# Use slicing to pull out the subarray consisting of the first 2 rows
# and columns 1 and 2; b is the following array of shape (2, 2):
# [[2 3]
#  [6 7]]
b = a[:2, 1:3]

# A slice of an array is a view into the same data, so modifying it
# will modify the original array.
print(a[0, 1])   # Prints "2"
b[0, 0] = 77    # b[0, 0] is the same piece of data as a[0, 1]
print(a[0, 1])   # Prints "77"
```

You can also mix integer indexing with slice indexing. However, doing so will yield an array of lower rank than the original array. Note that this is quite different from the way that MATLAB handles array slicing:

```
import numpy as np

# Create the following rank 2 array with shape (3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])

# Two ways of accessing the data in the middle row of the array.
```

```
# Mixing integer indexing with slices yields an array of lower rank,
# while using only slices yields an array of the same rank as the
# original array:
row_r1 = a[1, :]    # Rank 1 view of the second row of a
row_r2 = a[1:2, :]  # Rank 2 view of the second row of a
print(row_r1, row_r1.shape)  # Prints "[5 6 7 8] (4,)"
print(row_r2, row_r2.shape)  # Prints "[[5 6 7 8]] (1, 4)"

# We can make the same distinction when accessing columns of an array:
col_r1 = a[:, 1]
col_r2 = a[:, 1:2]
print(col_r1, col_r1.shape)  # Prints "[ 2  6 10] (3,)"
print(col_r2, col_r2.shape)  # Prints "[[ 2]
                             #         [ 6]
                             #         [10]] (3, 1)"
```

Integer array indexing: When you index into numpy arrays using slicing, the resulting array view will always be a subarray of the original array. In contrast, integer array indexing allows you to construct arbitrary arrays using the data from another array. Here is an example:

```
import numpy as np

a = np.array([[1,2], [3, 4], [5, 6]])

# An example of integer array indexing.
# The returned array will have shape (3,) and
print(a[[0, 1, 2], [0, 1, 0]])  # Prints "[1 4 5]"

# The above example of integer array indexing is equivalent to this:
print(np.array([a[0, 0], a[1, 1], a[2, 0]]))  # Prints "[1 4 5]"

# When using integer array indexing, you can reuse the same
# element from the source array:
print(a[[0, 0], [1, 1]])  # Prints "[2 2]"

# Equivalent to the previous integer array indexing example
print(np.array([a[0, 1], a[0, 1]]))  # Prints "[2 2]"
```

One useful trick with integer array indexing is selecting or mutating one element from each row of a matrix:

```
import numpy as np

# Create a new array from which we will select elements
```

```
a = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])

print(a) # prints "array([[ 1,  2,  3],
         #              [ 4,  5,  6],
         #              [ 7,  8,  9],
         #              [10, 11, 12]])"

# Create an array of indices
b = np.array([0, 2, 0, 1])

# Select one element from each row of a using the indices in b
print(a[np.arange(4), b]) # Prints "[ 1  6  7 11]"

# Mutate one element from each row of a using the indices in b
a[np.arange(4), b] += 10

print(a) # prints "array([[11,  2,  3],
         #              [ 4,  5, 16],
         #              [17,  8,  9],
         #              [10, 21, 12]])
```

Boolean array indexing: Boolean array indexing lets you pick out arbitrary elements of an array. Frequently this type of indexing is used to select the elements of an array that satisfy some condition. Here is an example:

```
import numpy as np

a = np.array([[1,2], [3, 4], [5, 6]])

bool_idx = (a > 2)  # Find the elements of a that are bigger than 2;
                    # this returns a numpy array of Booleans of the same
                    # shape as a, where each slot of bool_idx tells
                    # whether that element of a is > 2.

print(bool_idx)     # Prints "[[False False]
                    #          [ True  True]
                    #          [ True  True]]"

# We use boolean array indexing to construct a rank 1 array
# consisting of the elements of a corresponding to the True values
# of bool_idx
print(a[bool_idx]) # Prints "[3 4 5 6]"

# We can do all of the above in a single concise statement:
print(a[a > 2])    # Prints "[3 4 5 6]"
```

INDEXING AND SLICING:

- **Indexing in Python** means referring to an element of an iterable by its position within the iterable.
- Each character can be accessed using their index number.
- To access characters in a string we have two ways:
  - **Positive index number**
  - **Negative index number**

Positive indexing example in Python

In **Python Positive indexing**, we pass a positive index that we want to access in square brackets. The index number starts from **0** which denotes the **first character** of a string.

Negative indexing example in Python

In **negative indexing in Python**, we pass the negative index which we want to access in square brackets. Here, the index number starts from index number **-1** which denotes the **last character** of a string.

Slicing in python is used for accessing parts of a sequence. The slice object is used to slice a given sequence or any object. We use slicing when we require a part of a string and not the complete string.

Syntax:

string[start : end : step]

Fancy Indexing:

Fancy indexing is conceptually simple: it means passing an array of indices to access multiple array elements at once. For example, consider the following array:

```
import numpy as np
rand = np.random.RandomState(42)

x = rand.randint(100, size=10)
print(x)
```

[51 92 14 71 60 20 82 86 74 74]

Suppose we want to access three different elements. We could do it like this:

In [2]:

```
[x[3], x[7], x[2]]
```

Out[2]:

[71, 86, 14]

Alternatively, we can pass a single list or array of indices to obtain the same result:

In [3]:

```
ind = [3, 7, 4]
```

```
x[ind]
```

Out[3]:

```
array([71, 86, 60])
```

When using fancy indexing, the shape of the result reflects the shape of the *index arrays* rather than the shape of the *array being indexed*:

In [4]:

```
ind = np.array([[3, 7],
                [4, 5]])
x[ind]
```

Out[4]:

```
array([[71, 86],
       [60, 20]])
```

Fancy indexing also works in multiple dimensions. Consider the following array:

In [5]:

```
X = np.arange(12).reshape((3, 4))
X
```

Out[5]:

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

Like with standard indexing, the first index refers to the row, and the second to the column:

In [6]:

```
row = np.array([0, 1, 2])
col = np.array([2, 1, 3])
X[row, col]
```

Out[6]:

```
array([ 2,  5, 11])
```

Notice that the first value in the result is `X[0, 2]`, the second is `X[1, 1]`, and the third is `X[2, 3]`. The pairing of indices in fancy indexing follows all the broadcasting rules that were mentioned in Computation on Arrays: Broadcasting. So, for example, if we combine a column vector and a row vector within the indices, we get a two-dimensional result:

In [7]:

```
X[row[:, np.newaxis], col]
```

Out[7]:

```
array([[ 2,  1,  3],
       [ 6,  5,  7],
       [10,  9, 11]])
```

Here, each row value is matched with each column vector, exactly as we saw in broadcasting of arithmetic operations. For example:

In [8]:

```
row[:, np.newaxis] * col
```

Out[8]:

```
array([[0, 0, 0],
       [2, 1, 3],
       [4, 2, 6]])
```

It is always important to remember with fancy indexing that the return value reflects the *broadcasted shape of the indices*, rather than the shape of the array being indexed.

NUMERICAL OPERATIONS ON ARRAYS:

Input arrays for performing arithmetic operations such as add(), subtract(), multiply(), and divide() must be either of the same shape or should conform to array broadcasting rules.

Example

```
import numpy as np
a = np.arange(9, dtype = np.float_).reshape(3,3)

print 'First array:'
print a
print '\n'

print 'Second array:'
b = np.array([10,10,10])
print b
print '\n'

print 'Add the two arrays:'
print np.add(a,b)
print '\n'

print 'Subtract the two arrays:'
print np.subtract(a,b)
print '\n'
```

```
print 'Multiply the two arrays:'
print np.multiply(a,b)
print '\n'

print 'Divide the two arrays:'
print np.divide(a,b)
```

It will produce the following output –

```
First array:
[[ 0. 1. 2.]
 [ 3. 4. 5.]
 [ 6. 7. 8.]]

Second array:
[10 10 10]

Add the two arrays:
[[ 10. 11. 12.]
 [ 13. 14. 15.]
 [ 16. 17. 18.]]

Subtract the two arrays:
[[-10. -9. -8.]
 [ -7. -6. -5.]
 [ -4. -3. -2.]]

Multiply the two arrays:
[[ 0. 10. 20.]
 [ 30. 40. 50.]
 [ 60. 70. 80.]]

Divide the two arrays:
[[ 0. 0.1 0.2]
 [ 0.3 0.4 0.5]
 [ 0.6 0.7 0.8]]
```

ARRAY FUNCTIONS:

Array functions in python are defined as functions that will have arrays as parameters to the function and perform set of instructions to perform a particular task on input parameters to achieve a particular task is called array functions in python. Array functions will take an array as an

argument, it will be implemented in such a way that it will operate on a different number of objects.

For example, we can define a function that will compute the average of an array of integers or float data types. Python by default takes arguments passed to the program will be stored in sys.argv[] which is an array of strings as the argument.

Methods of Python Array Functions

Given below are the different methods in python which will be used to perform different types of operations on array functions:

1. array(datatype, valuelist)

The above function, an array is used to create an array in python which accepts parameters as data type and value-list where data type is the type of the value-list like integer, decimal, float, etc. The value-list is a list of values of that particular data type.

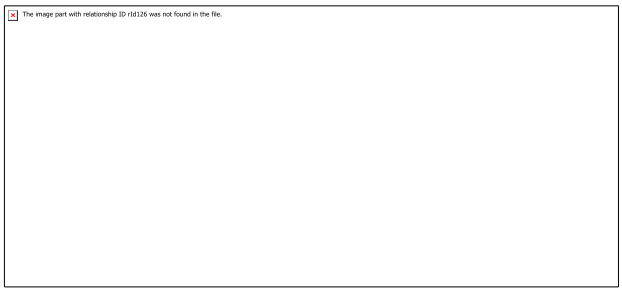
Example:

Code:

```
a = array('i',[1,2,3,4])
print(a)
```

The above example will create an array of 'integer' data type with values 1, 2, 3, 4 as its elements.

Output:



2. insert(pos, value)

The above function, insert() is used to insert an element to the array at a specific index or position. It will take pos, and value as its parameters where pos variables tell the position and value is the value need to insert into array.

DATA PROCESSING USING ARRAYS:

```
import numpy as np

a = np.array([1, 3, 5, 7])

np.savetxt('test1.txt', a, fmt='%d')

a2 = np.loadtxt('test1.txt', dtype=int)

print(a == a2)
```

Output:

```
[ True  True  True  True]
```

.

SAVING AND ARRAY:

Saving a numpy array stores it in a file and allows future programs to utilize it.

USE numpy.save() TO SAVE AN ARRAY

Call numpy.save(file_name, array) to save a numpy array to a file named file_name.
Use numpy.load(file_name) to load the saved array from file_name.

```
an_array = np.array([[1, 2, 3], [4, 5, 6]])
np.save("sample.npy", an_array)

loaded_array = np.load("sample.npy")
print(loaded_array)
```

OUTPUT

```
[[1 2 3]
 [4 5 6]]
```

LOADING AN ARRAY:

NumPy Linear Algebra

Numpy provides the following functions to perform the different algebraic calculations on the input data.

SN	Function	Definition
1	dot()	It is used to calculate the dot product of two arrays.
2	vdot()	It is used to calculate the dot product of two vectors.
3	inner()	It is used to calculate the inner product of two arrays.
4	matmul()	It is used to calculate the matrix multiplication of two arrays.
5	det()	It is used to calculate the determinant of a matrix.
6	solve()	It is used to solve the linear matrix equation.
7	inv()	It is used to calculate the multiplicative inverse of the matrix.

numpy.dot() function

This function is used to return the dot product of the two matrices. It is similar to the matrix multiplication. Consider the following example.

Example

```
1. import numpy as np
2. a = np.array([[100,200],[23,12]])
3. b = np.array([[10,20],[12,21]])
4. dot = np.dot(a,b)
5. print(dot)
```

Output:

```
[[3400 6200]
 [ 374  712]]
```

The dot product is calculated as:

[100 * 10 + 200 * 12, 100 * 20 + 200 * 21] [23*10+12*12, 23*20 + 12*21]

NUMPY RANDOM NUMBERS:

Random number does NOT mean a different number every time. Random means something that can not be predicted logically.

Pseudo Random and True Random.

Computers work on programs, and programs are definitive set of instructions. So it means there must be some algorithm to generate a random number as well.

If there is a program to generate random number it can be predicted, thus it is not truly random.

Random numbers generated through a generation algorithm are called *pseudo random*.

Generate Random Number

NumPy offers the random module to work with random numbers.

Example

Generate a random integer from 0 to 100:

```
from numpy import random

x = random.randint(100)

print(x)
```

OUTPUT:

```
91
```

Unit-4

Data Analysis with Pandas: An overview of the Pandas package, The Pandas data structureSeries, The DataFrame, The Essential Basic Functionality: Reindexing and altering labels , Head and tail, Binary operations, Functional statistics , Function application Sorting, Indexing and selecting data, Computational tools, Working with Missing Data, Advanced Uses of Pandas for Data Analysis - Hierarchical indexing, The Panel data

Panda

pandas is a Python package providing fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, **real-world** data analysis in Python. Additionally, it has the broader goal of becoming **the most powerful and flexible open source data analysis/manipulation tool available in any language**. It is already well on its way toward this goal.

pandas is well suited for many different kinds of data:

- Tabular data with heterogeneously-typed columns, as in an SQL table or Excel spreadsheet
- Ordered and unordered (not necessarily fixed-frequency) time series data.
- Arbitrary matrix data (homogeneously typed or heterogeneous) with row and column labels
- Any other form of observational / statistical data sets. The data need not be labeled at all to be placed into a pandas data structure

The two primary data structures of pandas, **Series** (1-dimensional) and **DataFrame** (2-dimensional), handle the vast majority of typical use cases in finance, statistics, social science, and many areas of engineering.

pandas is built on top of NumPy and is intended to integrate well within a scientific computing environment with many other 3rd party libraries.

What pandas does well:

- Easy handling of **missing data** (represented as NaN) in floating point as well as non-floating point data
- Size mutability: columns can be **inserted and deleted** from DataFrame and higher dimensional objects
- Automatic and explicit **data alignment**: objects can be explicitly aligned to a set of labels, or the user can simply ignore the labels and let Series, DataFrame, etc. automatically align the data for you in computations
- Powerful, flexible **group by** functionality to perform split-apply-combine operations on data sets, for both aggregating and transforming data
- Make it **easy to convert** ragged, differently-indexed data in other Python and NumPy data structures into DataFrame objects
- Intelligent label-based **slicing**, **fancy indexing**, and **subsetting** of large data sets
- Intuitive **merging** and **joining** data sets
- Flexible **reshaping** and pivoting of data sets
- **Hierarchical** labeling of axes (possible to have multiple labels per tick)
- Robust IO tools for loading data from **flat files** (CSV and delimited), Excel files, databases, and saving / loading data from the ultrafast **HDF5 format**
- **Time series**-specific functionality: date range generation and frequency conversion, moving window statistics, date shifting, and lagging.

Data structures

Dimensions	Name	Description
1	Series	1D labeled homogeneously-typed array
2	DataFrame	General 2D labeled, size-mutable tabular structure with potentially heterogeneously-typed column

import NumPy and load pandas into your namespace:

```
In [1]: import numpy as np

In [2]: import pandas as pd
```

Series

Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floating point numbers, Python objects, etc.). The axis labels are collectively referred to as the **index**. The basic method to create a Series is to call:

```
>>> s = pd.Series(data, index=index)
```

Here, data can be many different things:

- a Python dict
- an ndarray
- a scalar value

The passed **index** is a list of axis labels. Thus, this separates into a few cases depending on what **data is**:

From ndarray

If data is an ndarray, **index** must be the same length as **data**. If no index is passed, one will be created having values [0, ..., len(data) - 1].

```
In [3]: s = pd.Series(np.random.randn(5), index=["a", "b", "c", "d", "e"])

In [4]: s
Out[4]:
a   0.469112
b  -0.282863
c  -1.509059
d  -1.135632
e   1.212112
dtype: float64

In [5]: s.index
Out[5]: Index(['a', 'b', 'c', 'd', 'e'], dtype='object')

In [6]: pd.Series(np.random.randn(5))
Out[6]:
0  -0.173215
1   0.119209
2  -1.044236
3  -0.861849
4  -2.104569
dtype: float64
```

pandas supports non-unique index values. If an operation that does not support duplicate index values is attempted, an exception will be raised at that time.

From dict

Series can be instantiated from dicts:

```
In [7]: d = {"b": 1, "a": 0, "c": 2}

In [8]: pd.Series(d)
Out[8]:
b  1
a  0
c  2
dtype: int64
```

If an index is passed, the values in data corresponding to the labels in the index will be pulled out.

```
In [9]: d = {"a": 0.0, "b": 1.0, "c": 2.0}

In [10]: pd.Series(d)
Out[10]:
a  0.0
b  1.0
c  2.0
dtype: float64

In [11]: pd.Series(d, index=["b", "c", "d", "a"])
Out[11]:
b  1.0
c  2.0
d  NaN
a  0.0
dtype: float64
```

NaN (not a number) is the standard missing data marker used in pandas.

From scalar value

If data is a scalar value, an index must be provided. The value will be repeated to match the length of **index**.

```
In [12]: pd.Series(5.0, index=["a", "b", "c", "d", "e"])
Out[12]:
a    5.0
b    5.0
c    5.0
d    5.0
e    5.0
dtype: float64
```

Series is ndarray-like

Series acts very similarly to a ndarray, and is a valid argument to most NumPy functions. However, operations such as slicing will also slice the index.

```
In [13]: s[0]
Out[13]: 0.4691122999071863

In [14]: s[:3]
Out[14]:
a    0.469112
b   -0.282863
c   -1.509059
dtype: float64

In [15]: s[s > s.median()]
Out[15]:
a    0.469112
e    1.212112
dtype: float64

In [16]: s[[4, 3, 1]]
Out[16]:
e    1.212112
d   -1.135632
b   -0.282863
dtype: float64

In [17]: np.exp(s)
Out[17]:
a    1.598575
b    0.753623
c    0.221118
d    0.321219
```

```
e    3.360575
dtype: float64
```

like a NumPy array, a pandas Series has a **dtype**.

```
In [18]: s.dtype
Out[18]: dtype('float64')
```

This is often a NumPy dtype. However, pandas and 3rd-party libraries extend NumPy's type system in a few places, in which case the dtype would be an **ExtensionDtype**. Some examples within pandas are Categorical data and Nullable integer data type. See dtypes for more.

If you need the actual array backing a Series, use **Series.array**.

```
In [19]: s.array
Out[19]:
<PandasArray>
[ 0.4691122999071863, -0.2828633443286633, -1.5090585031735124,
 -1.1356323710171934,  1.2121120250208506]
Length: 5, dtype: float64
```

Accessing the array can be useful when you need to do some operation without the index (to disable automatic alignment, for example).

Series.array will always be an **ExtensionArray**. Briefly, an ExtensionArray is a thin wrapper around one or more *concrete* arrays like a **numpy.ndarray**. pandas knows how to take an ExtensionArray and store it in a Series or a column of a DataFrame. See dtypes for more.

While Series is ndarray-like, if you need an *actual* ndarray, then use **Series.to_numpy()**.

```
In [20]: s.to_numpy()
Out[20]: array([ 0.4691, -0.2829, -1.5091, -1.1356,  1.2121])
```

Even if the Series is backed by a **ExtensionArray**, **Series.to_numpy()** will return a NumPy ndarray.

Series is dict-like

A Series is like a fixed-size dict in that you can get and set values by index label:

```
In [21]: s["a"]
Out[21]: 0.4691122999071863

In [22]: s["e"] = 12.0

In [23]: s
Out[23]:
a     0.469112
b    -0.282863
c    -1.509059
d    -1.135632
e    12.000000
dtype: float64

In [24]: "e" in s
Out[24]: True

In [25]: "f" in s
Out[25]: False
```

If a label is not contained, an exception is raised:

```
>>> s["f"]
KeyError: 'f'
```

Using the get method, a missing label will return None or specified default:

```
In [26]: s.get("f")

In [27]: s.get("f", np.nan)
Out[27]: nan
```

.

Vectorized operations and label alignment with Series

When working with raw NumPy arrays, looping through value-by-value is usually not necessary. The same is true when working with Series in pandas. Series can also be passed into most NumPy methods expecting an ndarray.

```
In [28]: s + s
Out[28]:
```

```
a    0.938225
b   -0.565727
c   -3.018117
d   -2.271265
e   24.000000
dtype: float64

In [29]: s * 2
Out[29]:
a    0.938225
b   -0.565727
c   -3.018117
d   -2.271265
e   24.000000
dtype: float64

In [30]: np.exp(s)
Out[30]:
a       1.598575
b       0.753623
c       0.221118
d       0.321219
e  162754.791419
dtype: float64
```

A key difference between Series and ndarray is that operations between Series automatically align the data based on label. Thus, you can write computations without giving consideration to whether the Series involved have the same labels.

```
In [31]: s[1:] + s[:-1]
Out[31]:
a       NaN
b  -0.565727
c  -3.018117
d  -2.271265
e       NaN
dtype: float64
```

Name attribute

Series can also have a name attribute:

```
In [32]: s = pd.Series(np.random.randn(5), name="something")

In [33]: s
Out[33]:
0   -0.494929
1    1.071804
2    0.721555
3   -0.706771
4   -1.039575
Name: something, dtype: float64

In [34]: s.name
Out[34]: 'something'
```

The Series name will be assigned automatically in many cases, in particular when taking 1D slices of DataFrame as you will see below.

we can rename a Series with the **pandas.Series.rename()** method.

```
In [35]: s2 = s.rename("different")

In [36]: s2.name
Out[36]: 'different'
```

Note that s and s2 refer to different objects.

DataFrame

DataFrame is a 2-dimensional labeled data structure with columns of potentially different types. You can think of it like a spreadsheet or SQL table, or a dict of Series objects. It is generally the most commonly used pandas object. Like Series, DataFrame accepts many different kinds of input:

- Dict of 1D ndarrays, lists, dicts, or Series
- 2-D numpy.ndarray
- Structured or record ndarray
- A Series
- Another DataFrame

Along with the data, you can optionally pass **index** (row labels) and **columns** (column labels) arguments. If you pass an index and / or columns, you are guaranteeing the index and / or columns of the resulting DataFrame. Thus, a dict of Series plus a specific index will discard all data not matching up to the passed index.

If axis labels are not passed, they will be constructed from the input data based on common sense rules.

From dict of Series or dicts

The resulting **index** will be the **union** of the indexes of the various Series. If there are any nested dicts, these will first be converted to Series. If no columns are passed, the columns will be the ordered list of dict keys.

```
In [37]: d = {
   ....:    "one": pd.Series([1.0, 2.0, 3.0], index=["a", "b", "c"]),
   ....:    "two": pd.Series([1.0, 2.0, 3.0, 4.0], index=["a", "b", "c", "d"]),
   ....: }
   ....:

In [38]: df = pd.DataFrame(d)

In [39]: df
Out[39]:
   one  two
a  1.0  1.0
b  2.0  2.0
c  3.0  3.0
d  NaN  4.0

In [40]: pd.DataFrame(d, index=["d", "b", "a"])
Out[40]:
   one  two
d  NaN  4.0
b  2.0  2.0
a  1.0  1.0

In [41]: pd.DataFrame(d, index=["d", "b", "a"], columns=["two", "three"])
Out[41]:
   two three
d  4.0   NaN
b  2.0   NaN
a  1.0   NaN
```

The row and column labels can be accessed respectively by accessing the **index** and **columns** attributes:

From dict of ndarrays / lists

The ndarrays must all be the same length. If an index is passed, it must clearly also be the same length as the arrays. If no index is passed, the result will be range(n), where n is the array length.

```
In [44]: d = {"one": [1.0, 2.0, 3.0, 4.0], "two": [4.0, 3.0, 2.0, 1.0]}

In [45]: pd.DataFrame(d)
Out[45]:
   one  two
0  1.0  4.0
1  2.0  3.0
2  3.0  2.0
3  4.0  1.0

In [46]: pd.DataFrame(d, index=["a", "b", "c", "d"])
Out[46]:
   one  two
a  1.0  4.0
b  2.0  3.0
c  3.0  2.0
d  4.0  1.0
```

From structured or record array

This case is handled identically to a dict of arrays.

```
In [47]: data = np.zeros((2,), dtype=[("A", "i4"), ("B", "f4"), ("C", "a10")])

In [48]: data[:] = [(1, 2.0, "Hello"), (2, 3.0, "World")]

In [49]: pd.DataFrame(data)
Out[49]:
   A    B       C
0  1  2.0  b'Hello'
1  2  3.0  b'World'

In [50]: pd.DataFrame(data, index=["first", "second"])
Out[50]:
        A    B       C
first   1  2.0  b'Hello'
second  2  3.0  b'World'

In [51]: pd.DataFrame(data, columns=["C", "A", "B"])
Out[51]:
```

```
      C A   B
0 b'Hello' 1 2.0
1 b'World' 2 3.0
```

.

From a list of dicts

```
In [52]: data2 = [{"a": 1, "b": 2}, {"a": 5, "b": 10, "c": 20}]

In [53]: pd.DataFrame(data2)
Out[53]:
  a  b   c
0 1  2  NaN
1 5  10 20.0

In [54]: pd.DataFrame(data2, index=["first", "second"])
Out[54]:
       a  b   c
first  1  2  NaN
second 5  10 20.0

In [55]: pd.DataFrame(data2, columns=["a", "b"])
Out[55]:
  a  b
0 1  2
1 5  10
```

From a dict of tuples

You can automatically create a MultiIndexed frame by passing a tuples dictionary.

```
In [56]: pd.DataFrame(
   ....:    {
   ....:       ("a", "b"): {("A", "B"): 1, ("A", "C"): 2},
   ....:       ("a", "a"): {("A", "C"): 3, ("A", "B"): 4},
   ....:       ("a", "c"): {("A", "B"): 5, ("A", "C"): 6},
   ....:       ("b", "a"): {("A", "C"): 7, ("A", "B"): 8},
   ....:       ("b", "b"): {("A", "D"): 9, ("A", "B"): 10},
   ....:    }
   ....: )
   ....:
```

```
Out[56]:
    a         b
    b   a   c   a   b
A B 1.0 4.0 5.0 8.0 10.0
  C 2.0 3.0 6.0 7.0  NaN
  D NaN NaN NaN NaN  9.0
```

DataFrame.from_records

DataFrame.from_records takes a list of tuples or an ndarray with structured dtype. It works analogously to the normal DataFrame constructor, except that the resulting DataFrame index may be a specific field of the structured dtype. For example:

```
In [67]: data
Out[67]:
array([(1, 2., b'Hello'), (2, 3., b'World')],
    dtype=[('A', '<i4'), ('B', '<f4'), ('C', 'S10')])

In [68]: pd.DataFrame.from_records(data, index="C")
Out[68]:
       A   B
C
b'Hello'  1  2.0
b'World'  2  3.0
```

Column selection, addition, deletion

You can treat a DataFrame semantically like a dict of like-indexed Series objects. Getting, setting, and deleting columns works with the same syntax as the analogous dict operations:

```
In [69]: df["one"]
Out[69]:
a    1.0
b    2.0
c    3.0
d    NaN
Name: one, dtype: float64

In [70]: df["three"] = df["one"] * df["two"]

In [71]: df["flag"] = df["one"] > 2
```

```
In [72]: df
Out[72]:
  one  two  three  flag
a 1.0  1.0   1.0  False
b 2.0  2.0   4.0  False
c 3.0  3.0   9.0   True
d NaN  4.0   NaN  False
```

Columns can be deleted or popped like with a dict:

```
In [73]: del df["two"]

In [74]: three = df.pop("three")

In [75]: df
Out[75]:
  one   flag
a 1.0  False
b 2.0  False
c 3.0   True
d NaN  False
```

When inserting a scalar value, it will naturally be propagated to fill the column:

```
In [76]: df["foo"] = "bar"

In [77]: df
Out[77]:
  one   flag  foo
a 1.0  False  bar
b 2.0  False  bar
c 3.0   True  bar
d NaN  False  bar
```

When inserting a Series that does not have the same index as the DataFrame, it will be conformed to the DataFrame's index:

```
In [78]: df["one_trunc"] = df["one"][:2]

In [79]: df
Out[79]:
  one   flag  foo  one_trunc
```

```
a  1.0  False  bar     1.0
b  2.0  False  bar     2.0
c  3.0   True  bar     NaN
d  NaN  False  bar     NaN
```

Indexing / selection

The basics of indexing are as follows:

Operation	Syntax	Result
Select column	df[col]	Series
Select row by label	df.loc[label]	Series
Select row by integer location	df.iloc[loc]	Series
Slice rows	df[5:10]	DataFrame
Select rows by boolean vector	df[bool_vec]	DataFrame

Row selection, for example, returns a Series whose index is the columns of the DataFrame:

```
In [89]: df.loc["b"]
Out[89]:
one         2.0
bar         2.0
flag      False
foo         bar
one_trunc   2.0
Name: b, dtype: object

In [90]: df.iloc[2]
Out[90]:
one         3.0
bar         3.0
flag       True
foo         bar
one_trunc   NaN
Name: c, dtype: object
```

For a more exhaustive treatment of sophisticated label-based indexing and slicing, see the section on indexing. We will address the fundamentals of reindexing / conforming to new sets of labels in the section on reindexing.

Data alignment and arithmetic

Data alignment between DataFrame objects automatically align on **both the columns and the index (row labels)**. Again, the resulting object will have the union of the column and row labels.

```
In [91]: df = pd.DataFrame(np.random.randn(10, 4), columns=["A", "B", "C", "D"])

In [92]: df2 = pd.DataFrame(np.random.randn(7, 3), columns=["A", "B", "C"])

In [93]: df + df2
Out[93]:
          A         B         C   D
0  0.045691 -0.014138  1.380871 NaN
1 -0.955398 -1.501007  0.037181 NaN
2 -0.662690  1.534833 -0.859691 NaN
3 -2.452949  1.237274 -0.133712 NaN
4  1.414490  1.951676 -2.320422 NaN
5 -0.494922 -1.649727 -1.084601 NaN
6 -1.047551 -0.748572 -0.805479 NaN
7       NaN       NaN       NaN NaN
8       NaN       NaN       NaN NaN
9       NaN       NaN       NaN NaN
```

:

```
In [94]: df - df.iloc[0]
Out[94]:
          A         B         C         D
0  0.000000  0.000000  0.000000  0.000000
1 -1.359261 -0.248717 -0.453372 -1.754659
2  0.253128  0.829678  0.010026 -1.991234
3 -1.311128  0.054325 -1.724913 -1.620544
4  0.573025  1.500742 -0.676070  1.367331
5 -1.741248  0.781993 -1.241620 -2.053136
6 -1.240774 -0.869551 -0.153282  0.000430
7 -0.743894  0.411013 -0.929563 -0.282386
8 -1.194921  1.320690  0.238224 -1.482644
9  2.293786  1.856228  0.773289 -1.446531
```

For explicit control over the matching and broadcasting behavior, see the section on flexible binary operations.

Operations with scalars are just as you would expect:

```
In [95]: df * 5 + 2
```

Out[95]:

	A	B	C	D
0	3.359299	-0.124862	4.835102	3.381160
1	-3.437003	-1.368449	2.568242	-5.392133
2	4.624938	4.023526	4.885230	-6.575010
3	-3.196342	0.146766	-3.789461	-4.721559
4	6.224426	7.378849	1.454750	10.217815
5	-5.346940	3.785103	-1.373001	-6.884519
6	-2.844569	-4.472618	4.068691	3.383309
7	-0.360173	1.930201	0.187285	1.969232
8	-2.615303	6.478587	6.026220	-4.032059
9	14.828230	9.156280	8.701544	-3.851494

In [96]: 1 / df

Out[96]:

	A	B	C	D
0	3.678365	-2.353094	1.763605	3.620145
1	-0.919624	-1.484363	8.799067	-0.676395
2	1.904807	2.470934	1.732964	-0.583090
3	-0.962215	-2.697986	-0.863638	-0.743875
4	1.183593	0.929567	-9.170108	0.608434
5	-0.680555	2.800959	-1.482360	-0.562777
6	-1.032084	-0.772485	2.416988	3.614523
7	-2.118489	-71.634509	-2.758294	-162.507295
8	-1.083352	1.116424	1.241860	-0.828904
9	0.389765	0.698687	0.746097	-0.854483

In [97]: df ** 4

Out[97]:

	A	B	C	D
0	0.005462	3.261689e-02	0.103370	5.822320e-03
1	1.398165	2.059869e-01	0.000167	4.777482e+00
2	0.075962	2.682596e-02	0.110877	8.650845e+00
3	1.166571	1.887302e-02	1.797515	3.265879e+00
4	0.509555	1.339298e+00	0.000141	7.297019e+00
5	4.661717	1.624699e-02	0.207103	9.969092e+00
6	0.881334	2.808277e+00	0.029302	5.858632e-03
7	0.049647	3.797614e-08	0.017276	1.433866e-09
8	0.725974	6.437005e-01	0.420446	2.118275e+00
9	43.329821	4.196326e+00	3.227153	1.875802e+00

Boolean operators work as well:

```
In [98]: df1 = pd.DataFrame({"a": [1, 0, 1], "b": [0, 1, 1]}, dtype=bool)

In [99]: df2 = pd.DataFrame({"a": [0, 1, 1], "b": [1, 1, 0]}, dtype=bool)

In [100]: df1 & df2
Out[100]:
      a     b
0  False  False
1  False   True
2   True  False

In [101]: df1 | df2
Out[101]:
     a    b
0  True  True
1  True  True
2  True  True

In [102]: df1 ^ df2
Out[102]:
      a     b
0  True   True
1  True  False
2 False   True

In [103]: -df1
Out[103]:
      a     b
0 False   True
1  True  False
2 False  False
```

Reindexing and altering labels

reindex() is the fundamental data alignment method in pandas. It is used to implement nearly all other features relying on label-alignment functionality. To *reindex* means to conform the data to match a given set of labels along a particular axis. This accomplishes several things:

- Reorders the existing data to match a new set of labels

- Inserts missing value (NA) markers in label locations where no data for that label existed
- If specified, **fill** data for missing labels using logic (highly relevant to working with time series data)

Here is a simple example:

```
In [1]: s = pd.Series(np.random.randn(5), index=['a', 'b', 'c', 'd', 'e'])

In [2]: s
Out[2]:
a    0.734560
b   -0.445120
c   -0.703433
d    0.320412
e    0.185202
dtype: float64

In [3]: s.reindex(['e', 'b', 'f', 'd'])
Out[3]:
e    0.185202
b   -0.445120
f         NaN
d    0.320412
dtype: float64
```

Here, the `f` label was not contained in the Series and hence appears as `NaN` in the result.

With a DataFrame, you can simultaneously reindex the index and columns:

```
In [4]: df
Out[4]:
        one    three      two
a  0.851097      NaN -0.429037
b  0.266049 -0.330979  0.963385
c  1.117346 -0.409168  2.243459
d       NaN -0.305334 -0.432789

In [5]: df.reindex(index=['c', 'f', 'b'], columns=['three', 'two', 'one'])
Out[5]:
      three      two      one
c -0.409168  2.243459  1.117346
f      NaN      NaN      NaN
```

```
b -0.330979  0.963385  0.266049
```

For convenience, you may utilize the **reindex_axis()** method, which takes the labels and a keyword `axis` parameter.

Note that the `Index` objects containing the actual axis labels can be **shared** between objects. So if we have a Series and a DataFrame, the following can be done:

```
In [6]: rs = s.reindex(df.index)

In [7]: rs
Out[7]:
a    0.734560
b   -0.445120
c   -0.703433
d    0.320412
dtype: float64

In [8]: rs.index is df.index
Out[8]: True
```

Aligning objects with each other with `align`

The **align()** method is the fastest way to simultaneously align two objects. It supports a `join` argument (related to joining and merging):

- `join='outer'` : take the union of the indexes (default)
- `join='left'` : use the calling object's index
- `join='right'` : use the passed object's index
- `join='inner'` : intersect the indexes

It returns a tuple with both of the reindexed Series:

```
In [12]: s = pd.Series(np.random.randn(5), index=['a', 'b', 'c', 'd', 'e'])

In [13]: s1 = s[:4]

In [14]: s2 = s[1:]

In [15]: s1.align(s2)
```

```
Out[15]:
(a    0.498698
 b   -0.643722
 c   -0.028228
 d    0.070209
 e         NaN
 dtype: float64, a         NaN
 b   -0.643722
 c   -0.028228
 d    0.070209
 e   -0.791176
 dtype: float64)

In [16]: s1.align(s2, join='inner')
Out[16]:
(b   -0.643722
 c   -0.028228
 d    0.070209
 dtype: float64, b   -0.643722
 c   -0.028228
 d    0.070209
 dtype: float64)

In [17]: s1.align(s2, join='left')
Out[17]:
(a    0.498698
 b   -0.643722
 c   -0.028228
 d    0.070209
 dtype: float64, a         NaN
 b   -0.643722
 c   -0.028228
 d    0.070209
 dtype: float64)
```

For DataFrames, the join method will be applied to both the index and the columns by default:

```
In [18]: df.align(df2, join='inner')
Out[18]:
(        one       two
 a  0.851097 -0.429037
 b  0.266049  0.963385
 c  1.117346  2.243459,        one       two
 a  0.851097 -0.429037
 b  0.266049  0.963385
```

```
 c  1.117346  2.243459)
```

You can also pass an `axis` option to only align on the specified axis:

```
In [19]: df.align(df2, join='inner', axis=0)
Out[19]:
(      one    three      two
 a  0.851097      NaN -0.429037
 b  0.266049 -0.330979  0.963385
 c  1.117346 -0.409168  2.243459,       one      two
 a  0.851097 -0.429037
 b  0.266049  0.963385
 c  1.117346  2.243459)
```

If you pass a Series to **DataFrame.align()**, you can choose to align both objects either on the DataFrame's index or columns using the `axis` argument:

```
In [20]: df.align(df2.ix[0], axis=1)
Out[20]:
(      one    three      two
 a  0.851097      NaN -0.429037
 b  0.266049 -0.330979  0.963385
 c  1.117346 -0.409168  2.243459
 d       NaN -0.305334 -0.432789, one      0.851097
 three        NaN
 two    -0.429037
 Name: a, dtype: float64)
```

Filling while reindexing

reindex() takes an optional parameter `method` which is a filling method chosen from the following table:

Method	Action
pad / ffill	Fill values forward
bfill / backfill	Fill values backward
nearest	Fill from the nearest index value

We illustrate these fill methods on a simple Series:

```
In [21]: rng = pd.date_range('1/3/2000', periods=8)

In [22]: ts = pd.Series(np.random.randn(8), index=rng)

In [23]: ts2 = ts[[0, 3, 6]]

In [24]: ts
Out[24]:
2000-01-03    0.393495
2000-01-04    2.410230
2000-01-05   -0.368339
2000-01-06   -1.934392
2000-01-07    2.398912
2000-01-08    0.521658
2000-01-09   -2.389278
2000-01-10    0.395639
Freq: D, dtype: float64

In [25]: ts2
Out[25]:
2000-01-03    0.393495
2000-01-06   -1.934392
2000-01-09   -2.389278
dtype: float64

In [26]: ts2.reindex(ts.index)
Out[26]:
2000-01-03    0.393495
2000-01-04         NaN
2000-01-05         NaN
2000-01-06   -1.934392
2000-01-07         NaN
2000-01-08         NaN
2000-01-09   -2.389278
2000-01-10         NaN
Freq: D, dtype: float64

In [27]: ts2.reindex(ts.index, method='ffill')
Out[27]:
2000-01-03    0.393495
2000-01-04    0.393495
2000-01-05    0.393495
2000-01-06   -1.934392
2000-01-07   -1.934392
2000-01-08   -1.934392
```

```
2000-01-09  -2.389278
2000-01-10  -2.389278
Freq: D, dtype: float64
```

```
In [28]: ts2.reindex(ts.index, method='bfill')
Out[28]:
2000-01-03   0.393495
2000-01-04  -1.934392
2000-01-05  -1.934392
2000-01-06  -1.934392
2000-01-07  -2.389278
2000-01-08  -2.389278
2000-01-09  -2.389278
2000-01-10        NaN
Freq: D, dtype: float64
```

```
In [29]: ts2.reindex(ts.index, method='nearest')
Out[29]:
2000-01-03   0.393495
2000-01-04   0.393495
2000-01-05  -1.934392
2000-01-06  -1.934392
2000-01-07  -1.934392
2000-01-08  -2.389278
2000-01-09  -2.389278
2000-01-10  -2.389278
Freq: D, dtype: float64
```

These methods require that the indexes are **ordered** increasing or decreasing.

Note that the same result could have been achieved using fillna (except for `method='nearest'`) or interpolate:

```
In [30]: ts2.reindex(ts.index).fillna(method='ffill')
Out[30]:
2000-01-03   0.393495
2000-01-04   0.393495
2000-01-05   0.393495
2000-01-06  -1.934392
2000-01-07  -1.934392
2000-01-08  -1.934392
2000-01-09  -2.389278
2000-01-10  -2.389278
Freq: D, dtype: float64
```

reindex() will raise a ValueError if the index is not monotonic increasing or decreasing. **fillna()** and **interpolate()** will not make any checks on the order of the index.

Limits on filling while reindexing

The limit and tolerance arguments provide additional control over filling while reindexing. Limit specifies the maximum count of consecutive matches:

```
In [31]: ts2.reindex(ts.index, method='ffill', limit=1)
Out[31]:
2000-01-03  0.393495
2000-01-04  0.393495
2000-01-05       NaN
2000-01-06 -1.934392
2000-01-07 -1.934392
2000-01-08       NaN
2000-01-09 -2.389278
2000-01-10 -2.389278
Freq: D, dtype: float64
```

In contrast, tolerance specifies the maximum distance between the index and indexer values:

```
In [32]: ts2.reindex(ts.index, method='ffill', tolerance='1 day')
Out[32]:
2000-01-03  0.393495
2000-01-04  0.393495
2000-01-05       NaN
2000-01-06 -1.934392
2000-01-07 -1.934392
2000-01-08       NaN
2000-01-09 -2.389278
2000-01-10 -2.389278
Freq: D, dtype: float64
```

Dropping labels from an axis

A method closely related to reindex is the **drop()** function. It removes a set of labels from an axis:

```
In [33]: df
Out[33]:
      one    three      two
a  0.851097      NaN -0.429037
b  0.266049 -0.330979  0.963385
c  1.117346 -0.409168  2.243459
d      NaN -0.305334 -0.432789

In [34]: df.drop(['a', 'd'], axis=0)
Out[34]:
      one    three      two
b  0.266049 -0.330979  0.963385
c  1.117346 -0.409168  2.243459

In [35]: df.drop(['one'], axis=1)
Out[35]:
    three      two
a      NaN -0.429037
b -0.330979  0.963385
c -0.409168  2.243459
d -0.305334 -0.432789
```

Note that the following also works, but is a bit less obvious / clean:

```
In [36]: df.reindex(df.index.difference(['a', 'd']))
Out[36]:
      one    three      two
b  0.266049 -0.330979  0.963385
c  1.117346 -0.409168  2.243459
```

Renaming / mapping labels

The **rename()** method allows you to relabel an axis based on some mapping (a dict or Series) or an arbitrary function.

```
In [37]: s
Out[37]:
a   0.498698
b   -0.643722
c   -0.028228
d    0.070209
e   -0.791176
dtype: float64
```

```
In [38]: s.rename(str.upper)
Out[38]:
A   0.498698
B  -0.643722
C  -0.028228
D   0.070209
E  -0.791176
dtype: float64
```

If you pass a function, it must return a value when called with any of the labels (and must produce a set of unique values). A dict or Series can also be used:

```
In [39]: df.rename(columns={'one' : 'foo', 'two' : 'bar'},
   ....:           index={'a' : 'apple', 'b' : 'banana', 'd' : 'durian'})
   ....:
Out[39]:
            foo     three      bar
apple   0.851097       NaN -0.429037
banana  0.266049 -0.330979  0.963385
c       1.117346 -0.409168  2.243459
durian       NaN -0.305334 -0.432789
```

If the mapping doesn't include a column/index label, it isn't renamed. Also extra labels in the mapping don't throw an error.

The **rename()** method also provides an `inplace` named parameter that is by default `False` and copies the underlying data. Pass `inplace=True` to rename the data in place.

rename() also accepts a scalar or list-like for altering the `Series.name` attribute.

```
In [40]: s.rename("scalar-name")
Out[40]:
a   0.498698
b  -0.643722
c  -0.028228
d   0.070209
e  -0.791176
Name: scalar-name, dtype: float64
```

The Panel class has a related **rename_axis()** class which can rename any of its three axes.

Pandas: Head and Tail

Complete list of Head and Tail with examples:

```
import numpy as np
import pandas as pd
```

In [3]:
```
index = pd.date_range('1/1/2019', periods=6)
```

In [4]:
```
s = pd.Series(np.random.randn(6), index=['a', 'b', 'c', 'd', 'e','f'])
```

In [7]:
```
df = pd.DataFrame(np.random.randn(6, 4), index=index,
            columns=['P', 'Q', 'R','S'])
```

To view a small sample of a Series or DataFrame object, use the head() and tail() methods. The default number
of elements to display is five, but you may pass a custom number.

In [8]:
```
long_series = pd.Series(np.random.randn(800))
```

In [9]:
```
long_series.head()
```

Out[9]:
```
0   1.298944
1  -0.677865
2   0.414972
3   0.318461
4  -0.869943
dtype: float64
```

In [10]:
```
long_series.tail(3)
```

Out[10]:
```
797   0.374511
798  -0.721997
799   0.587586
dtype: float64
```

In [8]:
```
import numpy as np
import pandas as pd
```

In [9]:
```
s = pd.Series(np.random.randn(5), index=['white', 'black', 'blue', 'red', 'green'])
```

In [10]:
```
df = pd.DataFrame({'color':['white', 'black', 'blue', 'red', 'green']})
```

In [13]:

df

Out[13]:

	Color
0	White
1	Black
2	Blue
3	Red
4	Green

In [16]:
df.tail(4)

Out[16]:

	Color
1	Black
2	Blue
3	Red
4	Green

Binary operations¶

Elementwise bit operations

bitwise_and(x1, x2, /[, out, where, ...])	Compute the bit-wise AND of two arrays element-wise.

bitwise_or(x1, x2, /[, out, where, casting, ...])	Compute the bit-wise OR of two arrays element-wise.
bitwise_xor(x1, x2, /[, out, where, ...])	Compute the bit-wise XOR of two arrays element-wise.
invert(x, /[, out, where, casting, order, ...])	Compute bit-wise inversion, or bit-wise NOT, element-wise.
left_shift(x1, x2, /[, out, where, casting, ...])	Shift the bits of an integer to the left.
right_shift(x1, x2, /[, out, where, ...])	Shift the bits of an integer to the right.

Bit packing

packbits(a, /[, axis, bitorder])	Packs the elements of a binary-valued array into bits in a uint8 array.
unpackbits(a, /[, axis, count, bitorder])	Unpacks elements of a uint8 array into a binary-valued output array.

Output formatting

binary_repr(num[, width])	Return the binary representation of the input number as a string.

Python Bitwise Operators with Syntax and Example

In this Python Bitwise Operators Bitwise AND, OR, XOR, Left-shift, Right-shift, and 1's complement Bitwise Operators in Python Programming.

```
>>> bin(5)
```

Output

```
'0b101'
```

```
>>> bin(7)
```

Output

```
'0b111'
```

Now let's try applying 'and' and '&' to 5 and 7.

```
>>> 5 and 7
```

Output

```
7
```

```
>>> 5&7
```

Output

```
5
```

You would have expected them to return the same thing, but they're not the same. One acts on the whole value, and one acts on each bit at once.

Actually, 'and' sees the value on the left. If it has a True Boolean value, it returns whatever value is on the right.

Otherwise, it returns False. So, here, 5 and 7 is the same as True and 7. Hence, it returns 7.

However, 5&7 is the same as 101&111. This results in 101, which is binary for 5. Let's look at each of these operators bit by bit (pun intended).

Let's move ahead with next Python Bitwise Operator

1. Python Bitwise AND (&) Operator

1 has a Boolean value of True, and 0 has that of False. Take a look at the following code.

```
>>> True/2
```

Output

```
0.5
```

```
>>> False*2
```

Output

```
0
```

This proves something. Now, the binary and (&) takes two values and performs an AND-ing on each pair of bits.

Let's take an example.

```
>>> 4 & 8
```

Binary for 4 is 0100, and that for 8 is 1000. So when we AND the corresponding bits, it gives us 0000, which is binary for 0. Hence, the output.

The following are the values when &-ing 0 and 1.

Python Bitwise Operators – AND Operators

0 & 0	0
0 & 1	0
1 & 0	0
1 & 1	1

As you can see, an &-ing returns 1 only if both bits are 1.

You cannot, however, & strings.

```
>>> '$'&'%'
```

Output

```
Traceback (most recent call last):File "<pyshell#30>", line 1, in <module>'$'&'%'

TypeError: unsupported operand type(s) for &: 'str' and 'str'
```

Since Boolean values True and False have equivalent integer values of 1 and 0, we can & them.

```
>>> False&True
```

Output

```
False
```

```
>>> True&True
```

Output

```
True
```

Let's try a few more combinations.

```
>>> 1&True
```

Output

```
1
```

```
>>> 1.0&1.0
```

Output

```
Traceback (most recent call last):File "<pyshell#36>", line 1, in <module>1.0&1.0

TypeError: unsupported operand type(s) for &: 'float' and 'float'
```

You can also type your numbers directly in binary, as we discussed in section 6a in our Python Numbers tutorial.

```
>>> 0b110 & 0b101
```

Output

```
4
```

Here, 110 is binary for 6, and 101 for 5. &-ing them, we get 100, which is binary for 4.

2. Python Bitwise OR (|) Operators

Now let's discuss Python Bitwise OR (|) Operator

Compared to &, this one returns 1 even if one of the two corresponding bits from the two operands is 1.

Python Bitwise Operators – OR Operators

0|0	0
0|1	1
1|0	1
1|1	1

```
>>> 6|1
```

Output

```
7
```

This is the same as the following.

```
>>> 0b110|0b001
```

Output

```
7
```

Let's see some more examples.

```
>>> True|False
```

Output

```
True
```

Let's move to another Python Bitwise Operator

3. Python Bitwise XOR (^) Operator

XOR (eXclusive OR) returns 1 if one operand is 0 and another is 1. Otherwise, it returns 0.

Python Bitwise Operators – XOR Operators

0^0	0
0^1	1
1^0	1
1^1	0

Let's take a few examples.

```
>>> 6^6
```

Here, this is the same as 0b110^0b110. This results in 0b000, which is binary for 0.

```
>>> 6^0
```

Output

```
6
```

This is equivalent to 0b110^0b000, which gives us 0b110. This is binary for 6.

```
>>> 6^3
```

Output

```
5
```

Here, 0b110^0b011 gives us 0b101, which is binary for 5.

Now let's discuss Bitwise 1's Complement (~)

4. Python Bitwise 1's Complement (~)

This one is a bit different from what we've studied so far. This operator takes a number's binary, and returns its one's complement.

For this, it flips the bits until it reaches the first 0 from right. ~x is the same as -x-1.

```
>>> ~2
```

Output

```
-3
```

```
>>> bin(2)
```

Output

```
'0b10'
```

```
>>> bin(-3)
```

Output

```
'-0b11'
```

To make it clear, we mention the binary values of both. Another example follows.

```
>>> ~45
```

Output

```
-46
```

```
>>> bin(45)
```

Output

```
'0b101101'
```

```
>>> bin(-46)
```

Output

```
'-0b101110'
```

5. Python Bitwise Left-Shift Operator (<<)

Finally, we arrive at left-shift and right-shift operators. The left-shift operator shifts the bits of the number by the specified number of places.

This means it adds 0s to the empty least-significant places now. Let's begin with an unusual example.

```
>>> True<<2
```

Output

```
4
```

Here, True has an equivalent integer value of 1. If we shift it by two places to the left, we get 100. This is binary for 4.

Now let's do it on integers.

```
>>> 2<<1
```

Output

```
4
```

10 shifted by one place to the left gives us 100, which is, again, 4.

```
>>> 3<<2
```

Output

```
12
```

Now, 11 shifted to the left by two places gives us 1100, which is binary for 12.

Now let's move to Next Python Bitwise Operator

6. Python Bitwise Right-Shift Operator (>>)

Now we'll see the same thing for right-shift. It shifts the bits to the right by the specified number of places.

This means that those many bits are lost now.

```
>>> 3>>1
```

Output

```
1
```

3 has a binary value of 11, which shifted one place to the right returns 1. But before closing on this tutorial, we'll take one last example.

Let's check what's the decimal value for 11111.

```
>>> int(ob11111)
```

Output

```
31
```

Now, let's shift it three places to the right.

```
>>> 31>>3
```

Output

```
3
```

Indexing and Selecting Data with Pandas

Indexing in Pandas :
Indexing in pandas means simply selecting particular rows and columns of data from a DataFrame. Indexing could mean selecting all the rows and some of the columns, some of the rows and all of the columns, or some of each of the rows and columns. Indexing can also be known as **Subset Selection**.

Selecting some rows and some columns

.

Name	Team	Number	Position	Age	Height	Weight	College	Salary
Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0
John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
R.J. Hunter	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0
Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
Amir Johnson	Boston Celtics	90.0	PF	29.0	6-9	240.0	NaN	12000000.0
Jordan Mickey	Boston Celtics	55.0	PF	21.0	6-8	235.0	LSU	1170960.0
Kelly Olynyk	Boston Celtics	41.0	C	25.0	7-0	238.0	Gonzaga	2165160.0
Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
Marcus Smart	Boston Celtics	36.0	PG	22.0	6-4	220.0	Oklahoma State	3431040.0
Jared Sullinger	Boston Celtics	7.0	C	24.0	6-9	260.0	Ohio State	2569260.0

Suppose we want to select columns Age, College and Salary for only rows with a labels Amir Johnson and Terry Rozier

Name	Team	Number	Position	Age	Height	Weight	College	Salary
Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0
John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
R.J. Hunter	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0
Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
Amir Johnson	Boston Celtics	90.0	PF	29.0	6-9	240.0	NaN	12000000.0
Jordan Mickey	Boston Celtics	55.0	PF	21.0	6-8	235.0	LSU	1170960.0
Kelly Olynyk	Boston Celtics	41.0	C	25.0	7-0	238.0	Gonzaga	2165160.0
Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
Marcus Smart	Boston Celtics	36.0	PG	22.0	6-4	220.0	Oklahoma State	3431040.0
Jared Sullinger	Boston Celtics	7.0	C	24.0	6-9	260.0	Ohio State	2569260.0

Our final DataFrame would look like this:

Name	Age	College	Salary
Amir Johnson	29.0	NaN	12000000.0
Terry Rozier	22.0	Louisville	1824360.0

Selecting some rows and all columns

Let's say we want to select row Amir Jhonson, Terry Rozier and John Holland with all columns in a dataframe.

Name	Team	Number	Position	Age	Height	Weight	College	Salary
Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0
John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
R.J. Hunter	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0
Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
Amir Johnson	Boston Celtics	90.0	PF	29.0	6-9	240.0	NaN	12000000.0
Jordan Mickey	Boston Celtics	55.0	PF	21.0	6-8	235.0	LSU	1170960.0
Kelly Olynyk	Boston Celtics	41.0	C	25.0	7-0	238.0	Gonzaga	2165160.0
Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
Marcus Smart	Boston Celtics	36.0	PG	22.0	6-4	220.0	Oklahoma State	3431040.0
Jared Sullinger	Boston Celtics	7.0	C	24.0	6-9	260.0	Ohio State	2569260.0

Our final DataFrame would look like this:

Name	Team	Number	Position	Age	Height	Weight	College	Salary
Amir Johnson	Boston Celtics	90.0	PF	29.0	6-9	240.0	NaN	12000000.0
Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN

Selecting some columns and all rows

Let's say we want to select columns Age, Height and Salary with all rows in a dataframe.

Name	Team	Number	Position	Age	Height	Weight	College	Salary
Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0
John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
R.J. Hunter	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0
Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
Amir Johnson	Boston Celtics	90.0	PF	29.0	6-9	240.0	NaN	12000000.0
Jordan Mickey	Boston Celtics	55.0	PF	21.0	6-8	235.0	LSU	1170960.0
Kelly Olynyk	Boston Celtics	41.0	C	25.0	7-0	238.0	Gonzaga	2165160.0
Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
Marcus Smart	Boston Celtics	36.0	PG	22.0	6-4	220.0	Oklahoma State	3431040.0
Jared Sullinger	Boston Celtics	7.0	C	24.0	6-9	260.0	Ohio State	2569260.0

Our final DataFrame would look like this:

Name	Age	Height	Salary
Avery Bradley	25.0	6-2	7730337.0
Jae Crowder	25.0	6-6	6796117.0
John Holland	27.0	6-5	NaN
R.J. Hunter	22.0	6-5	1148640.0
Jonas Jerebko	29.0	6-10	5000000.0
Amir Johnson	29.0	6-9	12000000.0
Jordan Mickey	21.0	6-8	1170960.0
Kelly Olynyk	25.0	7-0	2165160.0
Terry Rozier	22.0	6-2	1824360.0
Marcus Smart	22.0	6-4	3431040.0
Jared Sullinger	24.0	6-9	2569260.0

Pandas Indexing using [ ]

There are a lot of ways to pull the elements, rows, and columns from a DataFrame. There are some indexing method in Pandas which help in getting an element from a DataFrame. These indexing methods appear very similar but behave very differently. Pandas support four types of Multi-axes indexing they are:

- **Dataframe.[ ] ;** This function also known as indexing operator
- Dataframe.loc[ ] **:** This function is used for labels.
- Dataframe.iloc[ ] **:** This function is used for positions or integer based
- Dataframe.ix[] **:** This function is used for both label and integer based

Collectively, they are called the **indexers**. These are by far the most common ways to index data. These are four function which help in getting the elements, rows, and columns from a DataFrame.

Indexing a Dataframe using indexing operator [] :

Indexing operator is used to refer to the square brackets following an object.
The .loc and .iloc indexers also use the indexing operator to make selections. In this indexing operator to refer to df[].

Selecting a single columns

In order to select a single column, we simply put the name of the column in-between the brackets

```
# importing pandas package

import pandas as pd

# making data frame from csv file

data = pd.read_csv("nba.csv", index_col ="Name")

# retrieving columns by indexing operator

first = data["Age"]

print(first)
```

Output:

```
Name
Avery Bradley              25.0
Jae Crowder                25.0
John Holland               27.0
R.J. Hunter                22.0
Jonas Jerebko              29.0
Amir Johnson               29.0
Jordan Mickey              21.0
Kelly Olynyk               25.0
Terry Rozier               22.0
Marcus Smart               22.0
Jared Sullinger            24.0
Isaiah Thomas              27.0
                            ...
Joe Ingles                 28.0
Chris Johnson              26.0
Trey Lyles                 20.0
Shelvin Mack               26.0
Raul Neto                  24.0
Tibor Pleiss               26.0
Jeff Withey                26.0
NaN                         NaN
Name: Age, Length: 458, dtype: float64
```

Selecting multiple columns

In order to select multiple columns, we have to pass a list of columns in an indexing operator.

```
# importing pandas package
import pandas as pd
# making data frame from csv file
data = pd.read_csv("nba.csv", index_col ="Name")
# retrieving multiple columns by indexing operator
first = data[["Age", "College", "Salary"]]
first
```

Output:

Name	Age	College	Salary
Avery Bradley	25.0	Texas	7730337.0
Jae Crowder	25.0	Marquette	6796117.0
John Holland	27.0	Boston University	NaN
R.J. Hunter	22.0	Georgia State	1148640.0
Jonas Jerebko	29.0	NaN	5000000.0
Amir Johnson	29.0	NaN	12000000.0
Jordan Mickey	21.0	LSU	1170960.0
Kelly Olynyk	25.0	Gonzaga	2165160.0
Terry Rozier	22.0	Louisville	1824360.0
...	...	...	...
Joe Ingles	28.0	NaN	2050000.0
Chris Johnson	26.0	Dayton	981348.0
Trey Lyles	20.0	Kentucky	2239800.0
Shelvin Mack	26.0	Butler	2433333.0
Raul Neto	24.0	NaN	900000.0
Tibor Pleiss	26.0	NaN	2900000.0
Jeff Withey	26.0	Kansas	947276.0
NaN	NaN	NaN	NaN

458 rows × 3 columns

Indexing a DataFrame using .loc[ ] :

This function selects data by the **label** of the rows and columns. The df.loc indexer selects data in a different way than just the indexing operator. It can select subsets of rows or columns. It can also simultaneously select subsets of rows and columns.

Selecting a single row

In order to select a single row using .loc[], we put a single row label in a .loc function.

```
# importing pandas package

import pandas as pd
```

```
# making data frame from csv file
data = pd.read_csv("nba.csv", index_col ="Name")
# retrieving row by loc method
first = data.loc["Avery Bradley"]
second = data.loc["R.J. Hunter"]
print(first, "\n\n\n", second)
```

Output:
As shown in the output image, two series were returned since there was only one parameter both of the times.

```
Team        Boston Celtics
Number                   0
Position                PG
Age                     25
Height                 6-2
Weight                 180
College              Texas
Salary         7.73034e+06
Name: Avery Bradley, dtype: object


 Team        Boston Celtics
Number                   28
Position                 SG
Age                      22
Height                  6-5
Weight                  185
College       Georgia State
Salary          1.14864e+06
Name: R.J. Hunter, dtype: object
```

Selecting multiple rows

In order to select multiple rows, we put all the row labels in a list and pass that to .loc function.

```
import pandas as pd
# making data frame from csv file
data = pd.read_csv("nba.csv", index_col ="Name")
```

```
# retrieving multiple rows by loc method

first = data.loc[["Avery Bradley", "R.J. Hunter"]]

 print(first)
```

Output:

Name	Team	Number	Position	Age	Height	Weight	College	Salary
Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
R.J. Hunter	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0

Selecting two rows and three columns

In order to select two rows and three columns, we select a two rows which we want to select and three columns and put it in a separate list like this:

Dataframe.loc[["row1", "row2"], ["column1", "column2", "column3"]]

```
import pandas as pd

# making data frame from csv file

data = pd.read_csv("nba.csv", index_col ="Name")

# retrieving two rows and three columns by loc method

first = data.loc[["Avery Bradley", "R.J. Hunter"],

            ["Team", "Number", "Position"]]

 print(first)
```

Output:

```
                         Team  Number Position
Name
Avery Bradley  Boston Celtics     0.0       PG
R.J. Hunter    Boston Celtics    28.0       SG
```

Selecting all of the rows and some columns

In order to select all of the rows and some columns, we use single colon [:] to select all of rows and list of some columns which we want to select like this:
Dataframe.loc[[:, ["column1", "column2", "column3"]]

```
import pandas as pd

# making data frame from csv file

data = pd.read_csv("nba.csv", index_col ="Name")

# retrieving all rows and some columns by loc method

first = data.loc[:, ["Team", "Number", "Position"]]

 print(first)
```

Output:

```
                          Team  Number Position
Name
Avery Bradley     Boston Celtics     0.0       PG
Jae Crowder       Boston Celtics    99.0       SF
John Holland      Boston Celtics    30.0       SG
R.J. Hunter       Boston Celtics    28.0       SG
Jonas Jerebko     Boston Celtics     8.0       PF
Amir Johnson      Boston Celtics    90.0       PF
Jordan Mickey     Boston Celtics    55.0       PF
Kelly Olynyk      Boston Celtics    41.0        C
Terry Rozier      Boston Celtics    12.0       PG
Marcus Smart      Boston Celtics    36.0       PG
Jared Sullinger   Boston Celtics     7.0        C
.                            .       .        .
.                            .       .        .
.                            .       .        .
.                            .       .        .
.                            .       .        .
Rudy Gobert          Utah Jazz    27.0        C
Gordon Hayward       Utah Jazz    20.0       SF
Rodney Hood          Utah Jazz     5.0       SG
Joe Ingles           Utah Jazz     2.0       SF
Chris Johnson        Utah Jazz    23.0       SF
Trey Lyles           Utah Jazz    41.0       PF
Shelvin Mack         Utah Jazz     8.0       PG
Raul Neto            Utah Jazz    25.0       PG
Tibor Pleiss         Utah Jazz    21.0        C
Jeff Withey          Utah Jazz    24.0        C
NaN                         NaN     NaN      NaN

[458 rows x 3 columns]
```

Methods for indexing in DataFrame

Function	Description
Dataframe.head()	Return top n rows of a data frame.
Dataframe.tail()	Return bottom n rows of a data frame.
Dataframe.at[]	Access a single value for a row/column label pair.
Dataframe.iat[]	Access a single value for a row/column pair by integer position.
Dataframe.tail()	Purely integer-location based indexing for selection by position.
DataFrame.lookup()	Label-based "fancy indexing" function for DataFrame.
DataFrame.pop()	Return item and drop from frame.
DataFrame.xs()	Returns a cross-section (row(s) or column(s)) from the DataFrame.
DataFrame.get()	Get item from object for given key (DataFrame column, Panel slice, etc.).
DataFrame.isin()	Return boolean DataFrame showing whether each element in the DataFrame is contained in values.
DataFrame.where()	Return an object of same shape as self and whose corresponding entries are from self where cond is True and otherwise are from

	other.
DataFrame.mask()	Return an object of same shape as self and whose corresponding entries are from self where cond is False and otherwise are from other.
DataFrame.query()	Query the columns of a frame with a boolean expression.
DataFrame.insert()	Insert column into DataFrame at specified location.

Computational Tools

A Python program can be executed by any computer, regardless of its manufacturer or operating system, provided that support for the language is installed.

Working with Missing Data in Pandas

Missing Data can occur when no information is provided for one or more items or for a whole unit. Missing Data is a very big problem in a real-life scenarios. Missing Data can also refer to as NA(Not Available) values in pandas. In DataFrame sometimes many datasets simply arrive with missing data, either because it exists and was not collected or it never existed. For Example, Suppose different users being surveyed may choose not to share their income, some users may choose not to share the address in this way many datasets went missing.

In Pandas missing data is represented by two value:

- None: None is a Python singleton object that is often used for missing data in Python code.
- NaN : NaN (an acronym for Not a Number), is a special floating-point value recognized by all systems that use the standard IEEE floating-point representation

Pandas treat None and NaN as essentially interchangeable for indicating missing or null values. To facilitate this convention, there are several useful functions for detecting, removing, and replacing null values in Pandas DataFrame :

- isnull()
- notnull()
- dropna()
- fillna()

- replace()
- interpolate()

In this article we are using CSV file, to download the CSV file used, Click Here.

Checking for missing values using isnull() and notnull()

In order to check missing values in Pandas DataFrame, we use a function isnull() and notnull(). Both function help in checking whether a value is NaN or not. These function can also be used in Pandas Series in order to find null values in a series.

Checking for missing values using isnull()

In order to check null values in Pandas DataFrame, we use isnull() function this function return dataframe of Boolean values which are True for NaN values.

Code #1:

```
# importing pandas as pd

import pandas as pd

# importing numpy as np

import numpy as np


# dictionary of lists

dict = {'First Score':[100, 90, np.nan, 95],

      'Second Score': [30, 45, 56, np.nan],

      'Third Score':[np.nan, 40, 80, 98]}

# creating a dataframe from list

df = pd.DataFrame(dict)

# using isnull() function

df.isnull()
```

Output:

	First Score	Second Score	Third Score
0	False	False	True
1	False	False	False
2	True	False	False
3	False	True	False

Code #2:

```
# importing pandas package
import pandas as pd
# making data frame from csv file
data = pd.read_csv("employees.csv")
# creating bool series True for NaN values
bool_series = pd.isnull(data["Gender"])
# filtering data
# displaying data only with Gender = NaN
data[bool_series]
```

Output:
As shown in the output image, only the rows having Gender = NULL are displayed.

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
20	Lois	NaN	4/22/1995	7:18 PM	64714	4.934	True	Legal
22	Joshua	NaN	3/8/2012	1:58 AM	90816	18.816	True	Client Services
27	Scott	NaN	7/11/1991	6:58 PM	122367	5.218	False	Legal
31	Joyce	NaN	2/20/2005	2:40 PM	88657	12.752	False	Product
41	Christine	NaN	6/28/2015	1:08 AM	66582	11.308	True	Business Development
49	Chris	NaN	1/24/1980	12:13 PM	113590	3.055	False	Sales
51	NaN	NaN	12/17/2011	8:29 AM	41126	14.009	NaN	Sales
53	Alan	NaN	3/3/2014	1:28 PM	40341	17.578	True	Finance
60	Paula	NaN	11/23/2005	2:01 PM	48866	4.271	False	Distribution
64	Kathleen	NaN	4/11/1990	6:46 PM	77834	18.771	False	Business Development
69	Irene	NaN	7/14/2015	4:31 PM	100863	4.382	True	Finance
70	Todd	NaN	6/10/2003	2:26 PM	84692	6.617	False	Client Services
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
939	Ralph	NaN	7/28/1995	6:53 PM	70635	2.147	False	Client Services
945	Gerald	NaN	4/15/1989	12:44 PM	93712	17.426	True	Distribution
961	Antonio	NaN	6/18/1989	9:37 PM	103050	3.050	False	Legal
972	Victor	NaN	7/28/2006	2:49 PM	76381	11.159	True	Sales
985	Stephen	NaN	7/10/1983	8:10 PM	85668	1.909	False	Legal
989	Justin	NaN	2/10/1991	4:58 PM	38344	3.794	False	Legal
995	Henry	NaN	11/23/2014	6:09 AM	132483	16.655	False	Distribution

145 rows × 8 columns

Checking for missing values using notnull()

In order to check null values in Pandas Dataframe, we use notnull() function this function return dataframe of Boolean values which are False for NaN values.

Code #3:

```
# importing pandas as pd
import pandas as pd
# importing numpy as np
import numpy as np
# dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
```

```
    'Second Score': [30, 45, 56, np.nan],
    'Third Score':[np.nan, 40, 80, 98]}
# creating a dataframe using dictionary
df = pd.DataFrame(dict)
# using notnull() function
df.notnull()
```

Output:

	First Score	Second Score	Third Score
0	True	True	False
1	True	True	True
2	False	True	True
3	True	False	True

Code #4:

```
# importing pandas package
import pandas as pd
# making data frame from csv file
data = pd.read_csv("employees.csv")
# creating bool series True for NaN values
bool_series = pd.notnull(data["Gender"])
# filtering data
# displayind data only with Gender = Not NaN
data[bool_series]
```

Output:
As shown in the output image, only the rows having Gender = NOT NULL are displayed.

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	8/6/1993	12:42 PM	97308	6.945	True	Marketing
1	Thomas	Male	3/31/1996	6:53 AM	61933	4.170	True	NaN
2	Maria	Female	4/23/1993	11:17 AM	130590	11.858	False	Finance
3	Jerry	Male	3/4/2005	1:00 PM	138705	9.340	True	Finance
4	Larry	Male	1/24/1998	4:47 PM	101004	1.389	True	Client Services
5	Dennis	Male	4/18/1987	1:35 AM	115163	10.125	False	Legal
6	Ruby	Female	8/17/1987	4:20 PM	65476	10.012	True	Product
7	NaN	Female	7/20/2015	10:43 AM	45906	11.598	NaN	Finance
8	Angela	Female	11/22/2005	6:29 AM	95570	18.523	True	Engineering
9	Frances	Female	8/8/2002	6:51 AM	139852	7.524	True	Business Development
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
994	George	Male	6/21/2013	5:47 PM	98874	4.479	True	Marketing
996	Phillip	Male	1/31/1984	6:30 AM	42392	19.675	False	Finance
997	Russell	Male	5/20/2013	12:39 PM	96914	1.421	False	Product
998	Larry	Male	4/20/2013	4:45 PM	60500	11.985	False	Business Development
999	Albert	Male	5/15/2012	6:24 PM	129949	10.169	True	Sales

855 rows × 8 columns

Filling missing values using fillna(), replace() and interpolate()

In order to fill null values in a datasets, we use fillna(), replace() and interpolate() function these function replace NaN values with some value of their own. All these function help in filling a null values in datasets of a DataFrame. Interpolate() function is basically used to fill NA values in the dataframe but it uses various interpolation technique to fill the missing values rather than hard-coding the value.

Code #1: Filling null values with a single value

```
# importing pandas as pd

import pandas as pd

# importing numpy as np

import numpy as np

# dictionary of lists

dict = {'First Score':[100, 90, np.nan, 95],

      'Second Score': [30, 45, 56, np.nan],
```

```
        'Third Score':[np.nan, 40, 80, 98]}
# creating a dataframe from dictionary
df = pd.DataFrame(dict)
# filling missing value using fillna()
df.fillna(0)
```

Output:

	First Score	Second Score	Third Score
0	100.0	30.0	0.0
1	90.0	45.0	40.0
2	0.0	56.0	80.0
3	95.0	0.0	98.0

Code #2: Filling null values with the previous ones

```
# importing pandas as pd
import pandas as pd
# importing numpy as np
import numpy as np
# dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
        'Second Score': [30, 45, 56, np.nan],
        'Third Score':[np.nan, 40, 80, 98]}
# creating a dataframe from dictionary
df = pd.DataFrame(dict)
# filling a missing value with
# previous ones
df.fillna(method ='pad')
```

Output:

	First Score	Second Score	Third Score
0	100.0	30.0	NaN
1	90.0	45.0	40.0
2	90.0	56.0	80.0
3	95.0	56.0	98.0

Code #3: Filling null value with the next ones

```
# importing pandas as pd
import pandas as pd
# importing numpy as np
import numpy as np
# dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
        'Second Score': [30, 45, 56, np.nan],
        'Third Score':[np.nan, 40, 80, 98]}
# creating a dataframe from dictionary
df = pd.DataFrame(dict)
# filling  null value using fillna() function
df.fillna(method ='bfill')
```

Output:

	First Score	Second Score	Third Score
0	100.0	30.0	40.0
1	90.0	45.0	40.0
2	95.0	56.0	80.0
3	95.0	NaN	98.0

Code #4: Filling null values in CSV File

```
# importing pandas package
import pandas as pd
# making data frame from csv file
data = pd.read_csv("employees.csv")
# Printing the first 10 to 24 rows of
# the data frame for visualization
data[10:25]
```

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
10	Louise	Female	8/12/1980	9:01 AM	63241	15.132	True	NaN
11	Julie	Female	10/26/1997	3:19 PM	102508	12.637	True	Legal
12	Brandon	Male	12/1/1980	1:08 AM	112807	17.492	True	Human Resources
13	Gary	Male	1/27/2008	11:40 PM	109831	5.831	False	Sales
14	Kimberly	Female	1/14/1999	7:13 AM	41426	14.543	True	Finance
15	Lillian	Female	6/5/2016	6:09 AM	59414	1.256	False	Product
16	Jeremy	Male	9/21/2010	5:56 AM	90370	7.369	False	Human Resources
17	Shawn	Male	12/7/1986	7:45 PM	111737	6.414	False	Product
18	Diana	Female	10/23/1981	10:27 AM	132940	19.082	False	Client Services
19	Donna	Female	7/22/2010	3:48 AM	81014	1.894	False	Product
20	Lois	NaN	4/22/1995	7:18 PM	64714	4.934	True	Legal
21	Matthew	Male	9/5/1995	2:12 AM	100612	13.645	False	Marketing
22	Joshua	NaN	3/8/2012	1:58 AM	90816	18.816	True	Client Services
23	NaN	Male	6/14/2012	4:19 PM	125792	5.042	NaN	NaN
24	John	Male	7/1/1992	10:08 PM	97950	13.873	False	Client Services

Now we are going to fill all the null values in Gender column with "No Gender"

```
# importing pandas package
import pandas as pd
# making data frame from csv file
data = pd.read_csv("employees.csv")
```

```
# filling a null values using fillna()

data["Gender"].fillna("No Gender", inplace = True)

data
```

Output:

10	Louise	Female	8/12/1980	9:01 AM	63241	15.132	True	NaN
11	Julie	Female	10/26/1997	3:19 PM	102508	12.637	True	Legal
12	Brandon	Male	12/1/1980	1:08 AM	112807	17.492	True	Human Resources
13	Gary	Male	1/27/2008	11:40 PM	109831	5.831	False	Sales
14	Kimberly	Female	1/14/1999	7:13 AM	41426	14.543	True	Finance
15	Lillian	Female	6/5/2016	6:09 AM	59414	1.256	False	Product
16	Jeremy	Male	9/21/2010	5:56 AM	90370	7.369	False	Human Resources
17	Shawn	Male	12/7/1986	7:45 PM	111737	6.414	False	Product
18	Diana	Female	10/23/1981	10:27 AM	132940	19.082	False	Client Services
19	Donna	Female	7/22/2010	3:48 AM	81014	1.894	False	Product
20	Lois	No Gender	4/22/1995	7:18 PM	64714	4.934	True	Legal
21	Matthew	Male	9/5/1995	2:12 AM	100612	13.645	False	Marketing
22	Joshua	No Gender	3/8/2012	1:58 AM	90816	18.816	True	Client Services
23	NaN	Male	6/14/2012	4:19 PM	125792	5.042	NaN	NaN
24	John	Male	7/1/1992	10:08 PM	97950	13.873	False	Client Services

Code #5: Filling a null values using replace() method

```
# importing pandas package

import pandas as pd

# making data frame from csv file

data = pd.read_csv("employees.csv")

# Printing the first 10 to 24 rows of

# the data frame for visualization

data[10:25]
```

Output:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
10	Louise	Female	8/12/1980	9:01 AM	63241	15.132	True	NaN
11	Julie	Female	10/26/1997	3:19 PM	102508	12.637	True	Legal
12	Brandon	Male	12/1/1980	1:08 AM	112807	17.492	True	Human Resources
13	Gary	Male	1/27/2008	11:40 PM	109831	5.831	False	Sales
14	Kimberly	Female	1/14/1999	7:13 AM	41426	14.543	True	Finance
15	Lillian	Female	6/5/2016	6:09 AM	59414	1.256	False	Product
16	Jeremy	Male	9/21/2010	5:56 AM	90370	7.369	False	Human Resources
17	Shawn	Male	12/7/1986	7:45 PM	111737	6.414	False	Product
18	Diana	Female	10/23/1981	10:27 AM	132940	19.082	False	Client Services
19	Donna	Female	7/22/2010	3:48 AM	81014	1.894	False	Product
20	Lois	NaN	4/22/1995	7:18 PM	64714	4.934	True	Legal
21	Matthew	Male	9/5/1995	2:12 AM	100612	13.645	False	Marketing
22	Joshua	NaN	3/8/2012	1:58 AM	90816	18.816	True	Client Services
23	NaN	Male	6/14/2012	4:19 PM	125792	5.042	NaN	NaN

Now we are going to replace the all Nan value in the data frame with -99 value.

```
# importing pandas package
import pandas as pd
# making data frame from csv file
data = pd.read_csv("employees.csv")
# will replace  Nan value in dataframe with value -99
data.replace(to_replace = np.nan, value = -99)
```

Output:

10	Louise	Female	8/12/1980	9:01 AM	63241	15.132	True	-99
11	Julie	Female	10/26/1997	3:19 PM	102508	12.637	True	Legal
12	Brandon	Male	12/1/1980	1:08 AM	112807	17.492	True	Human Resources
13	Gary	Male	1/27/2008	11:40 PM	109831	5.831	False	Sales
14	Kimberly	Female	1/14/1999	7:13 AM	41426	14.543	True	Finance
15	Lillian	Female	6/5/2016	6:09 AM	59414	1.256	False	Product
16	Jeremy	Male	9/21/2010	5:56 AM	90370	7.369	False	Human Resources
17	Shawn	Male	12/7/1986	7:45 PM	111737	6.414	False	Product
18	Diana	Female	10/23/1981	10:27 AM	132940	19.082	False	Client Services
19	Donna	Female	7/22/2010	3:48 AM	81014	1.894	False	Product
20	Lois	-99	4/22/1995	7:18 PM	64714	4.934	True	Legal
21	Matthew	Male	9/5/1995	2:12 AM	100612	13.645	False	Marketing
22	Joshua	-99	3/8/2012	1:58 AM	90816	18.816	True	Client Services
23	-99	Male	6/14/2012	4:19 PM	125792	5.042	-99	-99
24	John	Male	7/1/1992	10:08 PM	97950	13.873	False	Client Services

Code #6: Using interpolate() function to fill the missing values using linear method.

```
# importing pandas as pd
import pandas as pd
# Creating the dataframe
df = pd.DataFrame({"A":[12, 4, 5, None, 1],
                   "B":[None, 2, 54, 3, None],
                   "C":[20, 16, None, 3, 8],
                   "D":[14, 3, None, None, 6]})
# Print the dataframe
df
```

	A	B	C	D
0	12.0	NaN	20.0	14.0
1	4.0	2.0	16.0	3.0
2	5.0	54.0	NaN	NaN
3	NaN	3.0	3.0	NaN
4	1.0	NaN	8.0	6.0

Let's interpolate the missing values using Linear method. Note that Linear method ignore the index and treat the values as equally spaced.

```
# to interpolate the missing values

df.interpolate(method ='linear', limit_direction ='forward')
```

Output:

	A	B	C	D
0	12.0	NaN	20.0	14.0
1	4.0	2.0	16.0	3.0
2	5.0	54.0	9.5	4.0
3	3.0	3.0	3.0	5.0
4	1.0	3.0	8.0	6.0

Dropping missing values using dropna()

In order to drop a null values from a dataframe, we used dropna() function this function drop Rows/Columns of datasets with Null values in different ways.

Code #1: Dropping rows with at least 1 null value.

```
# importing pandas as pd

import pandas as pd

# importing numpy as np

import numpy as np

# dictionary of lists

dict = {'First Score':[100, 90, np.nan, 95],

      'Second Score': [30, np.nan, 45, 56],
```

```
    'Third Score':[52, 40, 80, 98],
    'Fourth Score':[np.nan, np.nan, np.nan, 65]}
# creating a dataframe from dictionary
df = pd.DataFrame(dict)
df
```

	First Score	Second Score	Third Score	Fourth Score
0	100.0	30.0	52	NaN
1	90.0	NaN	40	NaN
2	NaN	45.0	80	NaN
3	95.0	56.0	98	65.0

Now we drop rows with at least one Nan value (Null value)

```
# importing pandas as pd
import pandas as pd
# importing numpy as np
import numpy as np
# dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
    'Second Score': [30, np.nan, 45, 56],
    'Third Score':[52, 40, 80, 98],
    'Fourth Score':[np.nan, np.nan, np.nan, 65]}
# creating a dataframe from dictionary
df = pd.DataFrame(dict)
# using dropna() function
df.dropna()
```

Output:

	First Score	Second Score	Third Score	Fourth Score
3	95.0	56.0	98	65.0

Code #2: Dropping rows if all values in that row are missing.

```
# importing pandas as pd
import pandas as pd
# importing numpy as np
import numpy as np
# dictionary of lists
dict = {'First Score':[100, np.nan, np.nan, 95],
        'Second Score': [30, np.nan, 45, 56],
        'Third Score':[52, np.nan, 80, 98],
        'Fourth Score':[np.nan, np.nan, np.nan, 65]}

# creating a dataframe from dictionary
df = pd.DataFrame(dict)
df
```

	First Score	Second Score	Third Score	Fourth Score
0	100.0	30.0	52.0	NaN
1	NaN	NaN	NaN	NaN
2	NaN	45.0	80.0	NaN
3	95.0	56.0	98.0	65.0

Now we drop a rows whose all data is missing or contain null values(NaN)

```
# importing pandas as pd
import pandas as pd
# importing numpy as np
```

```
import numpy as np
# dictionary of lists
dict = {'First Score':[100, np.nan, np.nan, 95],
        'Second Score': [30, np.nan, 45, 56],
        'Third Score':[52, np.nan, 80, 98],
        'Fourth Score':[np.nan, np.nan, np.nan, 65]}
df = pd.DataFrame(dict)
# using dropna() function
df.dropna(how = 'all')
```

Output:

	First Score	Second Score	Third Score	Fourth Score
0	100.0	30.0	52.0	NaN
2	NaN	45.0	80.0	NaN
3	95.0	56.0	98.0	65.0

Code #3: Dropping columns with at least 1 null value.

```
# importing pandas as pd
import pandas as pd
# importing numpy as np
import numpy as np
# dictionary of lists
dict = {'First Score':[100, np.nan, np.nan, 95],
        'Second Score': [30, np.nan, 45, 56],
        'Third Score':[52, np.nan, 80, 98],
        'Fourth Score':[60, 67, 68, 65]}

# creating a dataframe from dictionary
```

```
df = pd.DataFrame(dict)
```

df

	First Score	Second Score	Third Score	Fourth Score
0	100.0	30.0	52.0	60
1	NaN	NaN	NaN	67
2	NaN	45.0	80.0	68
3	95.0	56.0	98.0	65

Now we drop a columns which have at least 1 missing values

```
# importing pandas as pd
import pandas as pd
# importing numpy as np
import numpy as np
# dictionary of lists
dict = {'First Score':[100, np.nan, np.nan, 95],
        'Second Score': [30, np.nan, 45, 56],
        'Third Score':[52, np.nan, 80, 98],
        'Fourth Score':[60, 67, 68, 65]}
# creating a dataframe from dictionary
df = pd.DataFrame(dict)
# using dropna() function
df.dropna(axis = 1)
```

Output :

	Fourth Score
0	60
1	67
2	68
3	65

Code #4: Dropping Rows with at least 1 null value in CSV file

```
# importing pandas module
import pandas as pd
# making data frame from csv file
data = pd.read_csv("employees.csv")
# making new data frame with dropped NA values
new_data = data.dropna(axis = 0, how ='any')
new_data
```

Output:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	8/6/1993	12:42 PM	97308	6.945	True	Marketing
2	Maria	Female	4/23/1993	11:17 AM	130590	11.858	False	Finance
3	Jerry	Male	3/4/2005	1:00 PM	138705	9.340	True	Finance
4	Larry	Male	1/24/1998	4:47 PM	101004	1.389	True	Client Services
5	Dennis	Male	4/18/1987	1:35 AM	115163	10.125	False	Legal
6	Ruby	Female	8/17/1987	4:20 PM	65476	10.012	True	Product
8	Angela	Female	11/22/2005	6:29 AM	95570	18.523	True	Engineering
9	Frances	Female	8/8/2002	6:51 AM	139852	7.524	True	Business Development
11	Julie	Female	10/26/1997	3:19 PM	102508	12.637	True	Legal
12	Brandon	Male	12/1/1980	1:08 AM	112807	17.492	True	Human Resources
13	Gary	Male	1/27/2008	11:40 PM	109831	5.831	False	Sales
14	Kimberly	Female	1/14/1999	7:13 AM	41426	14.543	True	Finance
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
993	Tina	Female	5/15/1997	3:53 PM	56450	19.040	True	Engineering
994	George	Male	6/21/2013	5:47 PM	98874	4.479	True	Marketing
996	Phillip	Male	1/31/1984	6:30 AM	42392	19.675	False	Finance
997	Russell	Male	5/20/2013	12:39 PM	96914	1.421	False	Product
998	Larry	Male	4/20/2013	4:45 PM	60500	11.985	False	Business Development
999	Albert	Male	5/15/2012	6:24 PM	129949	10.169	True	Sales

764 rows × 8 columns

Now we compare sizes of data frames so that we can come to know how many rows had at least 1 Null value

```
print("Old data frame length:", len(data))

print("New data frame length:", len(new_data))

print("Number of rows with at least 1 NA value: ", (len(data)-len(new_data)))
```

Output :
Old data frame length: 1000

New data frame length: 764

Number of rows with at least 1 NA value: 236

Since the difference is 236, there were 236 rows which had at least 1 Null value in any column.

Hierarchical Indexes
Hierarchical Indexes are also known as multi-indexing is setting more than one column name as the index. In this article, we are going to use homelessness.csv file.

```
# importing pandas library as alias pd
import pandas as pd

# calling the pandas read_csv() function.
# and storing the result in DataFrame df
df = pd.read_csv('homelessness.csv')

print(df.head())
```

Output:

	region	state	individuals	family_members	state_pop
0	East South Central	Alabama	2570.0	864.0	4887681
1	Pacific	Alaska	1434.0	582.0	735139
2	Mountain	Arizona	7259.0	2606.0	7158024
3	West South Central	Arkansas	2280.0	432.0	3009733
4	Pacific	California	109008.0	20964.0	39461588

In the following data frame, there is no indexing.

Columns in the Dataframe:

- Python3

```
# using the pandas columns attribute.
col = df.columns
print(col)
```

Output:
Index(['Unnamed: 0', 'region', 'state', 'individuals', 'family_members',
'state_pop'],
dtype='object')

To make the column an index, we use the Set_index() function of pandas. If we want to make one column an index, we can simply pass the name of the column as a string in set_index(). If we want to do multi-indexing or Hierarchical Indexing, we pass the list of column names in the set_index().
Below Code demonstrates Hierarchical Indexing in pandas:

- Python3

```
# using the pandas set_index() function.
df_ind3 = df.set_index(['region', 'state', 'individuals'])
```

```
# we can sort the data by using sort_index()
df_ind3.sort_index()

print(df_ind3.head(10))
```

Output:

			family_members	state_pop
region	state	individuals		
East South Central	Alabama	2570.0	864.0	4887681
Pacific	Alaska	1434.0	582.0	735139
Mountain	Arizona	7259.0	2606.0	7158024
West South Central	Arkansas	2280.0	432.0	3009733
Pacific	California	109008.0	20964.0	39461588
Mountain	Colorado	7607.0	3250.0	5691287
New England	Connecticut	2280.0	1696.0	3571520
South Atlantic	Delaware	708.0	374.0	965479
	District of Columbia	3770.0	3134.0	701547
	Florida	21443.0	9587.0	21244317

Now the dataframe is using Hierarchical Indexing or multi-indexing.

Note that here we have made 3 columns as an index ('region', 'state', 'individuals'). The first index 'region' is called level(0) index, which is on top of the Hierarchy of indexes, next index 'state' is level(1) index which is below the main or level(0) index, and so on. So, the Hierarchy of indexes is formed that's why this is called **Hierarchical indexing**.
We may sometimes need to make a column as an index, or we want to convert an index column into the normal column, so there is a pandas reset_index(inplace = True) function, which makes the index column the normal column.
Selecting Data in a Hierarchical Index or using the Hierarchical Indexing:
For selecting the data from the dataframe using the .loc() method we have to pass the name of the indexes in a list.

```
# selecting the 'Pacific' and 'Mountain'
# region from the dataframe.

# selecting data using level(0) index or main index.
df_ind3_region = df_ind3.loc[['Pacific', 'Mountain']]

print(df_ind3_region.head(10))
```

Output:

```
                                family_members  state_pop
region   state      individuals
Pacific  Alaska     1434.0               582.0     735139
         California 109008.0            20964.0   39461588
         Hawaii     4131.0               2399.0    1420593
         Oregon     11139.0              3337.0    4181886
         Washington 16424.0              5880.0    7523869
Mountain Arizona    7259.0               2606.0    7158024
         Colorado   7607.0               3250.0    5691287
         Idaho      1297.0               715.0     1750536
         Montana    983.0                422.0     1060665
         Nevada     7058.0               486.0     3027341
```

We cannot use only level(1) index for getting data from the dataframe, if we do so it will give an error. We can only use level (1) index or the inner indexes with the level(0) or main index with the help list of tuples.

```
# using the inner index 'state' for getting data.
df_ind3_state = df_ind3.loc[['Alaska', 'California', 'Idaho']]

print(df_ind3_state.head(10))
```

Output:

```
~/anaconda3/lib/python3.8/site-packages/pandas/core/indexing.py in _getitem_axis(self, key, axis)
   1097                     raise ValueError("Cannot index with multidimensional key")
   1098
-> 1099                 return self._getitem_iterable(key, axis=axis)
   1100
   1101             # nested tuple slicing

~/anaconda3/lib/python3.8/site-packages/pandas/core/indexing.py in _getitem_iterable(self, key, axis)
   1035
   1036         # A collection of keys
-> 1037         keyarr, indexer = self._get_listlike_indexer(key, axis, raise_missing=False)
   1038         return self.obj._reindex_with_indexers(
   1039             {axis: [keyarr, indexer]}, copy=True, allow_dups=True
```

Using inner levels indexes with the help of a list of tuples:

Syntax:

df.loc[[(level(0) , level(1) , level(2))]]

```
# selecting data by passing all levels index.
df_ind3_region_state = df_ind3.loc[[("Pacific", "Alaska", 1434),
                                    ("Pacific", "Hawaii", 4131),
                                    ("Mountain", "Arizona", 7259),
                                    ("Mountain", "Idaho", 1297)]]
df_ind3_region_state
```

Output:

region	state	individuals	family_members	state_pop
Pacific	Alaska	1434.0	582.0	735139
	Hawaii	4131.0	2399.0	1420593
Mountain	Arizona	7259.0	2606.0	7158024
	Idaho	1297.0	715.0	1750536

Python Pandas – Panel data

A **panel** is a 3D container of data. The term **Panel data** is derived from econometrics and is partially responsible for the name pandas – **pan(el)-da(ta)**-s.

The names for the 3 axes are intended to give some semantic meaning to describing operations involving panel data. They are –

- **items** – axis 0, each item corresponds to a DataFrame contained inside.
- **major_axis** – axis 1, it is the index (rows) of each of the DataFrames.
- **minor_axis** – axis 2, it is the columns of each of the DataFrames.

pandas.Panel()

A Panel can be created using the following constructor –

```
pandas.Panel(data, items, major_axis, minor_axis, dtype, copy)
```

The parameters of the constructor are as follows –

Parameter	Description
data	Data takes various forms like ndarray, series, map, lists, dict, constants and also another DataFrame
items	axis=0
major_axis	axis=1

minor_axis	axis=2
dtype	Data type of each column
copy	Copy data. Default, **false**

Create Panel

A Panel can be created using multiple ways like –

- From ndarrays
- From dict of DataFrames

```
# creating an empty panel
import pandas as pd
import numpy as np

data = np.random.rand(2,4,5)
p = pd.Panel(data)
print p
```

Its **output** is as follows –

```
<class 'pandas.core.panel.Panel'>
Dimensions: 2 (items) x 4 (major_axis) x 5 (minor_axis)
Items axis: 0 to 1
Major_axis axis: 0 to 3
Minor_axis axis: 0 to 4
```

Note – Observe the dimensions of the empty panel and the above panel, all the objects are different.

From dict of DataFrame Objects

```
#creating an empty panel
import pandas as pd
import numpy as np
```

```
data = {'Item1' : pd.DataFrame(np.random.randn(4, 3)),
   'Item2' : pd.DataFrame(np.random.randn(4, 2))}
p = pd.Panel(data)
print p
```

Its **output** is as follows –

```
Dimensions: 2 (items) x 4 (major_axis) x 3 (minor_axis)
Items axis: Item1 to Item2
Major_axis axis: 0 to 3
Minor_axis axis: 0 to 2
```

Create an Empty Panel

An empty panel can be created using the Panel constructor as follows –

```
#creating an empty panel
import pandas as pd
p = pd.Panel()
print p
```

Its **output** is as follows –

```
<class 'pandas.core.panel.Panel'>
Dimensions: 0 (items) x 0 (major_axis) x 0 (minor_axis)
Items axis: None
Major_axis axis: None
Minor_axis axis: None
```

Selecting the Data from Panel

Select the data from the panel using –

- Items
- Major_axis
- Minor_axis

Using Items

```
# creating an empty panel
import pandas as pd
import numpy as np
data = {'Item1' : pd.DataFrame(np.random.randn(4, 3)),
   'Item2' : pd.DataFrame(np.random.randn(4, 2))}
p = pd.Panel(data)
print p['Item1']
```

Its **output** is as follows –

```
          0         1         2
0  0.488224 -0.128637  0.930817
1  0.417497  0.896681  0.576657
2 -2.775266  0.571668  0.290082
3 -0.400538 -0.144234  1.110535
```

We have two items, and we retrieved item1. The result is a DataFrame with 4 rows and 3 columns, which are the **Major_axis** and **Minor_axis** dimensions.

Using major_axis

Data can be accessed using the method **panel.major_axis(index)**.

```
# creating an empty panel
import pandas as pd
import numpy as np
data = {'Item1' : pd.DataFrame(np.random.randn(4, 3)),
  'Item2' : pd.DataFrame(np.random.randn(4, 2))}
p = pd.Panel(data)
print p.major_xs(1)
```

Its **output** is as follows –

```
      Item1     Item2
0  0.417497  0.748412
1  0.896681 -0.557322
2  0.576657       NaN
```

Using minor_axis

Data can be accessed using the method **panel.minor_axis(index).**

```
# creating an empty panel
import pandas as pd
import numpy as np
data = {'Item1' : pd.DataFrame(np.random.randn(4, 3)),
  'Item2' : pd.DataFrame(np.random.randn(4, 2))}
p = pd.Panel(data)
print p.minor_xs(1)
```

Its **output** is as follows –

```
      Item1     Item2
0 -0.128637 -1.047032
1  0.896681 -0.557322
2  0.571668  0.431953
3 -0.144234  1.302466
```

Unit-5

Data Analysis Application Examples: Data munging,Cleaning data, Filtering, Merging data, Reshaping data, Data aggregation, Grouping data

Data Wrangling or Munging in Python

Data Wrangling is the process of gathering, collecting, and transforming Raw data into another format for better understanding, decision-making, accessing, and analysis in less time. Data Wrangling is also known as Data Munging.

Pandas is an open-source library specifically developed for Data Analysis and Data Science. The process like data sorting or filtration, Data grouping, etc.

Data wrangling in python deals with the below functionalities:

1. **Data exploration:** In this process, the data is studied, analyzed and understood by visualizing representations of data.
2. **Dealing with missing values:** Most of the datasets having a vast amount of data contain missing values of *NaN, they are needed to be taken* care of by replacing them with mean, mode, the most frequent value of the column or simply by dropping the row having a *NaN* value.
3. **Reshaping data:** In this process, data is manipulated according to the requirements, where new data can be added or pre-existing data can be modified.
4. **Filtering data:** Some times datasets are comprised of unwanted rows or columns which are required to be removed or filtered
5. **Other:** After dealing with the raw dataset with the above functionalities we get an efficient dataset as per our requirements and then it can be used for a required purpose like data analyzing, machine learning, data visualization, model training etc.

Below is an example which implements the above functionalities on a raw dataset:

- **Data exploration**, here we assign the data, and then we visualize the data in a tabular format.

Example:

```
# Import pandas package

import pandas as pd

# Assign data

data = {'Name': ['Jai', 'Princi', 'Gaurav','Anuj', 'Ravi', 'Natasha', 'Riya'],

        'Age': [17, 17, 18, 17, 18, 17, 17],

        'Gender': ['M', 'F', 'M', 'M', 'M', 'F', 'F'],
```

```
                'Marks': [90, 76, 'NaN', 74, 65, 'NaN', 71]}

# Convert into DataFrame

df = pd.DataFrame(data)

# Display data

df
```

Output:

	Name	Age	Gender	Marks
0	Jai	17	M	90
1	Princi	17	F	76
2	Gaurav	18	M	NaN
3	Anuj	17	M	74
4	Ravi	18	M	65
5	Natasha	17	F	NaN
6	Riya	17	F	71

- **Dealing with missing values**, as we can see from the previous output, there are *NaN* values present in the *MARKS* column which are going to be taken care of by replacing them with the column mean.

  Example:

```
# Compute average
c = avg = 0
for ele in df['Marks']:
    if str(ele).isnumeric():
            c += 1
            avg += ele
           avg /= c

# Replace missing values
df = df.replace(to_replace="NaN",
                              value=avg)
```

```
# Display data
Df
```

	Name	Age	Gender	Marks
0	Jai	17	M	90.0
1	Princi	17	F	76.0
2	Gaurav	18	M	75.2
3	Anuj	17	M	74.0
4	Ravi	18	M	65.0
5	Natasha	17	F	75.2
6	Riya	17	F	71.0

- **Reshaping data,** in the *GENDER* column, we can reshape the data by categorizing them into different numbers.

```
# Categorize gender

df['Gender'] = df['Gender'].map({'M': 0,'F': 1, }).astype(float)

# Display data

df
```

Output:

	Name	Age	Gender	Marks
0	Jai	17	0.0	90.0
1	Princi	17	1.0	76.0
2	Gaurav	18	0.0	75.2
3	Anuj	17	0.0	74.0
4	Ravi	18	0.0	65.0
5	Natasha	17	1.0	75.2
6	Riya	17	1.0	71.0

- **Filtering data**,

- suppose there is a requirement for the details regarding name, gender, marks of the top-scoring students. Here we need to remove some unwanted data.

```
# Filter top scoring students
df = df[df['Marks'] >= 75]

# Remove age row
df = df.drop(['Age'], axis=1)

# Display data
Df
```

Output:

	Name	Gender	Marks
0	Jai	0.0	90.0
1	Princi	1.0	76.0
2	Gaurav	0.0	75.2
5	Natasha	1.0	75.2

Wrangling Data Using Merge Operation

Merge operation is used to merge raw data and into the desired format.
Syntax:

pd.merge(data_frame1,data_frame2, on="field ")

Here the field is the name of the column which is similar on both data-frame.
For example: Suppose that a Teacher has two types of Data, first type of Data consist of Details of Students and Second type of Data Consist of Pending Fees Status which is taken from Account Office. So The Teacher will use merge operation here in order to merge the data and provide it meaning. So that teacher will analyze it easily and it also reduces time and effort of Teacher from Manual Merging.

FIRST TYPE OF DATA:

```
# import module

import pandas as pd


# creating DataFrame for Student Details

details = pd.DataFrame({

        'ID': [101, 102, 103, 104, 105, 106,

                107, 108, 109, 110],

        'NAME': ['Jagroop', 'Praveen', 'Harjot',

                        'Pooja', 'Rahul', 'Nikita',

                        'Saurabh', 'Ayush', 'Dolly', "Mohit"],

        'BRANCH': ['CSE', 'CSE', 'CSE', 'CSE', 'CSE',

                        'CSE', 'CSE', 'CSE', 'CSE', 'CSE']})


# printing details
```

```
print(details)
```

Output:

```
      ID     NAME BRANCH
0   101  Jagroop    CSE
1   102  Praveen    CSE
2   103   Harjot    CSE
3   104    Pooja    CSE
4   105    Rahul    CSE
5   106   Nikita    CSE
6   107  Saurabh    CSE
7   108    Ayush    CSE
8   109    Dolly    CSE
9   110    Mohit    CSE
```

SECOND TYPE OF DATA

```
# Import module

import pandas as pd


# Creating Dataframe for Fees_Status

fees_status = pd.DataFrame(

        {'ID': [101, 102, 103, 104, 105,

                        106, 107, 108, 109, 110],

        'PENDING': ['5000', '250', 'NIL',

                                '9000', '15000', 'NIL',

                                '4500', '1800', '250', 'NIL']})


# Printing fees_status

print(fees_status)
```

Output:

```
    ID PENDING
0  101    5000
1  102     250
2  103     NIL
3  104    9000
4  105   15000
5  106     NIL
6  107    4500
7  108    1800
8  109     250
9  110     NIL
```

WRANGLING DATA USING MERGE OPERATION:

```
# Import module

import pandas as pd

# Creating Dataframe

details = pd.DataFrame({

        'ID': [101, 102, 103, 104, 105,

                106, 107, 108, 109, 110],

        'NAME': ['Jagroop', 'Praveen', 'Harjot',

                        'Pooja', 'Rahul', 'Nikita',

                        'Saurabh', 'Ayush', 'Dolly', "Mohit"],

        'BRANCH': ['CSE', 'CSE', 'CSE', 'CSE', 'CSE',

                        'CSE', 'CSE', 'CSE', 'CSE', 'CSE']})

# Creating Dataframe

fees_status = pd.DataFrame(

        {'ID': [101, 102, 103, 104, 105,

                        106, 107, 108, 109, 110],

        'PENDING': ['5000', '250', 'NIL',

                                '9000', '15000', 'NIL',
```

```
                                    '4500', '1800', '250', 'NIL']})


# Merging Dataframe

print(pd.merge(details, fees_status, on='ID'))
```

Output:

	ID	NAME	BRANCH	PENDING
0	101	Jagroop	CSE	5000
1	102	Praveen	CSE	250
2	103	Harjot	CSE	NIL
3	104	Pooja	CSE	9000
4	105	Rahul	CSE	15000
5	106	Nikita	CSE	NIL
6	107	Saurabh	CSE	4500
7	108	Ayush	CSE	1800
8	109	Dolly	CSE	250
9	110	Mohit	CSE	NIL

Wrangling Data using Grouping Method

The grouping method in Data analysis is used to provide results in terms of various groups taken out from Large Data. This method of pandas is used to group the outset of data from the large data set.
Example: There is a Car Selling company and this company have different Brands of various Car Manufacturing Company like Maruti, Toyota, Mahindra, Ford, etc. and have data where different cars are sold in different years. So the Company wants to wrangle only that data where cars are sold during the year 2010. For this problem, we use another Wrangling technique that is *groupby()* method.

CARS SELLING DATA:

```
# Import module

import pandas as pd

# Creating Data

car_selling_data = {'Brand': ['Maruti', 'Maruti', 'Maruti',

                              'Maruti', 'Hyundai', 'Hyundai',
```

```
                    'Toyota', 'Mahindra', 'Mahindra',
                    'Ford', 'Toyota', 'Ford'],
            'Year': [2010, 2011, 2009, 2013,
                    2010, 2011, 2011, 2010,
                    2013, 2010, 2010, 2011],
            'Sold': [6, 7, 9, 8, 3, 5,
                    2, 8, 7, 2, 4, 2]}

# Creating Dataframe of car_selling_data
df = pd.DataFrame(car_selling_data)
# printing Dataframe
print(df)
```

Output:

	Brand	Year	Sold
0	Maruti	2010	6
1	Maruti	2011	7
2	Maruti	2009	9
3	Maruti	2013	8
4	Hyundai	2010	3
5	Hyundai	2011	5
6	Toyota	2011	2
7	Mahindra	2010	8
8	Mahindra	2013	7
9	Ford	2010	2
10	Toyota	2010	4
11	Ford	2011	2

DATA OF THE YEAR 2010:

```
# Import module
import pandas as pd
```

```
# Creating Data

car_selling_data = {'Brand': ['Maruti', 'Maruti', 'Maruti',
                              'Maruti', 'Hyundai', 'Hyundai',
                              'Toyota', 'Mahindra', 'Mahindra',
                              'Ford', 'Toyota', 'Ford'],
                    'Year': [2010, 2011, 2009, 2013,
                             2010, 2011, 2011, 2010,
                             2013, 2010, 2010, 2011],
                    'Sold': [6, 7, 9, 8, 3, 5,
                             2, 8, 7, 2, 4, 2]}


# Creating Dataframe for Provided Data

df = pd.DataFrame(car_selling_data)


# Group the data when year = 2010

grouped = df.groupby('Year')

print(grouped.get_group(2010))
```

Output:

	Brand	Year	Sold
0	Maruti	2010	6
4	Hyundai	2010	3
7	Mahindra	2010	8
9	Ford	2010	2
10	Toyota	2010	4

Wrangling data by removing Duplication

Pandas *duplicates()* method helps us to remove duplicate values from Large Data. An important part of Data Wrangling is removing Duplicate values from the large data set.

Syntax:

DataFrame.duplicated(subset=None, keep='first')

Here subset is the column value where we want to remove Duplicate value.

In *keep*, we have 3 options :

- if *keep ='first'* then the first value is marked as original rest all values if occur will be removed as it is considered as duplicate.
- if *keep='last'* then the last value is marked as original rest all above same values will be removed as it is considered as duplicate values.
- if *keep ='false'* the all the values which occur more than once will be removed as all considered as a duplicate value.

For example, A University will organize the event. In order to participate Students have to fill their details in the online form so that they will contact them. It may be possible that a student will fill the form multiple time. It may cause difficulty for the event organizer if a single student will fill multiple entries. The Data that the organizers will get can be Easily Wrangles by removing duplicate values.

DETAILS STUDENTS DATA WHO WANT TO PARTICIPATE IN THE EVENT:

```
# Import module
import pandas as pd

# Initializing Data
student_data = {'Name': ['Amit', 'Praveen', 'Jagroop',
                         'Rahul', 'Vishal', 'Suraj',
                         'Rishab', 'Satyapal', 'Amit',
                         'Rahul', 'Praveen', 'Amit'],

                'Roll_no': [23, 54, 29, 36, 59, 38,
                            12, 45, 34, 36, 54, 23],

                'Email': ['xxxx@gmail.com', 'xxxxxx@gmail.com',
                          'xxxxxx@gmail.com', 'xx@gmail.com',
                          'xxxx@gmail.com', 'xxxxx@gmail.com',
                          'xxxxx@gmail.com', 'xxxxx@gmail.com',
                          'xxxxx@gmail.com', 'xxxxxx@gmail.com',
                          'xxxxxxxxxx@gmail.com', 'xxxxxxxxxx@gmail.com']}

# Creating Dataframe of Data
df = pd.DataFrame(student_data)
```

# Printing Dataframe
print(df)

Output:

	Name	Roll_no	Email
0	Amit	23	xxxx@gmail.com
1	Praveen	54	xxxxxx@gmail.com
2	Jagroop	29	xxxxxx@gmail.com
3	Rahul	36	xx@gmail.com
4	Vishal	59	xxxx@gmail.com
5	Suraj	38	xxxxx@gmail.com
6	Rishab	12	xxxxx@gmail.com
7	Satyapal	45	xxxxx@gmail.com
8	Amit	34	xxxxx@gmail.com
9	Rahul	36	xxxxxx@gmail.com
10	Praveen	54	xxxxxxxxxx@gmail.com
11	Amit	23	xxxxxxxxxx@gmail.com

DATA WRANGLED BY REMOVING DUPLICATE ENTRIES:

```
# import module

import pandas as pd

# initializing Data

student_data = {'Name': ['Amit', 'Praveen', 'Jagroop',
                         'Rahul', 'Vishal', 'Suraj',
                         'Rishab', 'Satyapal', 'Amit',
                         'Rahul', 'Praveen', 'Amit'],


                'Roll_no': [23, 54, 29, 36, 59, 38,
                            12, 45, 34, 36, 54, 23],
                'Email': ['xxxx@gmail.com', 'xxxxxx@gmail.com',
                          'xxxxxx@gmail.com', 'xx@gmail.com',
                          'xxxx@gmail.com', 'xxxxx@gmail.com',
```

```
                                    'xxxxx@gmail.com', 'xxxxx@gmail.com',
                                    'xxxxx@gmail.com', 'xxxxxx@gmail.com',
                                    'xxxxxxxxxx@gmail.com',
'xxxxxxxxxx@gmail.com']}

# creating dataframe
df = pd.DataFrame(student_data)
# Here df.duplicated() list duplicate Entries in ROllno.
# So that ~(NOT) is placed in order to get non duplicate values.
non_duplicate = df[~df.duplicated('Roll_no')]
# printing non-duplicate values
print(non_duplicate)
```

	Name	Roll_no	Email
0	Amit	23	xxxx@gmail.com
1	Praveen	54	xxxxxx@gmail.com
2	Jagroop	29	xxxxxx@gmail.com
3	Rahul	36	xx@gmail.com
4	Vishal	59	xxxx@gmail.com
5	Suraj	38	xxxxx@gmail.com
6	Rishab	12	xxxxx@gmail.com
7	Satyapal	45	xxxxx@gmail.com
8	Amit	34	xxxxx@gmail.com

Reshaping data sets

Python has operations for rearranging tabular data, known as reshaping or pivoting operations. For example, hierarchical indexing provides a consistent way to rearrange data in a DataFrame. There are two primary functions in hierarchical indexing:

- **stack():** rotates or pivots data from columns to rows
- **unstack():** pivots data from rows to columns

Here is the syntax for both the functions:

DataFrame.stack(level=- 1, dropna=True)

DataFrame.unstack(level=- 1, fill_value=None)
Let's try these operations with some examples. Use these code snippets:
First, create a dummy DataFrame.
Code:

```
data = pd.DataFrame(np.arange(6).reshape((2,3)), index=pd.Index(['Victoria', 'NSW'], name='state'), columns=pd.Index(['one','two','three'], name='number'))data
```

Output:

number	one	two	three
state			
Victoria	0	1	2
NSW	3	4	5

Next, we use the stack() function, we will pivot the columns into rows
Code:

```
data_stack = data.stack()data_stack
```

Output:

```
state     number
Victoria  one       0
          two       1
          three     2
NSW       one       3
          two       4
          three     5
dtype: int32
```

You can see that:

- the operation converted the columns to row labels
- the values now have hierarchical indexing (state and number)
- the operation converted the DataFrame to a series.

You can confirm these changes with this code:

type(data_stack)
Output:
pandas.core.series.Series

data_stack.index
Output:

```
MultiIndex(levels=[['Victoria', 'NSW'], ['one', 'two', 'three']],
 labels=[[0, 0, 0, 1, 1, 1], [0, 1, 2, 0, 1, 2]], names=['state', 'number'])
```

From a hierarchically indexed series, you can rearrange the data back into a DataFrame with the unstack() function.
Try this code:

data = data_stack.unstack()data
Output:

number	one	two	three
state			
Victoria	0	1	2
NSW	3	4	5

By default, the innermost level is unstacked. In our example, it was a number. However, you can unstack a different level by passing a level number or name as a parameter to the unstack method.
For example, try this code that unstacks data_stack at the level of state, rather than number:
Code:

data_state = data_stack.unstack('state')data_state

Output:

state	Victoria	NSW
number		
one	0	3
two	1	4
three	2	5

Output:

	Name	Roll_no	Email
0	Amit	23	xxxx@gmail.com
1	Praveen	54	xxxxxx@gmail.com
2	Jagroop	29	xxxxxx@gmail.com
3	Rahul	36	xx@gmail.com
4	Vishal	59	xxxx@gmail.com
5	Suraj	38	xxxxx@gmail.com
6	Rishab	12	xxxxx@gmail.com
7	Satyapal	45	xxxxx@gmail.com
8	Amit	34	xxxxx@gmail.com

Data Aggregation

Python has several methods are available to perform aggregations on data. It is done using the pandas and numpy libraries. The data must be available or converted to a dataframe to apply the aggregation functions.

Applying Aggregations on DataFrame

Let us create a DataFrame and apply aggregations on it.

```
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randn(10, 4),
   index = pd.date_range('1/1/2000', periods=10),
   columns = ['A', 'B', 'C', 'D'])

print df

r = df.rolling(window=3,min_periods=1)
print r
```

Its **output** is as follows –

```
                A         B         C         D
2000-01-01  1.088512  -0.650942  -2.547450  -0.566858
2000-01-02  0.790670  -0.387854  -0.668132   0.267283
2000-01-03 -0.575523  -0.965025   0.060427  -2.179780
2000-01-04  1.669653   1.211759  -0.254695   1.429166
2000-01-05  0.100568  -0.236184   0.491646  -0.466081
2000-01-06  0.155172   0.992975  -1.205134   0.320958
2000-01-07  0.309468  -0.724053  -1.412446   0.627919
2000-01-08  0.099489  -1.028040   0.163206  -1.274331
2000-01-09  1.639500  -0.068443   0.714008  -0.565969
2000-01-10  0.326761   1.479841   0.664282  -1.361169
```

Rolling [window=3,min_periods=1,center=False,axis=0]

We can aggregate by passing a function to the entire DataFrame, or select a column via the standard **get item** method.

Apply Aggregation on a Whole Dataframe

```
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randn(10, 4),
   index = pd.date_range('1/1/2000', periods=10),
   columns = ['A', 'B', 'C', 'D'])
print df

r = df.rolling(window=3,min_periods=1)
print r.aggregate(np.sum)
```

Its **output** is as follows –

```
                A         B         C         D
2000-01-01  1.088512  -0.650942  -2.547450  -0.566858
2000-01-02  1.879182  -1.038796  -3.215581  -0.299575
2000-01-03  1.303660  -2.003821  -3.155154  -2.479355
2000-01-04  1.884801  -0.141119  -0.862400  -0.483331
2000-01-05  1.194699   0.010551   0.297378  -1.216695
2000-01-06  1.925393   1.968551  -0.968183   1.284044
2000-01-07  0.565208   0.032738  -2.125934   0.482797
2000-01-08  0.564129  -0.759118  -2.454374  -0.325454
2000-01-09  2.048458  -1.820537  -0.535232  -1.212381
2000-01-10  2.065750   0.383357   1.541496  -3.201469
```

```
                A         B         C         D
2000-01-01  1.088512  -0.650942  -2.547450  -0.566858
```

```
2000-01-02  1.879182  -1.038796  -3.215581  -0.299575
2000-01-03  1.303660  -2.003821  -3.155154  -2.479355
2000-01-04  1.884801  -0.141119  -0.862400  -0.483331
2000-01-05  1.194699   0.010551   0.297378  -1.216695
2000-01-06  1.925393   1.968551  -0.968183   1.284044
2000-01-07  0.565208   0.032738  -2.125934   0.482797
2000-01-08  0.564129  -0.759118  -2.454374  -0.325454
2000-01-09  2.048458  -1.820537  -0.535232  -1.212381
2000-01-10  2.065750   0.383357   1.541496  -3.201469
```

Apply Aggregation on a Single Column of a Dataframe

```
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randn(10, 4),
   index = pd.date_range('1/1/2000', periods=10),
   columns = ['A', 'B', 'C', 'D'])
print df
r = df.rolling(window=3,min_periods=1)
print r['A'].aggregate(np.sum)
```

Its **output** is as follows –

```
                   A         B         C         D
2000-01-01  1.088512  -0.650942  -2.547450  -0.566858
2000-01-02  1.879182  -1.038796  -3.215581  -0.299575
2000-01-03  1.303660  -2.003821  -3.155154  -2.479355
2000-01-04  1.884801  -0.141119  -0.862400  -0.483331
2000-01-05  1.194699   0.010551   0.297378  -1.216695
2000-01-06  1.925393   1.968551  -0.968183   1.284044
2000-01-07  0.565208   0.032738  -2.125934   0.482797
2000-01-08  0.564129  -0.759118  -2.454374  -0.325454
2000-01-09  2.048458  -1.820537  -0.535232  -1.212381
2000-01-10  2.065750   0.383357   1.541496  -3.201469
2000-01-01  1.088512
2000-01-02  1.879182
2000-01-03  1.303660
2000-01-04  1.884801
2000-01-05  1.194699
2000-01-06  1.925393
2000-01-07  0.565208
2000-01-08  0.564129
2000-01-09  2.048458
2000-01-10  2.065750
Freq: D, Name: A, dtype: float64
```

Apply Aggregation on Multiple Columns of a DataFrame

```
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randn(10, 4),
   index = pd.date_range('1/1/2000', periods=10),
   columns = ['A', 'B', 'C', 'D'])
print df
r = df.rolling(window=3,min_periods=1)
print r[['A','B']].aggregate(np.sum)
```

Its **output** is as follows –

```
                   A         B         C         D
2000-01-01  1.088512  -0.650942  -2.547450  -0.566858
2000-01-02  1.879182  -1.038796  -3.215581  -0.299575
2000-01-03  1.303660  -2.003821  -3.155154  -2.479355
2000-01-04  1.884801  -0.141119  -0.862400  -0.483331
2000-01-05  1.194699   0.010551   0.297378  -1.216695
2000-01-06  1.925393   1.968551  -0.968183   1.284044
2000-01-07  0.565208   0.032738  -2.125934   0.482797
2000-01-08  0.564129  -0.759118  -2.454374  -0.325454
2000-01-09  2.048458  -1.820537  -0.535232  -1.212381
2000-01-10  2.065750   0.383357   1.541496  -3.201469
                   A         B
2000-01-01  1.088512  -0.650942
2000-01-02  1.879182  -1.038796
2000-01-03  1.303660  -2.003821
2000-01-04  1.884801  -0.141119
2000-01-05  1.194699   0.010551
2000-01-06  1.925393   1.968551
2000-01-07  0.565208   0.032738
2000-01-08  0.564129  -0.759118
2000-01-09  2.048458  -1.820537
2000-01-10  2.065750   0.383357
```

Unit-6

Data Visualization: The matplotlib API primer-Line properties, Figures and subplots, Exploring plot types-

Scatter plots, Bar plots, Histogram plots, Legends and annotations, Plotting functions with Pandas

Matplotlib

Matplotlib is a low level graph plotting library in python that serves as a visualization utility.

Matplotlib was created by John D. Hunter.

Matplotlib is open source and we can use it freely.

Matplotlib is mostly written in python, a few segments are written in C, Objective-C and Javascript for Platform compatibility.

Pyplot

Most of the Matplotlib utilities lies under the pyplot submodule, and are usually imported under the plt alias:

```
import matplotlib.pyplot as plt
```

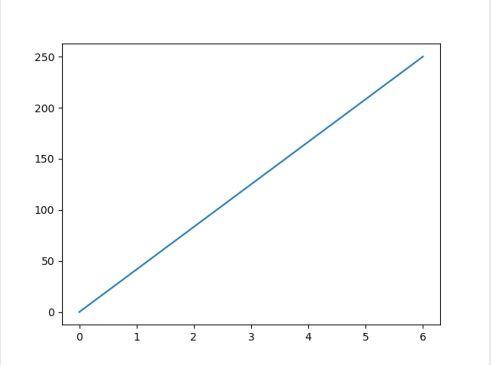
Now the Pyplot package can be referred to as plt.

Example

Draw a line in a diagram from position (0,0) to position (6,250):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([0, 6])
ypoints = np.array([0, 250])

plt.plot(xpoints, ypoints)
plt.show()
```

Result:



Matplotlib Plotting

Plotting x and y points

The plot() function is used to draw points (markers) in a diagram.

By default, the plot() function draws a line from point to point.

The function takes parameters for specifying points in the diagram.

Parameter 1 is an array containing the points on the **x-axis**.

Parameter 2 is an array containing the points on the **y-axis**.

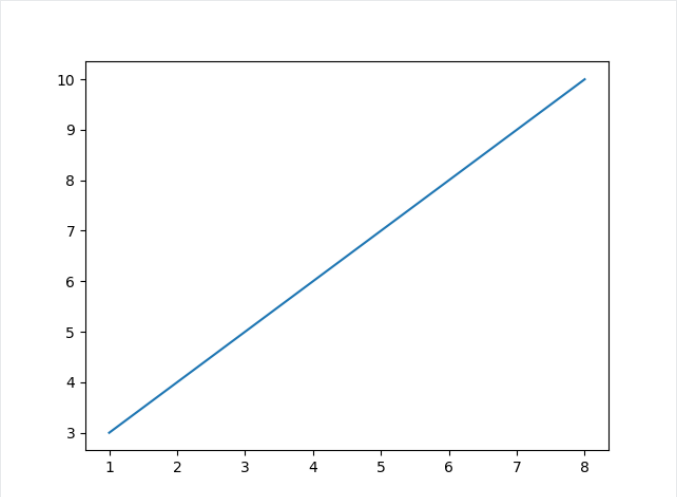
If we need to plot a line from (1, 3) to (8, 10), we have to pass two arrays [1, 8] and [3, 10] to the plot function.

Example

Draw a line in a diagram from position (1, 3) to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints)
plt.show()
```

Result:



The **x-axis** is the horizontal axis.

The **y-axis** is the vertical axis.

Plotting Without Line

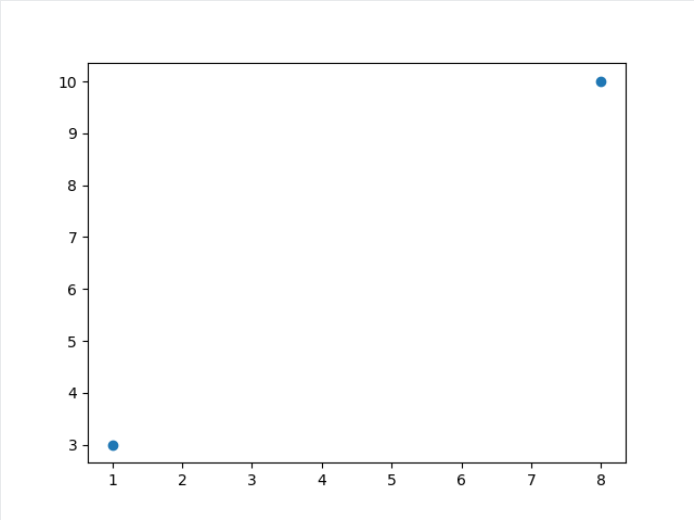
To plot only the markers, you can use *shortcut string notation* parameter 'o', which means 'rings'.

Example

Draw two points in the diagram, one at position (1, 3) and one in position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints, 'o')
plt.show()
```

Result:



Multiple Points

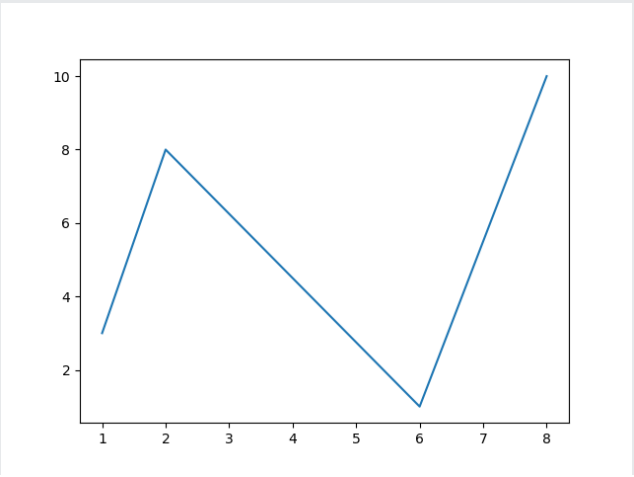
You can plot as many points as you like, just make sure you have the same number of points in both axis.

Example

Draw a line in a diagram from position (1, 3) to (2, 8) then to (6, 1) and finally to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 2, 6, 8])
ypoints = np.array([3, 8, 1, 10])

plt.plot(xpoints, ypoints)
plt.show()
```

Result:



Default X-Points

If we do not specify the points in the x-axis, they will get the default values 0, 1, 2, 3, (etc. depending on the length of the y-points.

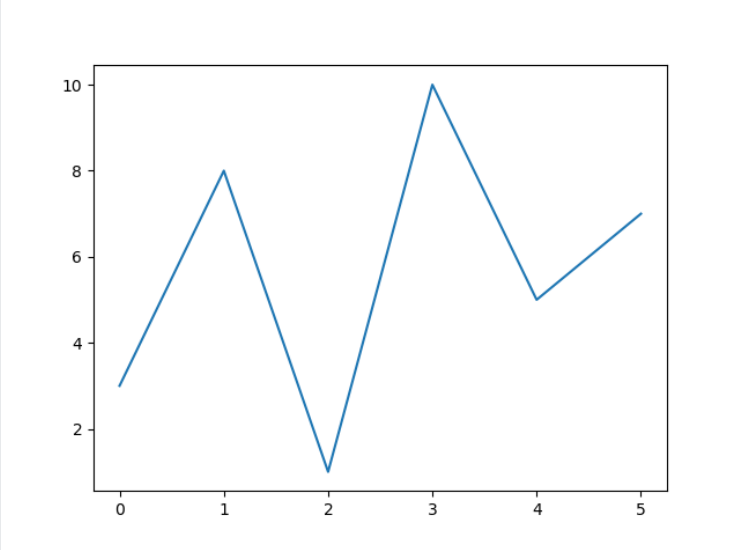
Example

Plotting without x-points:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10, 5, 7])

plt.plot(ypoints)
plt.show()
```

Result:



Matplotlib Line

Linestyle

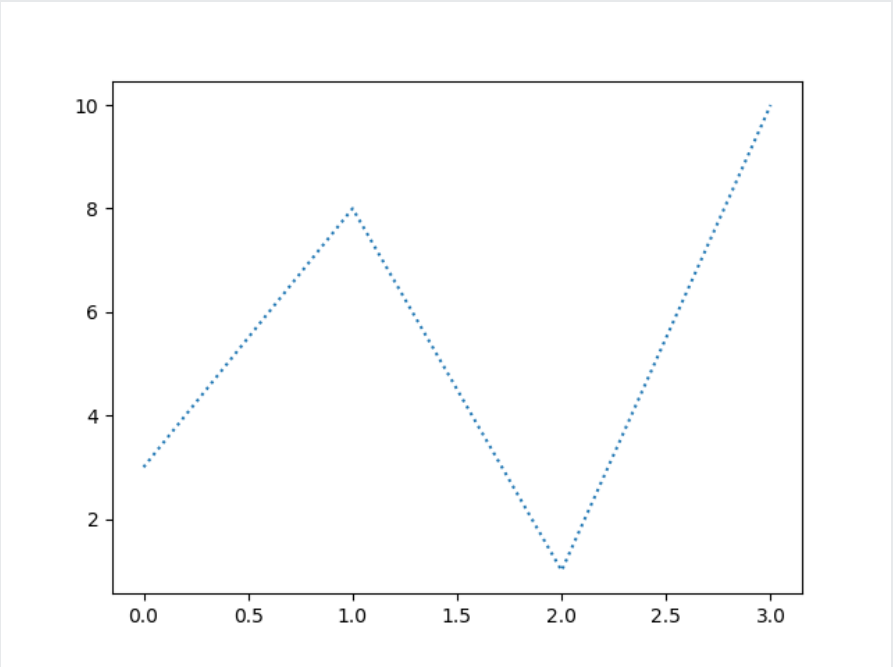
You can use the keyword argument linestyle, or shorter ls, to change the style of the plotted line:

Example

Use a dotted line:

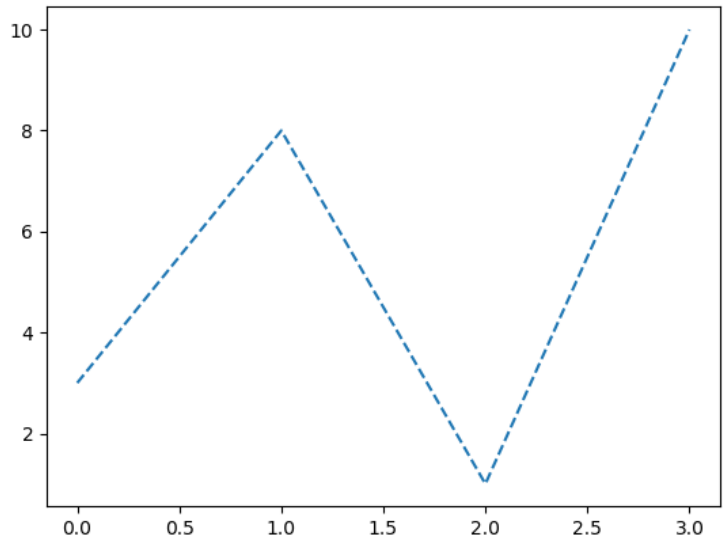
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, linestyle = 'dotted')
plt.show()
```

Result:



Use a dashed line:

```
plt.plot(ypoints, linestyle = 'dashed')
```



Matplotlib Subplots

Display Multiple Plots

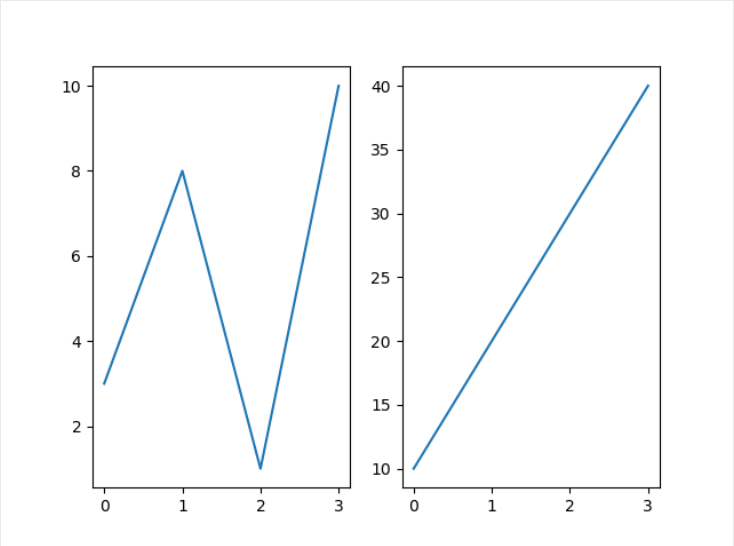
With the subplots() function you can draw multiple plots in one figure:

Example

Draw 2 plots:

```
import matplotlib.pyplot as plt
import numpy as np

#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(1, 2, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(1, 2, 2)
plt.plot(x,y)

plt.show()
```

Result:



The subplots() Function

The subplots() function takes three arguments that describes the layout of the figure.

The layout is organized in rows and columns, which are represented by the *first* and *second* argument.

The third argument represents the index of the current plot.

```
plt.subplot(1, 2, 1)
#the figure has 1 row, 2 columns, and this plot is the first plot.
```

```
plt.subplot(1, 2, 2)
#the figure has 1 row, 2 columns, and this plot is the second plot.
```

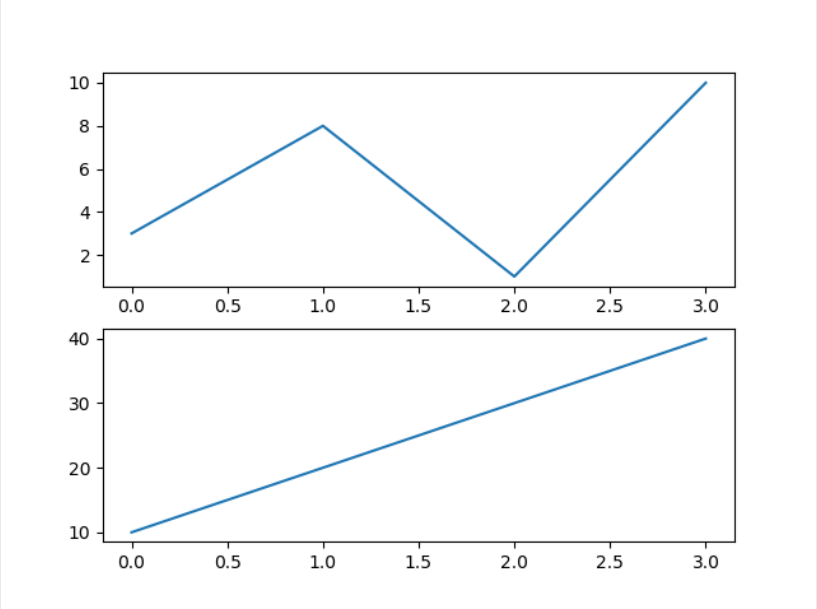
So, if we want a figure with 2 rows an 1 column (meaning that the two plots will be displayed on top of each other instead of side-by-side), we can write the syntax like this:

Example

Draw 2 plots on top of each other:

```
import matplotlib.pyplot as plt
import numpy as np

#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 1, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 1, 2)
plt.plot(x,y)

plt.show()
```

Result:



You can draw as many plots you like on one figure, just descibe the number of rows, columns, and the index of the plot.

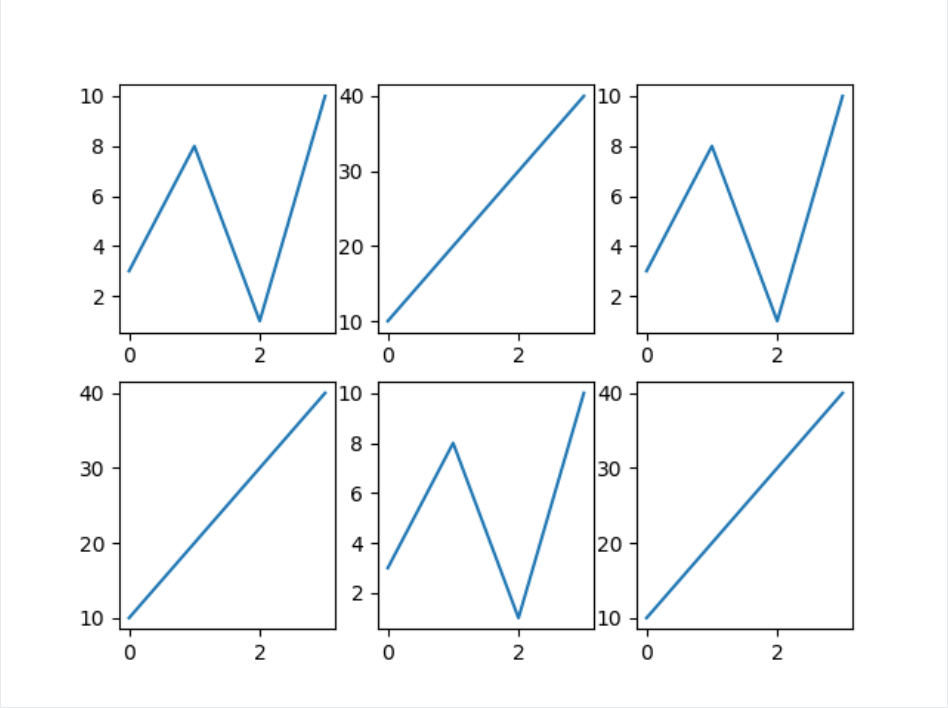
Example

Draw 6 plots:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 3, 1)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 3, 2)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
```

```
plt.subplot(2, 3, 3)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 3, 4)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 3, 5)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 3, 6)
plt.plot(x,y)

plt.show()
```

Result:



Matplotlib Scatter

Creating Scatter Plots

With Pyplot, you can use the scatter() function to draw a scatter plot.

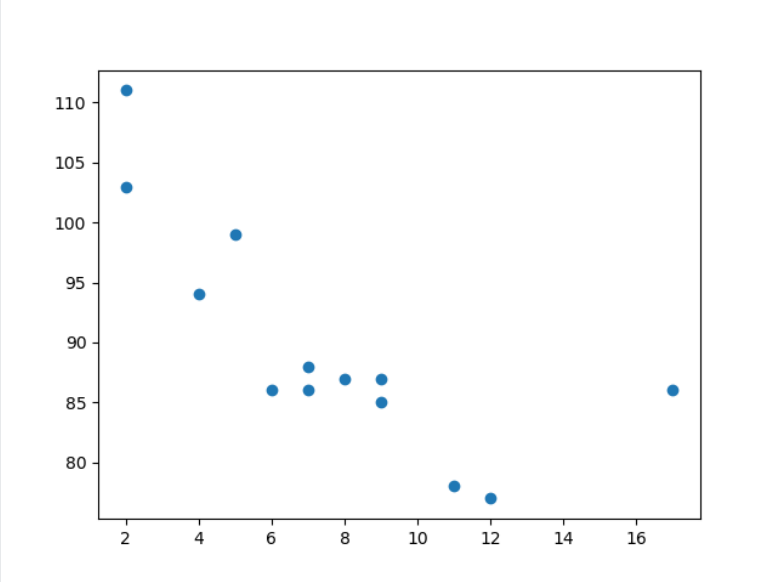
The scatter() function plots one dot for each observation. It needs two arrays of the same length, one for the values of the x-axis, and one for values on the y-axis:

Example

A simple scatter plot:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x, y)
plt.show()
```

Result:



The observation in the example above is the result of 13 cars passing by.

The X-axis shows how old the car is.

The Y-axis shows the speed of the car when it passes.

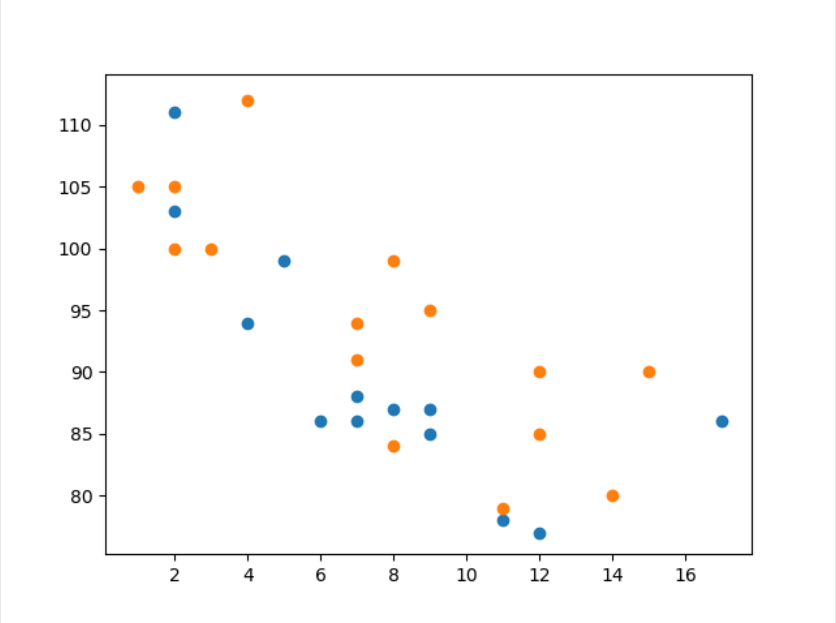
Compare Plots

Example

Draw two plots on the same figure:

```
import matplotlib.pyplot as plt
import numpy as np

#day one, the age and speed of 13 cars:
x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
plt.scatter(x, y)

#day two, the age and speed of 15 cars:
x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12])
y = np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85])
plt.scatter(x, y)

plt.show()
```

Result:



By comparing the two plots, I think it is safe to say that they both gives us the same conclusion: the newer the car, the faster it drives.

Colors

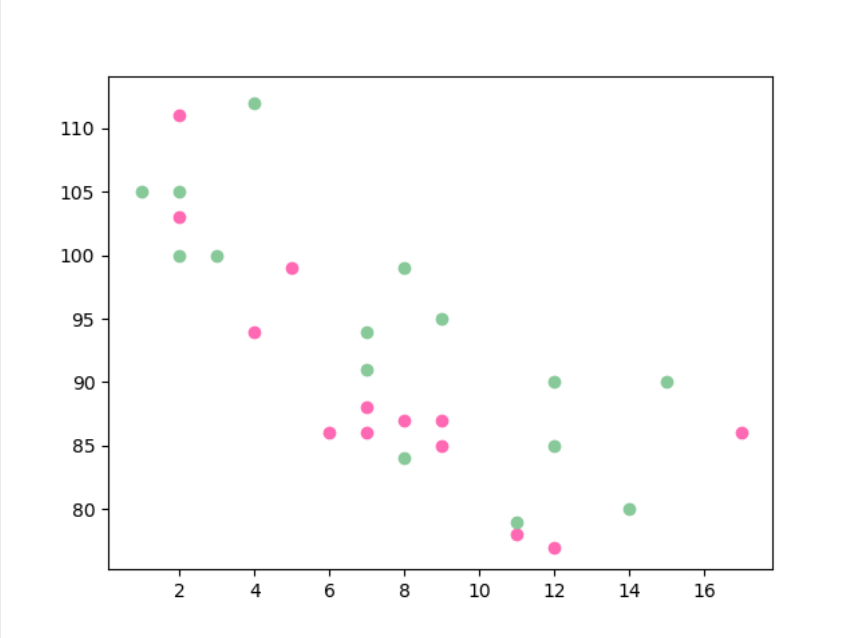
You can set your own color for each scatter plot with the color or the c argument:

Example

Set your own color of the markers:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
plt.scatter(x, y, color = 'hotpink')

x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12])
y = np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85])
plt.scatter(x, y, color = '#88c999')

plt.show()
```

Result:



Color Each Dot

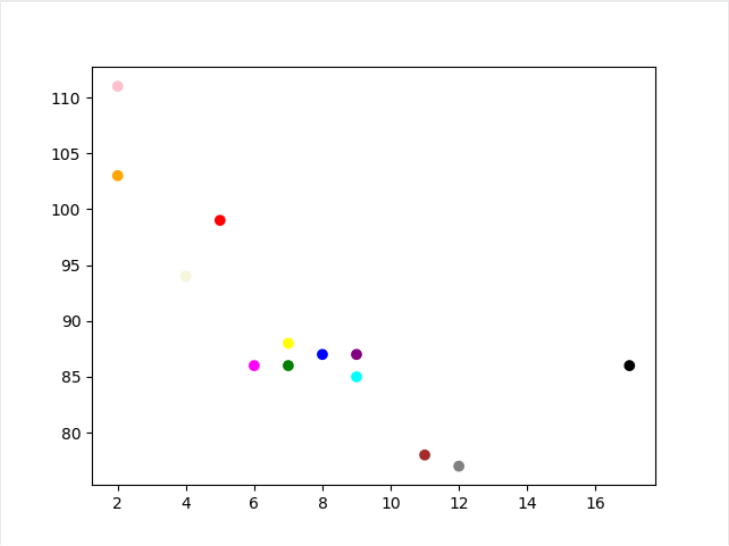
You can even set a specific color for each dot by using an array of colors as value for the c argument:

Example

Set your own color of the markers:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
colors =
np.array(["red","green","blue","yellow","pink","black","orange","purple","beige","brown","gray","cyan","magenta"])

plt.scatter(x, y, c=colors)

plt.show()
```

Result:



ColorMap

The Matplotlib module has a number of available colormaps.

A colormap is like a list of colors, where each color has a value that ranges from 0 to 100.

Here is an example of a colormap:

This colormap is called 'viridis' and as you can see it ranges from 0, which is a purple color, and up to 100, which is a yellow color.

How to Use the ColorMap

You can specify the colormap with the keyword argument cmap with the value of the colormap, in this case 'viridis' which is one of the built-in colormaps available in Matplotlib.

In addition you have to create an array with values (from 0 to 100), one value for each of the point in the scatter plot:

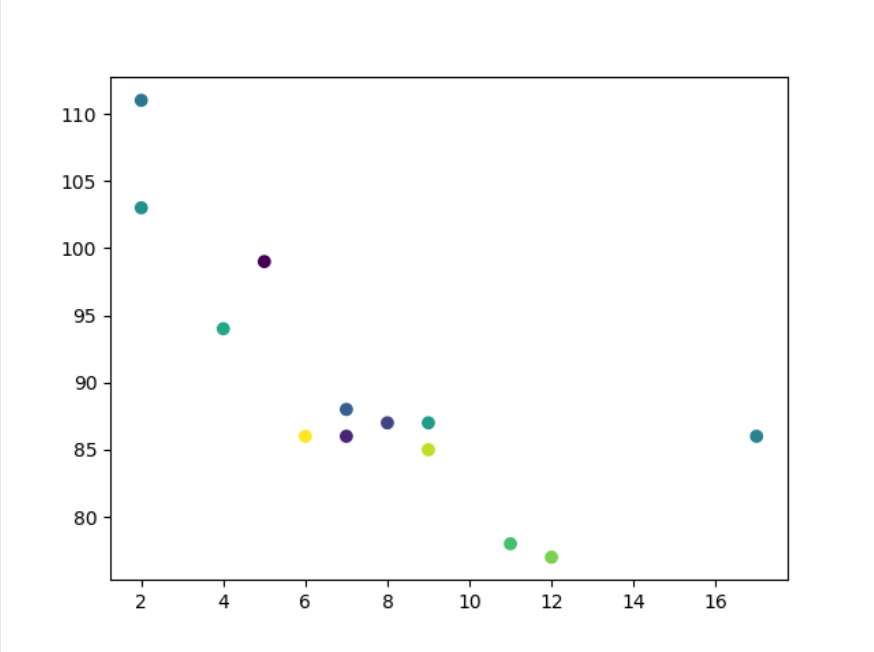
Example

Create a color array, and specify a colormap in the scatter plot:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
colors = np.array([0, 10, 20, 30, 40, 45, 50, 55, 60, 70, 80, 90, 100])

plt.scatter(x, y, c=colors, cmap='viridis')
```

```
plt.show()
```

Result:



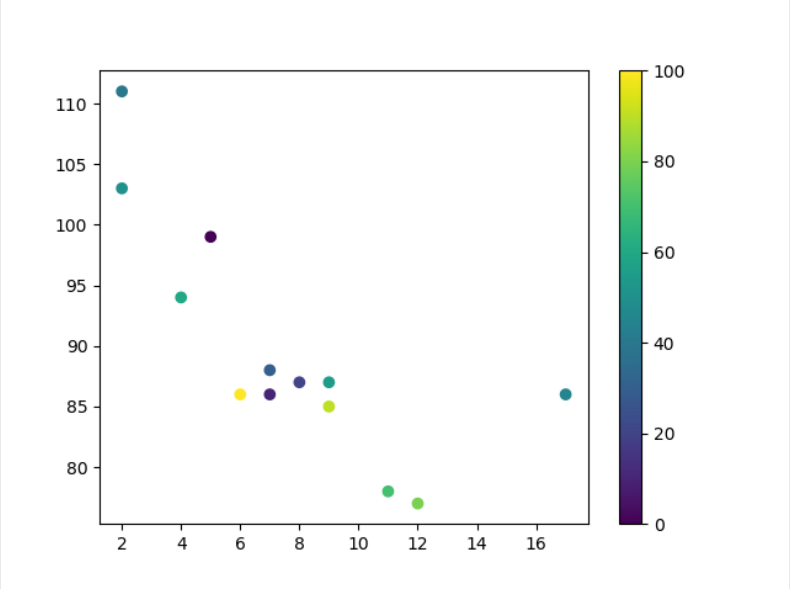
You can include the colormap in the drawing by including the plt.colorbar() statement:

Example

Include the actual colormap:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
colors = np.array([0, 10, 20, 30, 40, 45, 50, 55, 60, 70, 80, 90, 100])

plt.scatter(x, y, c=colors, cmap='viridis')

plt.colorbar()

plt.show()
```

Result:



Matplotlib Bars

Creating Bars

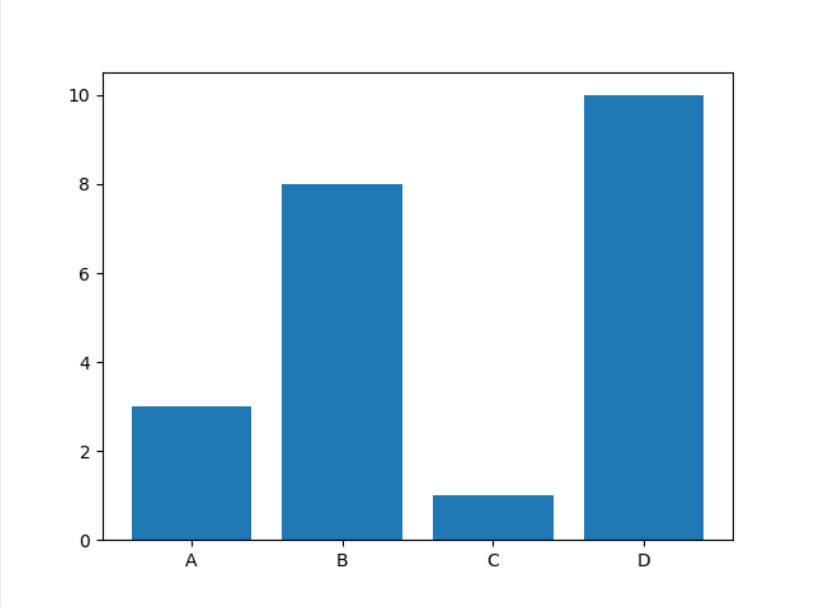
With Pyplot, you can use the bar() function to draw bar graphs:

Example

Draw 4 bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y)
plt.show()
```

Result:



The bar() function takes arguments that describes the layout of the bars.

The categories and their values represented by the *first* and *second* argument as arrays.

Example

```
x = ["APPLES", "BANANAS"]
y = [400, 350]
plt.bar(x, y)
```

Horizontal Bars

If you want the bars to be displayed horizontally instead of vertically, use the barh() function:

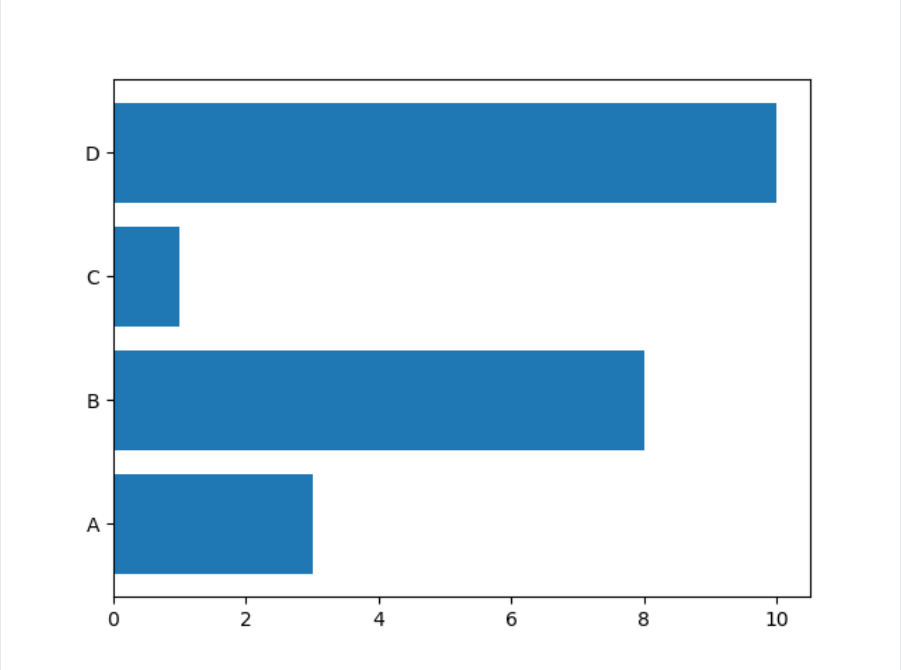
Example

Draw 4 horizontal bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])
```

```
plt.barh(x, y)
plt.show()
```

Result:



Bar Color

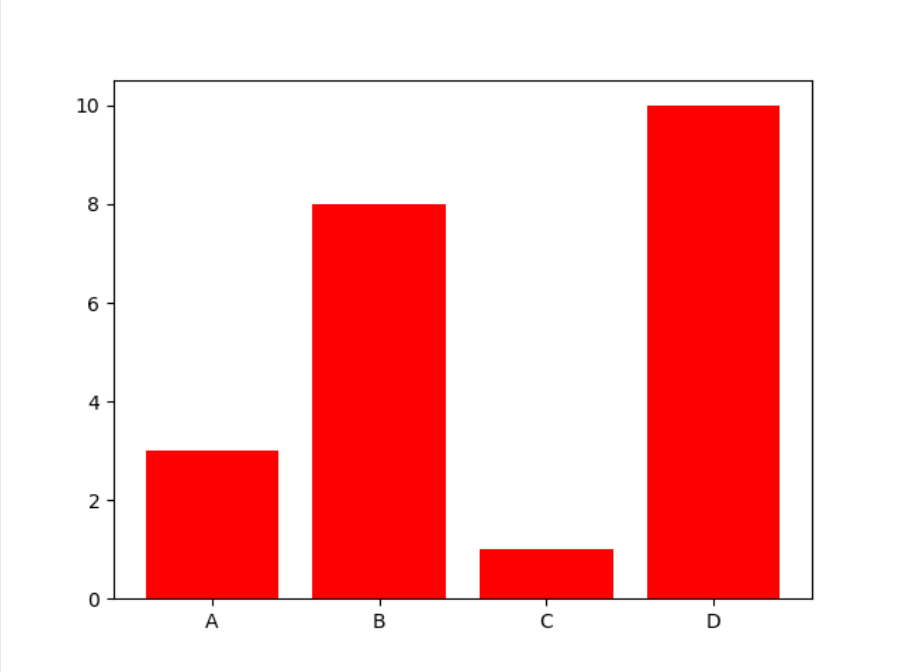
The bar() and barh() takes the keyword argument color to set the color of the bars:

Example

Draw 4 red bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x, y, color = "red")
plt.show()
```

Result:



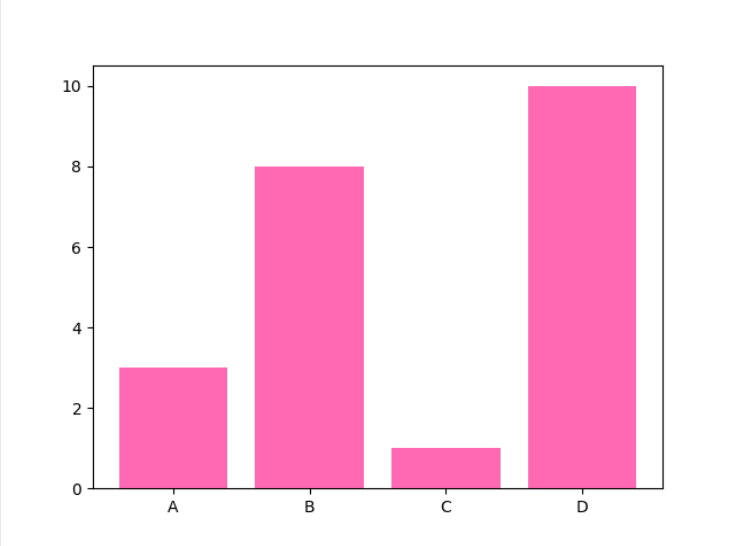
Color Names

Example

Draw 4 "hot pink" bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x, y, color = "hotpink")
plt.show()
```

Result:



Color Hex

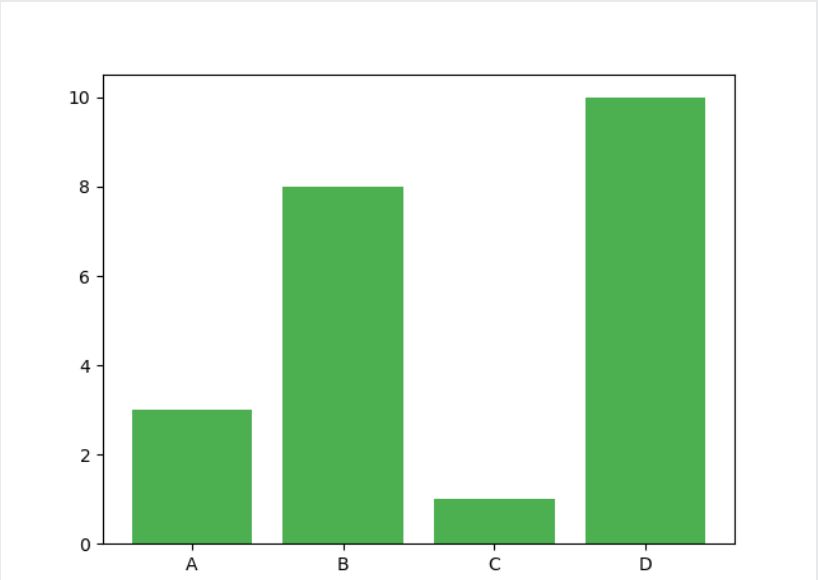
Or you can use Hexadecimal color values:

Example

Draw 4 bars with a beautiful green color:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x, y, color = "#4CAF50")
plt.show()
```

Result:



Bar Width

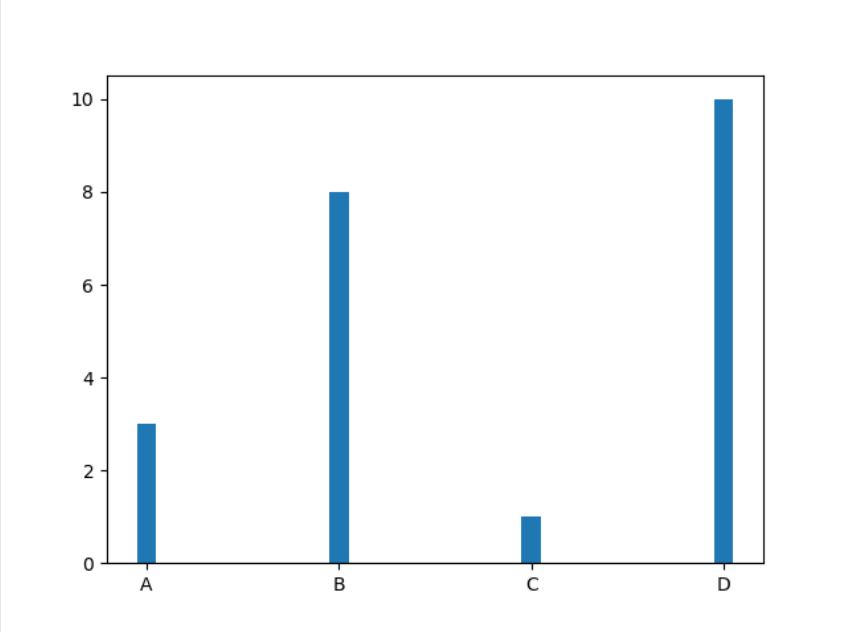
The bar() takes the keyword argument width to set the width of the bars:

Example

Draw 4 very thin bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x, y, width = 0.1)
plt.show()
```

Result:



The default width value is 0.8

Bar Height

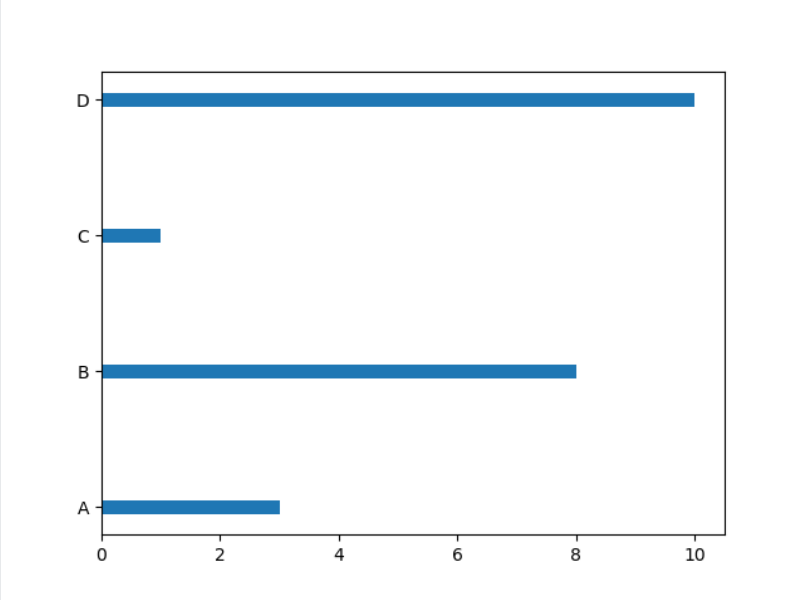
The barh() takes the keyword argument height to set the height of the bars:

Example

Draw 4 very thin bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.barh(x, y, height = 0.1)
plt.show()
```

Result:



The default height value is 0.8

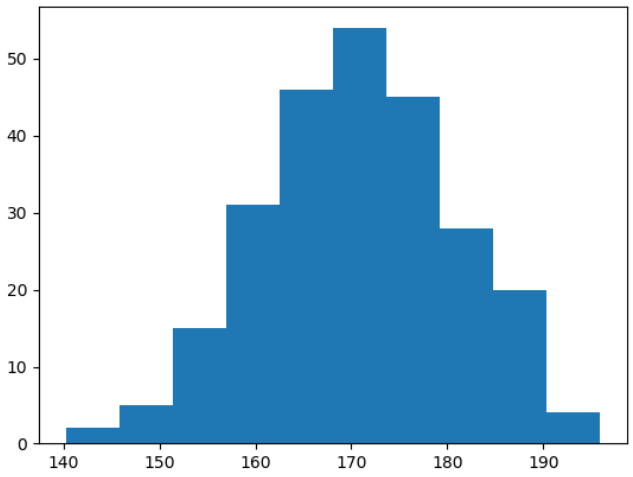
Matplotlib Histograms

Histogram

A histogram is a graph showing *frequency* distributions.

It is a graph showing the number of observations within each given interval.

Example: Say you ask for the height of 250 people, you might end up with a histogram like this:



You can read from the histogram that there are approximately:

2 people from 140 to 145cm
5 people from 145 to 150cm
15 people from 151 to 156cm
31 people from 157 to 162cm
46 people from 163 to 168cm
53 people from 168 to 173cm
45 people from 173 to 178cm
28 people from 179 to 184cm
21 people from 185 to 190cm
4 people from 190 to 195cm

Create Histogram

In Matplotlib, we use the hist() function to create histograms.

The hist() function will use an array of numbers to create a histogram, the array is sent into the function as an argument.

For simplicity we use NumPy to randomly generate an array with 250 values, where the values will concentrate around 170, and the standard deviation is 10. Learn more about Normal Data Distribution in our Machine Learning Tutorial.

Example

A Normal Data Distribution by NumPy:

```
import numpy as np

x = np.random.normal(170, 10, 250)

print(x)
```

Result:

This will generate a *random* result, and could look like this:

```
[167.62255766 175.32495609 152.84661337 165.50264047 163.17457988
 162.29867872 172.83638413 168.67303667 164.57361342 180.81120541
 170.57782187 167.53075749 176.15356275 176.95378312 158.4125473
 187.8842668  159.03730075 166.69284332 160.73882029 152.22378865
 164.01255164 163.95288674 176.58146832 173.19849526 169.40206527
 166.88861903 149.90348576 148.39039643 177.90349066 166.72462233
```

```
177.44776004 170.93335636 173.26312881 174.76534435 162.28791953
166.77301551 160.53785202 170.67972019 159.11594186 165.36992993
178.38979253 171.52158489 173.32636678 159.63894401 151.95735707
175.71274153 165.00458544 164.80607211 177.50988211 149.28106703
179.43586267 181.98365273 170.98196794 179.1093176  176.91855744
168.32092784 162.33939782 165.18364866 160.52300507 174.14316386
163.01947601 172.01767945 173.33491959 169.75842718 198.04834503
192.82490521 164.54557943 206.36247244 165.47748898 195.26377975
164.37569092 156.15175531 162.15564208 179.34100362 167.22138242
147.23667125 162.86940215 167.84986671 172.99302505 166.77279814
196.6137667  159.79012341 166.5840824  170.68645637 165.62204521
174.5559345  165.0079216  187.92545129 166.86186393 179.78383824
161.0973573  167.44890343 157.38075812 151.35412246 171.3107829
162.57149341 182.49985133 163.24700057 168.72639903 169.05309467
167.19232875 161.06405208 176.87667712 165.48750185 179.68799986
158.7913483  170.22465411 182.66432721 173.5675715  176.85646836
157.31299754 174.88959677 183.78323508 174.36814558 182.55474697
180.03359793 180.53094948 161.09560099 172.29179934 161.22665588
171.88382477 159.04626132 169.43886536 163.75793589 157.73710983
174.68921523 176.19843414 167.39315397 181.17128255 174.2674597
186.05053154 177.06516302 171.78523683 166.14875436 163.31607668
174.01429569 194.98819875 169.75129209 164.25748789 180.25773528
170.44784934 157.81966006 171.33315907 174.71390637 160.55423274
163.92896899 177.29159542 168.30674234 165.42853878 176.46256226
162.61719142 166.60810831 165.83648812 184.83238352 188.99833856
161.3054697  175.30396693 175.28109026 171.54765201 162.08762813
164.53011089 189.86213299 170.83784593 163.25869004 198.68079225
166.95154328 152.03381334 152.25444225 149.75522816 161.79200594
162.13535052 183.37298831 165.40405341 155.59224806 172.68678385
179.35359654 174.19668349 163.46176882 168.26621173 162.97527574
192.80170974 151.29673582 178.65251432 163.17266558 165.11172588
183.11107905 169.69556831 166.35149789 178.74419135 166.28562032
169.96465166 178.24368042 175.3035525  170.16496554 158.80682882
187.10006553 178.90542991 171.65790645 183.19289193 168.17446717
155.84544031 177.96091745 186.28887898 187.89867406 163.26716924
169.71242393 152.9410412  158.68101969 171.12655559 178.1482624
187.45272185 173.02872935 163.8047623  169.95676819 179.36887054
157.01955088 185.58143864 170.19037101 157.221245   168.90639755
178.7045601  168.64074373 172.37416382 165.61890535 163.40873027
168.98683006 149.48186389 172.20815568 172.82947206 173.71584064
189.42642762 172.79575803 177.00005573 169.24498561 171.55576698
161.36400372 176.47928342 163.02642822 165.09656415 186.70951892
153.27990317 165.59289527 180.34566865 189.19506385 183.10723435
173.48070474 170.28701875 157.24642079 157.9096498  176.4248199 ]
```

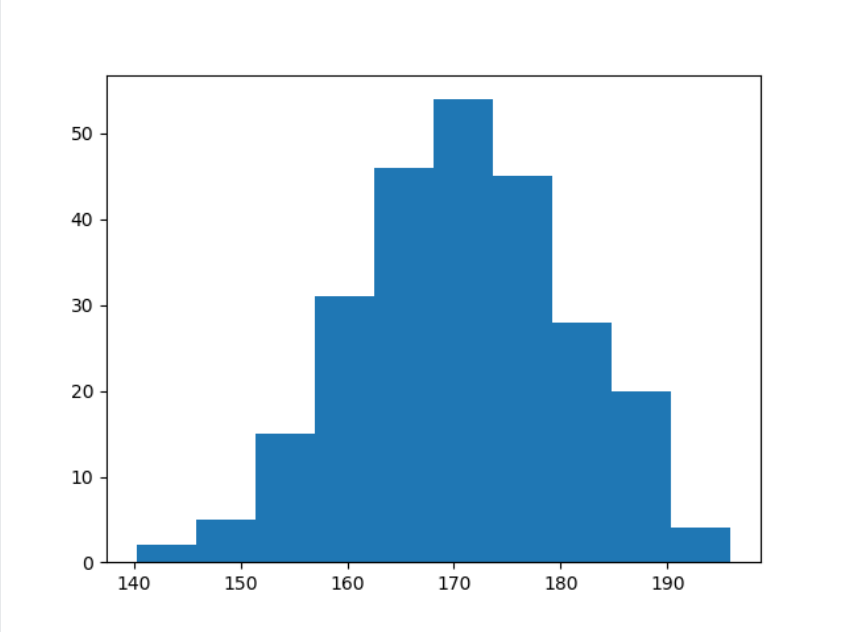
The hist() function will read the array and produce a histogram:

Example

A simple histogram:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.normal(170, 10, 250)

plt.hist(x)
plt.show()
```

Result:



Matplotlib Pie Charts

Creating Pie Charts

With Pyplot, you can use the pie() function to draw pie charts:

Example

A simple pie chart:

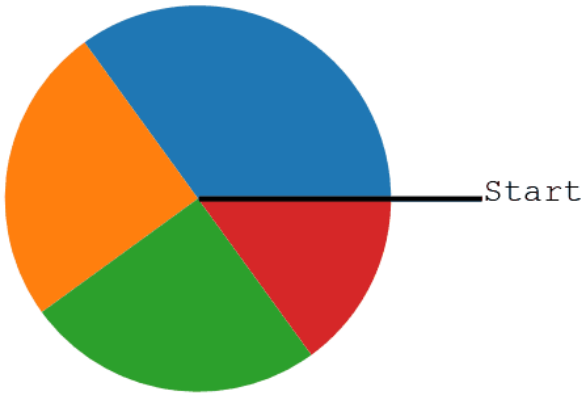
```
import matplotlib.pyplot as plt
import numpy as np
```

```
y = np.array([35, 25, 25, 15])

plt.pie(y)
plt.show()
```

Result:

As you can see the pie chart draws one piece (called a wedge) for each value in the array (in this case [35, 25, 25, 15]).

By default the plotting of the first wedge starts from the x-axis and move *counterclockwise*:



Labels

Add labels to the pie chart with the label parameter.

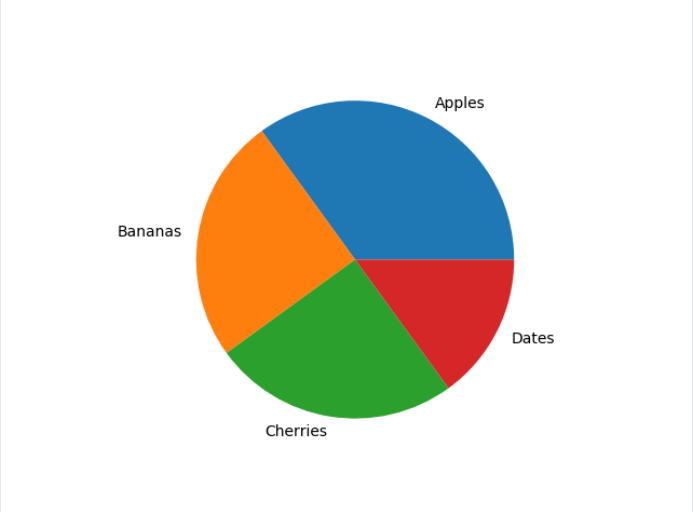
The label parameter must be an array with one label for each wedge:

Example

A simple pie chart:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.show()
```

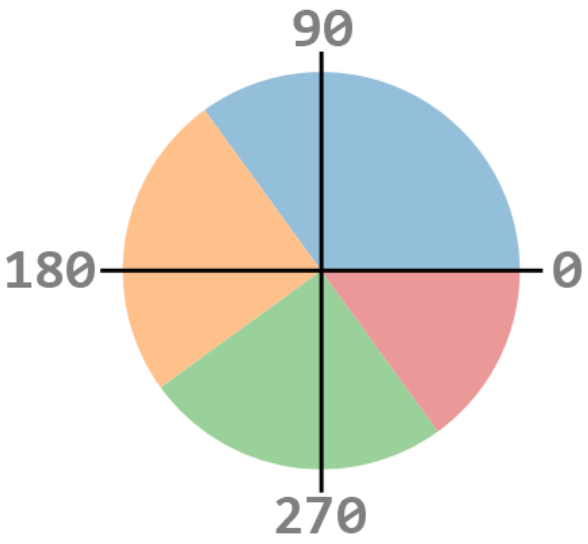
Result:



Start Angle

As mentioned the default start angle is at the x-axis, but you can change the start angle by specifying a startangle parameter.

The startangle parameter is defined with an angle in degrees, default angle is 0:

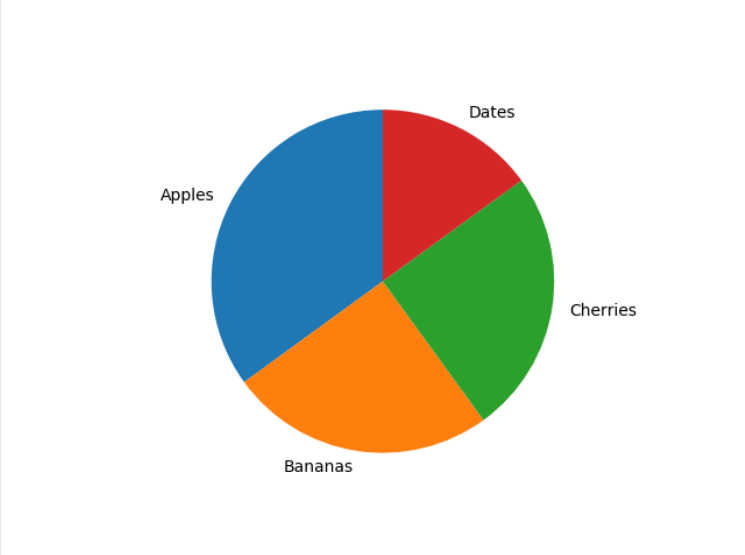


Example

Start the first wedge at 90 degrees:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels, startangle = 90)
plt.show()
```

Result:



Explode

Maybe you want one of the wedges to stand out? The explode parameter allows you to do that.

The explode parameter, if specified, and not None, must be an array with one value for each wedge.

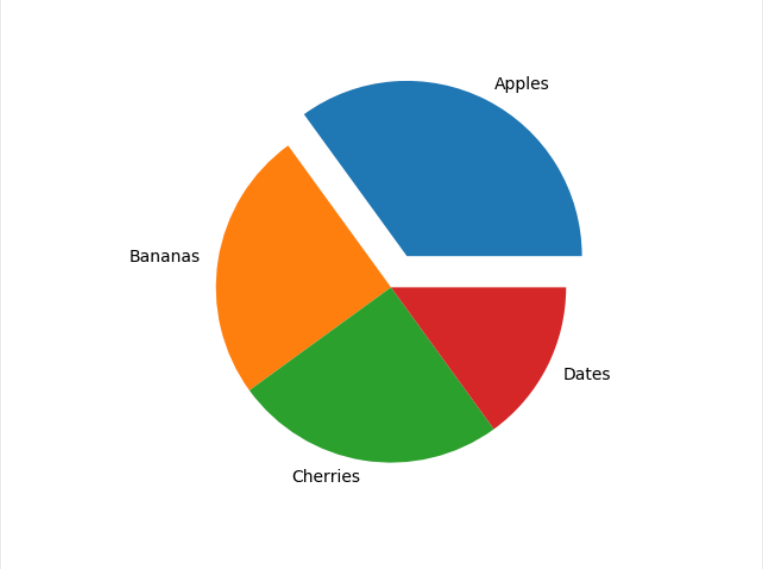
Each value represents how far from the center each wedge is displayed:

Example

Pull the "Apples" wedge 0.2 from the center of the pie:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0.2, 0, 0, 0]

plt.pie(y, labels = mylabels, explode = myexplode)
plt.show()
```

Result:



Shadow

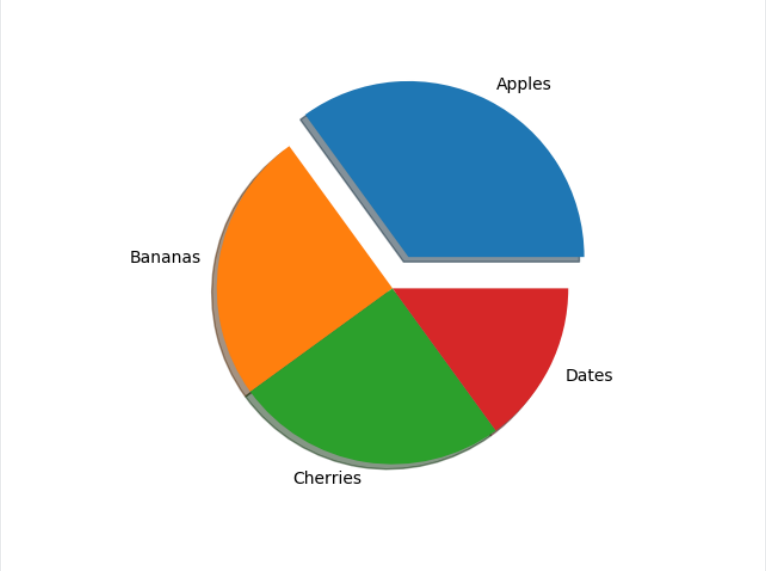
Add a shadow to the pie chart by setting the shadows parameter to True:

Example

Add a shadow:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0.2, 0, 0, 0]

plt.pie(y, labels = mylabels, explode = myexplode, shadow = True)
plt.show()
```

Result:



Colors

You can set the color of each wedge with the colors parameter.

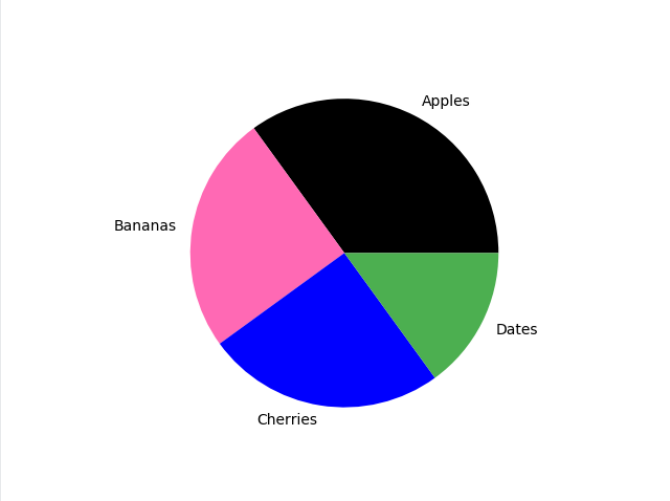
The colors parameter, if specified, must be an array with one value for each wedge:

Example

Specify a new color for each wedge:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
mycolors = ["black", "hotpink", "b", "#4CAF50"]

plt.pie(y, labels = mylabels, colors = mycolors)
plt.show()
```

Result:



You can use Hexadecimal color values, any of the 140 supported color names, or one of these shortcuts:

'r' - Red
'g' - Green
'b' - Blue
'c' - Cyan
'm' - Magenta
'y' - Yellow
'k' - Black
'w' - White

Legend

To add a list of explanation for each wedge, use the legend() function:

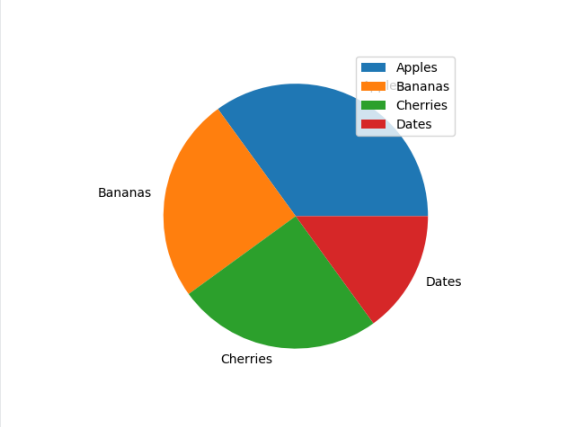
Example

Add a legend:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
```

```
plt.legend()
plt.show()
```

Result:



Legend With Header

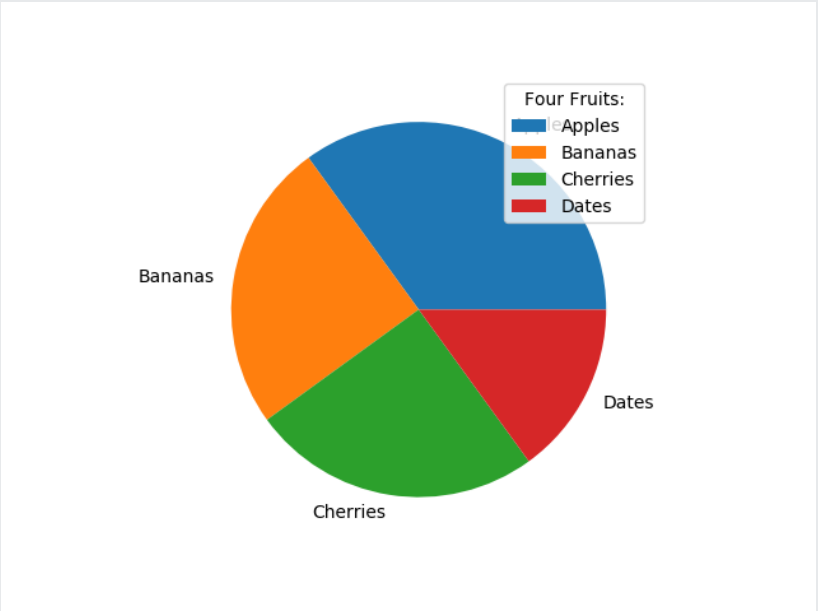
To add a header to the legend, add the title parameter to the legend function.

Example

Add a legend with a header:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.legend(title = "Four Fruits:")
plt.show()
```

Result:


Four Fruits:
Apples
Bananas
Cherries
Dates
Bananas
Dates
Cherries